
Rerank와 Modular RAG

1. LLM to RAG

목차 LLM to RAG

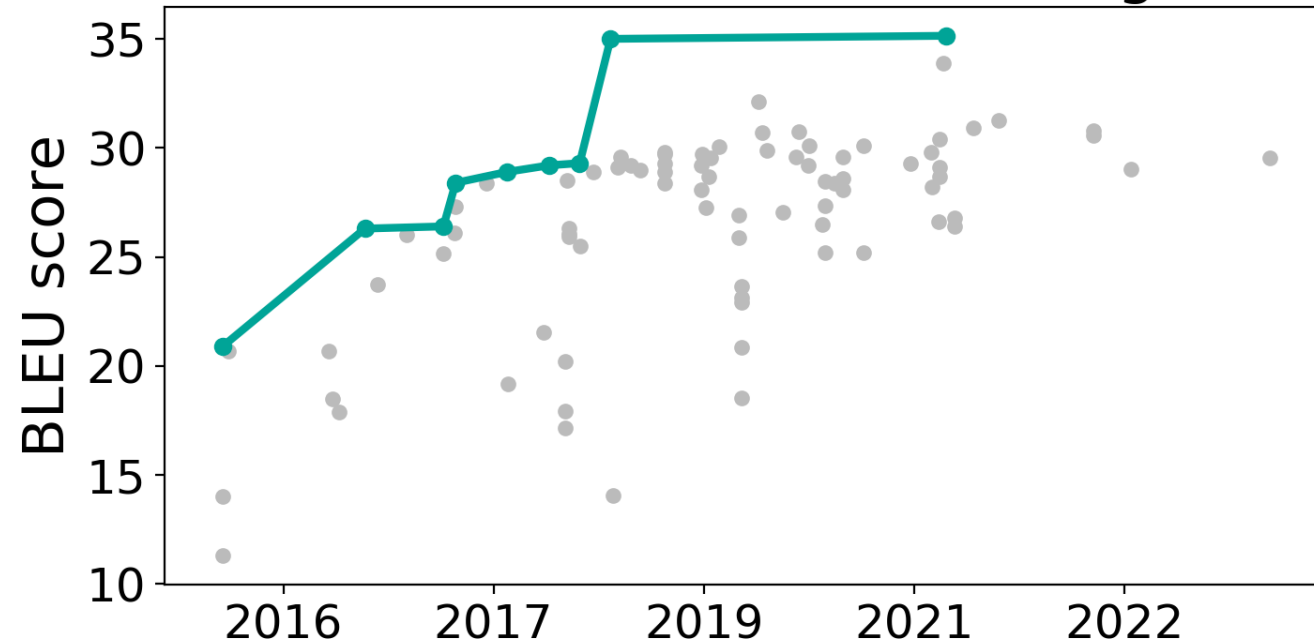
1. Transformer & LLM
2. LLM 활용의 한계와 Prompt Engineering
3. Naïve RAG와 Limitation
4. Self-Reflective RAG
5. Adaptive RAG

1. Transformer & LLM

Attention is all you need(NIPS, 2017)

- ...
- We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely.
- 이 논문에서는 RNN과 CNN을 요구하지 않는, 새롭고 간결한 모델인 Transformer을 제시한다.
- ...

Machine Translation on WMT2014 English-German



Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	



Transformer 구조

- Encoder-decoder 구조

- 논문에서 공개된 Transformer 모델은 번역기(Neural Machine Translation)

- English – French(WMT2014)

- English - German(WMT2014)

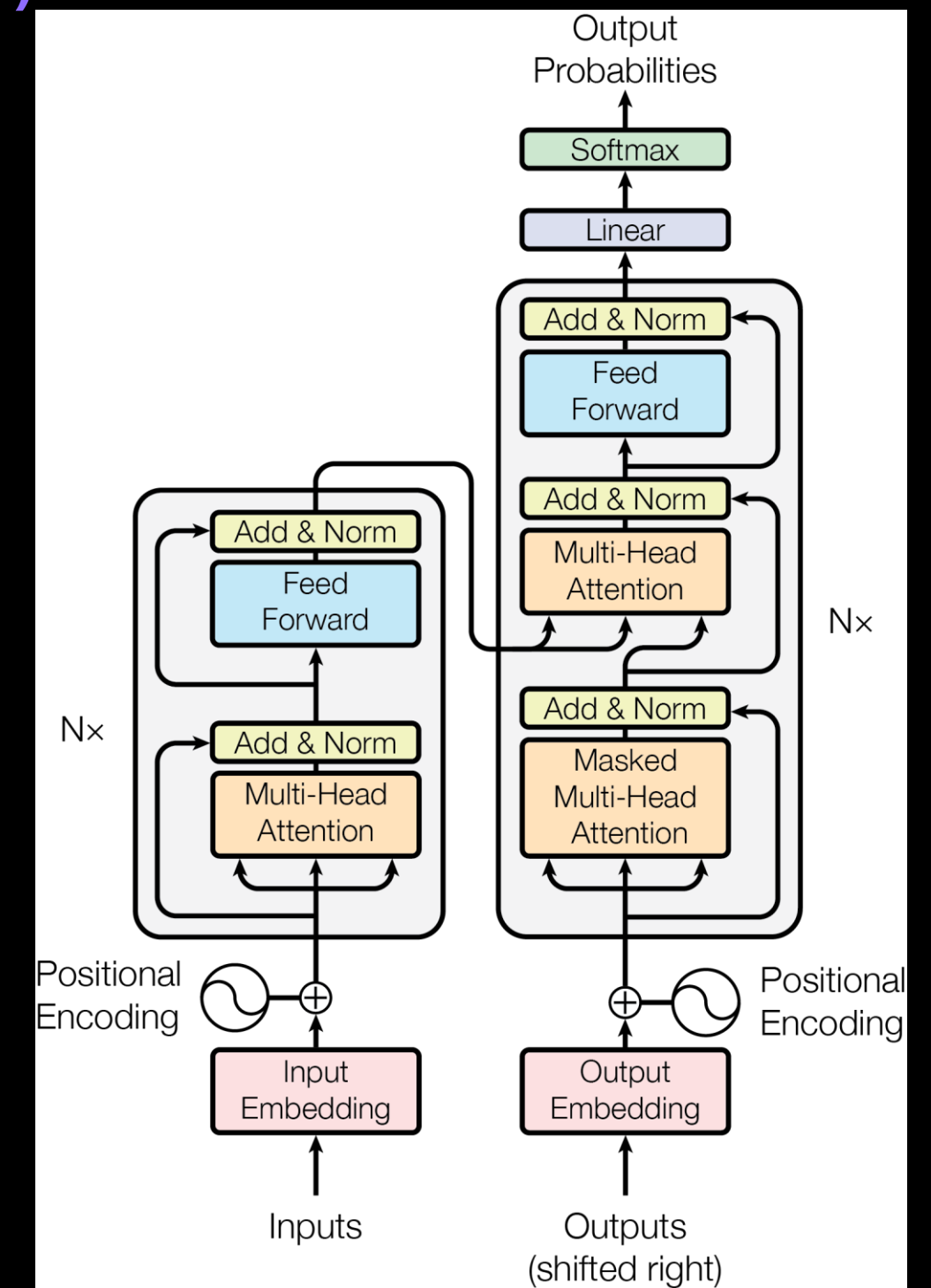
- 인코더

- Source(영어)문장을 학습

- 디코더

- Target(프랑스어/독일어)문장을 학습

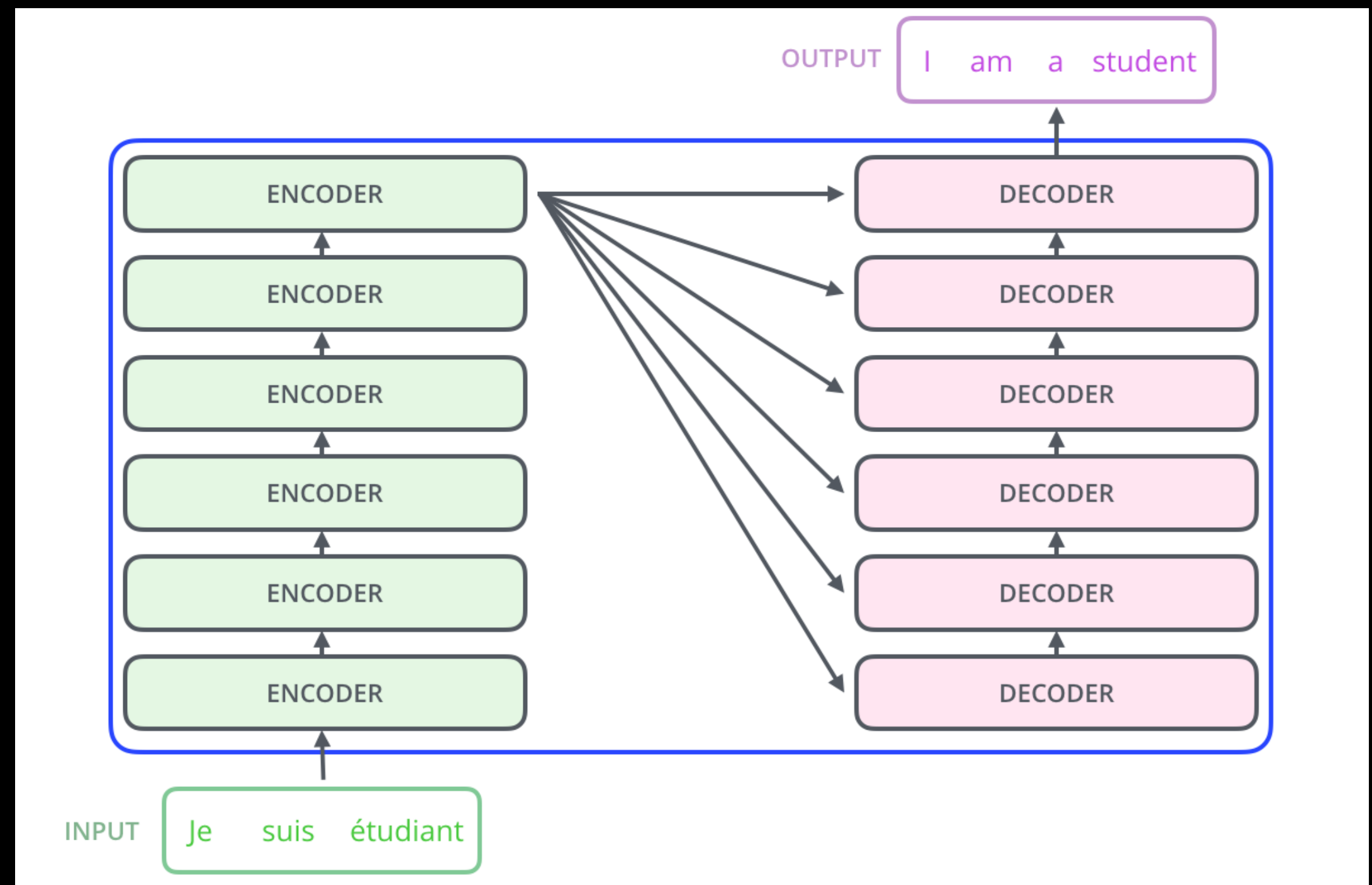
- Source(영어)문장의 문맥을 참조하여 Target(프랑스어/독일어)문장을 생성





Transformer 구조

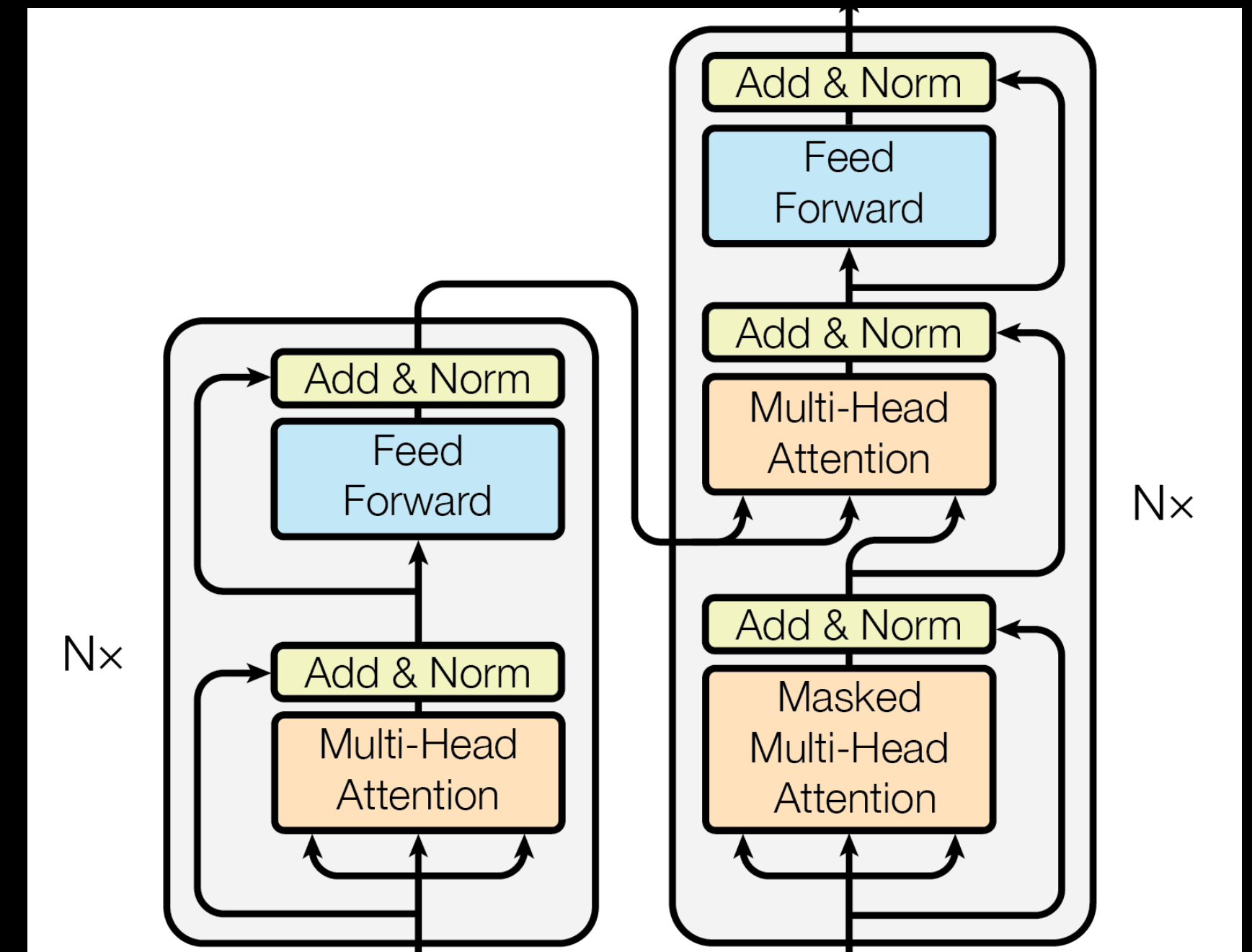
- Encoder-decoder 구조
 - 한 모델에서 인코더와 디코더를 여러 층으로 쌓음
 - 논문 기준
 - 작은 모델: 6층
 - 큰 모델: 12층
 - 모델의 크기를 원하는 대로 조정 가능
 - 학습 데이터의 크기에 따라
 - 데이터의 복잡도에 따라





Transformer 구조

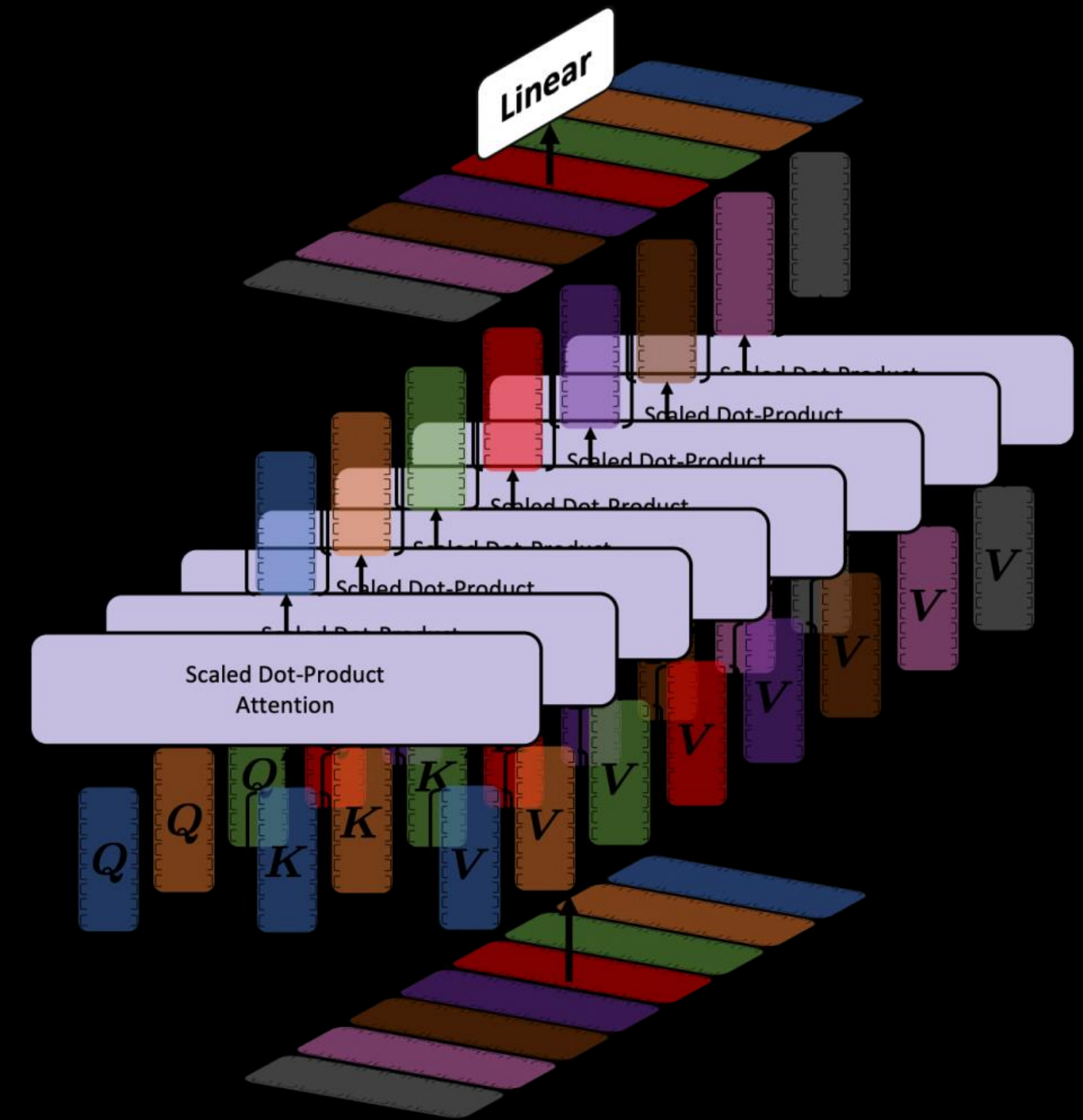
- Attention Block들이 자연어 문장에서의 문맥을 학습
 - 인코더
 - Multi-head attention: Src 문장 내 문맥 학습
 - 디코더
 - Masked multi-head attention: Tgt 문장 내 문맥 학습
 - Multi-head attention(위): Tgt 문장 생성 학습





Transformer 구조

- Multi-Head Attention
 - 여러 개의 Attention 메커니즘 동시 사용
 - 병렬 연산 가능
 - 여러 모델을 동시 학습시키는 것과 유사(Ensemble)
 - 각 Head는 다른 표현 공간에서의 Attention 정보를 캡처
 - 데이터에서 다양한 Feature를 추출할 수 있음
 - 모든 Head의 결과를 결합하여 더 풍부한 정보 표현 생성





Transformer의 영향

- 거대 언어 모델(LLM)의 등장
 - 대규모 데이터와 컴퓨팅 파워의 결합
 - Ex) GPT-4, LLaMA
 - 일반적인 태스크에서 높은 성능, 다양한 응용 가능

[-] sairjy 3h

< 2 >

× 0 ✓

We can give a good estimate of the amount of compute they used given what they leaked. The supercomputer has tens of thousands of A100s (25k according to the JP Morgan note), and they trained firstly GPT-3.5 on it 1 year ago and then GPT-4. They also say that they finish the training of GPT-4 in August, that gives a 3-4 months max training time.

25k GPUs A100s * 300 TFlop/s dense FP16 * 50% peak efficiency * 90 days * 86400 is roughly $3e25$ flops, which is almost 10x Palm and 100x Chinchilla/GPT-3.

Reply

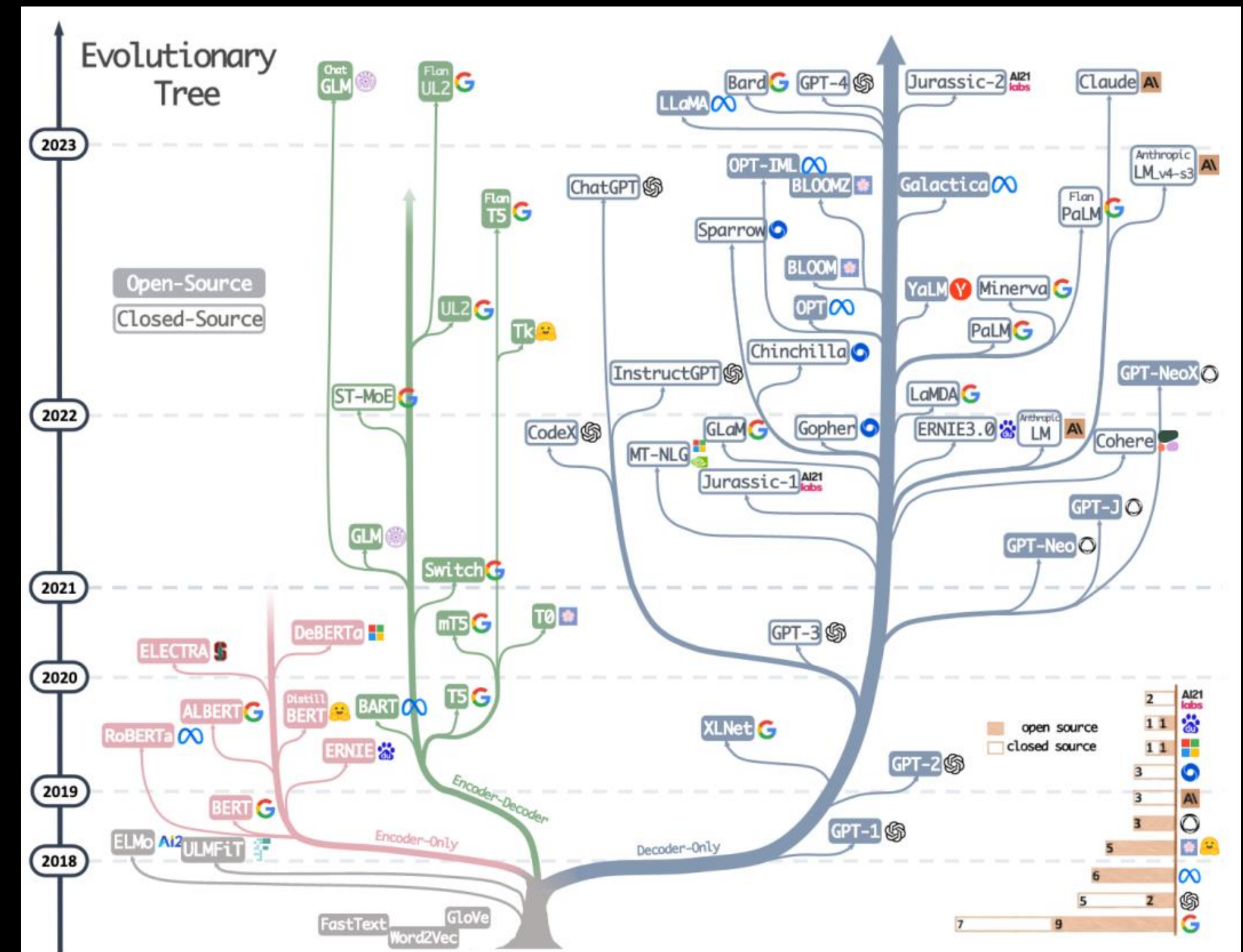


Fig. 1. The evolutionary tree of modern LLMs traces the development of language models in recent years and highlights some of the most well-known models. Models on the same branch have closer relationships. Transformer-based models are shown in non-grey colors: decoder-only models in the blue branch, encoder-only models in the pink branch, and encoder-decoder models in the green branch. The vertical position of the models on the timeline represents their release dates. Open-source models are represented by solid squares, while closed-source models are represented by hollow ones. The stacked bar plot in the bottom right corner shows the number of models from various companies and institutions.



- Bidirectional Encoder Representation from Transformers
 - Transformer의 인코더 구조를 기반으로 한 모델
 - 양방향으로 문맥을 고려하여 토큰을 인코딩
 - 주어진 텍스트에서 일부 토큰을 마스킹하고 이를 예측하는 방식으로 사전 학습
 - QA, 문장 분류 등의 다양한 NLP 문제에서 SOTA 달성
 - 일반적으로 분류 문제에 사용(적은 출력 값의 수)
 - 이어질 LLM 분야의 패러다임을 바꿈



BERT 이전 모델의 학습 문제

- 지도 학습
 - 데이터(X)와 이에 대한 레이블(y_{true})를 제공
 - 데이터를 통해 모델이 예측한 값(y_{pred})과 정답 간 차이를 계산
 - 차이가 수치화되므로 모델이 어느 방향으로 학습해야할 지 지시할 수 있음
 - 그러나 지도학습 데이터를 만드는 것은 매우 고된 일
 - 노동력
 - 비용
 - 시간
 - 텍스트 데이터의 경우 사실상 불가능



BERT 이전 모델의 학습 문제

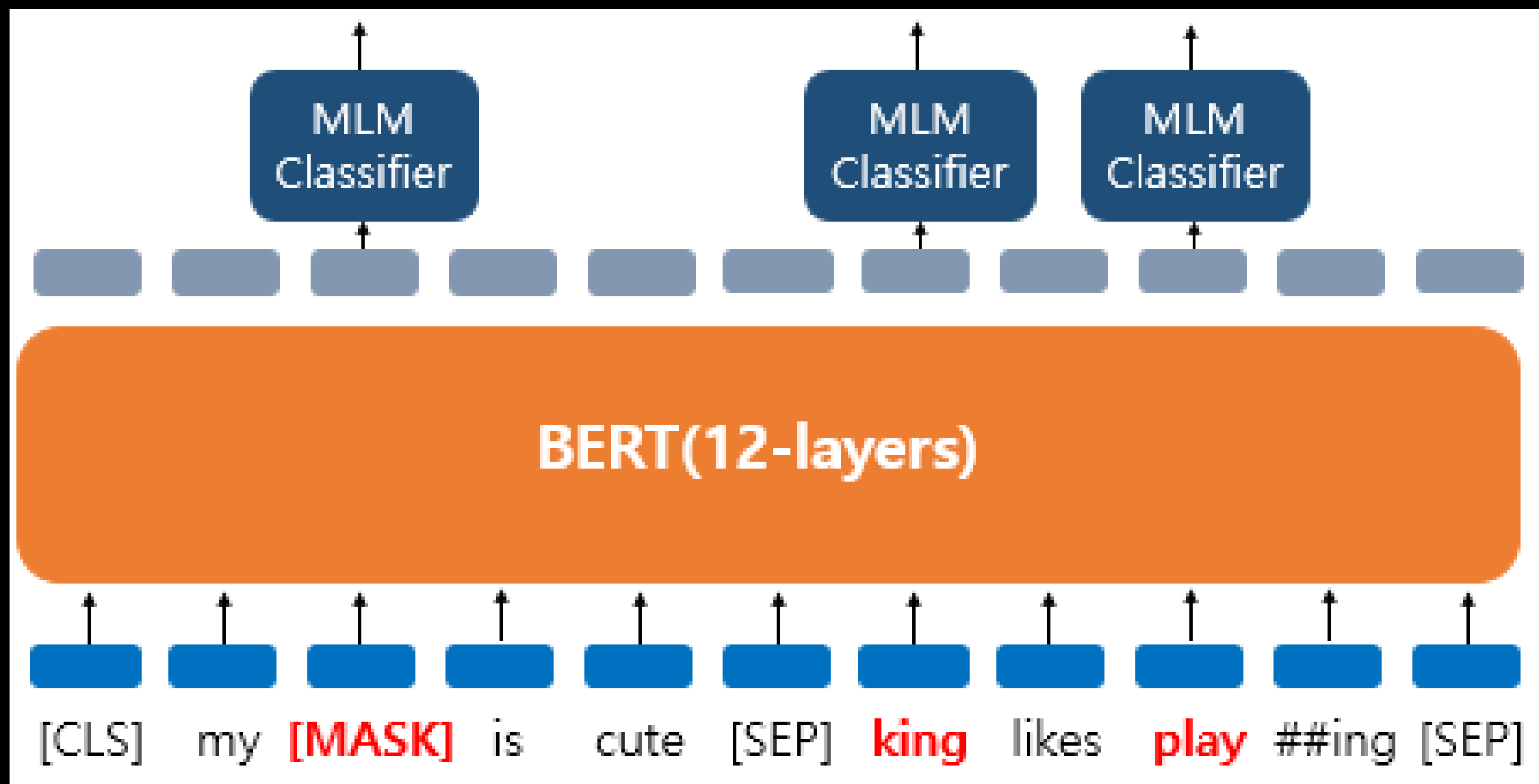
- 비지도 학습
 - 데이터만으로 모델을 학습
 - 비용이 매우 저렴
 - 방법이 매우 모호하며 추상적
 - 모델의 학습 방향을 제시하기 어려움
- BERT 이전 자연어처리 모델은 비지도 학습을 통해 의미있는 결과를 만들어내지 못함

BERT의 특징 – 사전 학습(Pretraining)

- BERT에는 무한한 양의 지도학습 데이터가 투입
 - 지도학습의 형태로 비지도학습을 수행
 - 즉 레이블이 없는 데이터를 이용하여 지도학습을 수행(Self-supervised learning)
 - 두 가지 방식의 사전 학습이 진행
 - MLM(Masked Language Model)
 - NSP(Next Sentence Prediction)

BERT의 특징 - 사전 학습(Pretraining)

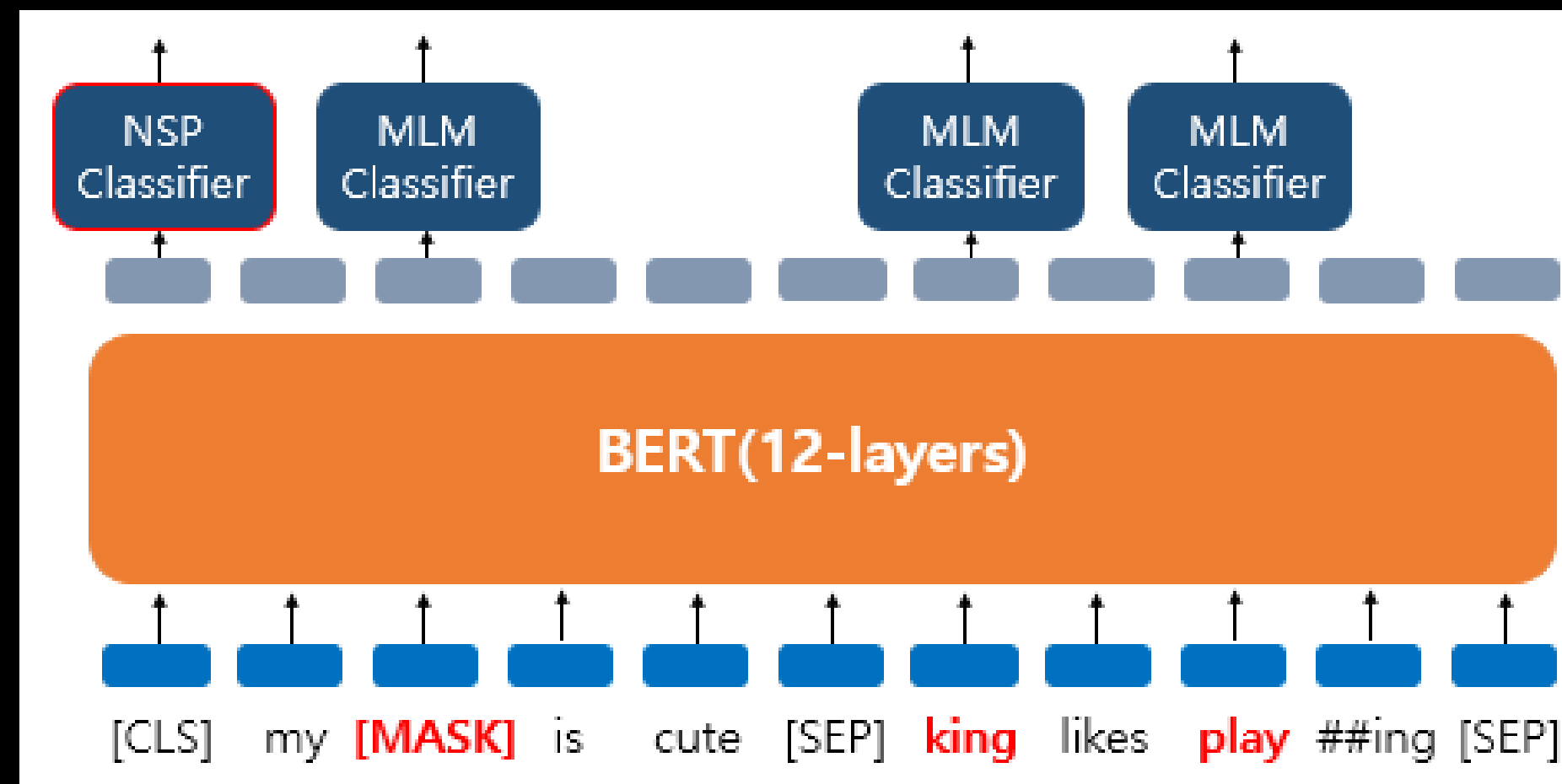
- BERT의 사전학습 - MLM(Masked Language Modelling)
- 일부(15%) 토큰을 임의로 마스킹하고 해당 토큰을 예측하는 방식으로 학습
- 이 과정을 통해 모델은 문맥을 기반으로 한 토큰의 의미를 깊게 이해



I am eating an ice cream.
I **am eating an** ice cream.
I am **eating an ice** cream.
I am eating **an ice cream**.
The boy is drinking milk.
The boy is drinking milk.
The **boy is drinking** milk.
The boy **is drinking** milk.

BERT의 특징 – 사전 학습(Pretraining)

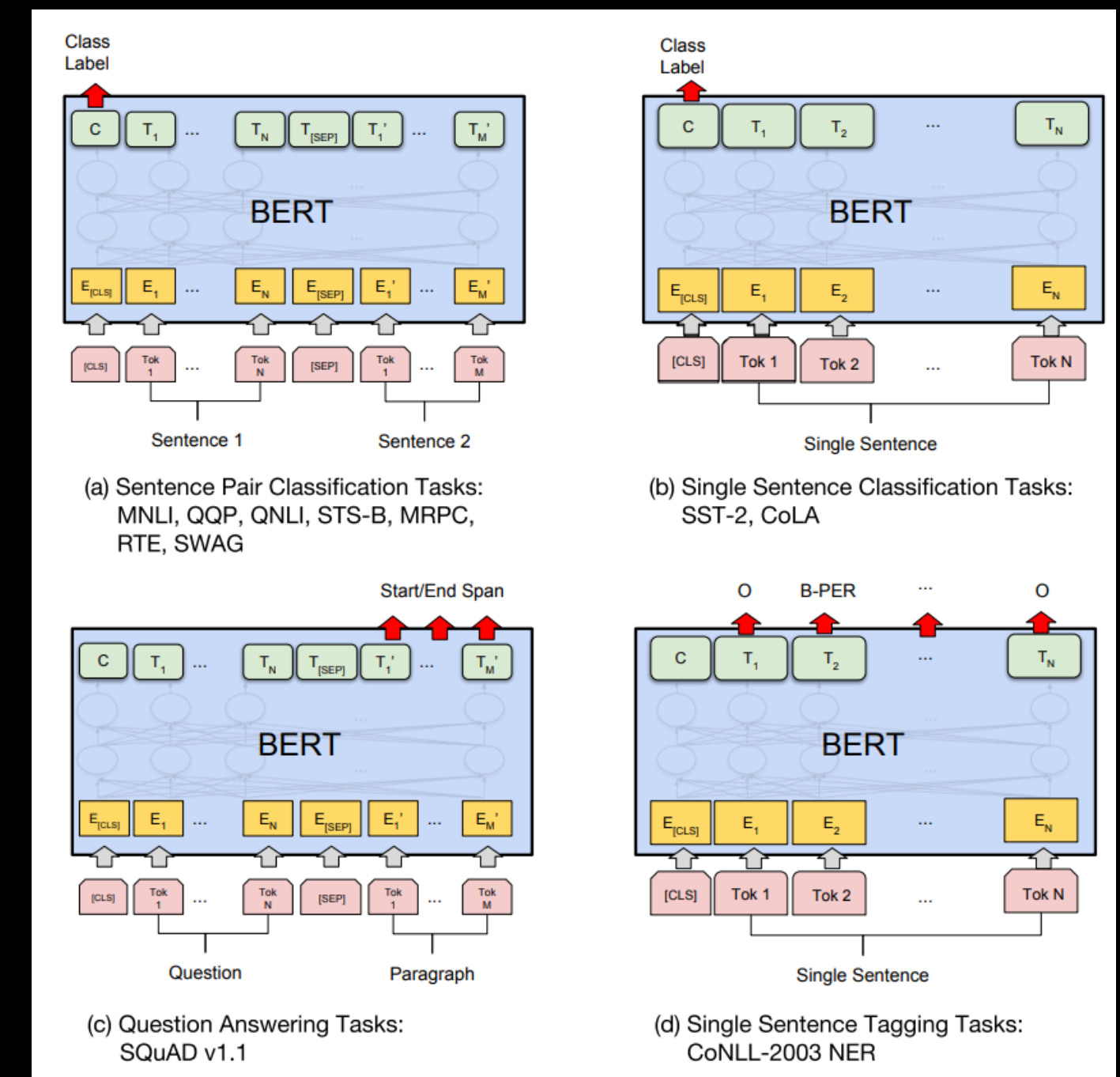
- BERT의 사전학습 – NSP(Next Sentence Prediction)
 - 두 개의 문장이 붙은 상태로 모델에 입력됨
 - 문장을 구분하기 위해 [SEP] 토큰을 구분자로 사용
 - 이진 분류를 통해, 두 문장의 연속성 여부를 학습함
 - [CLS]토큰의 위치에서 이진 분류 문제를 해결





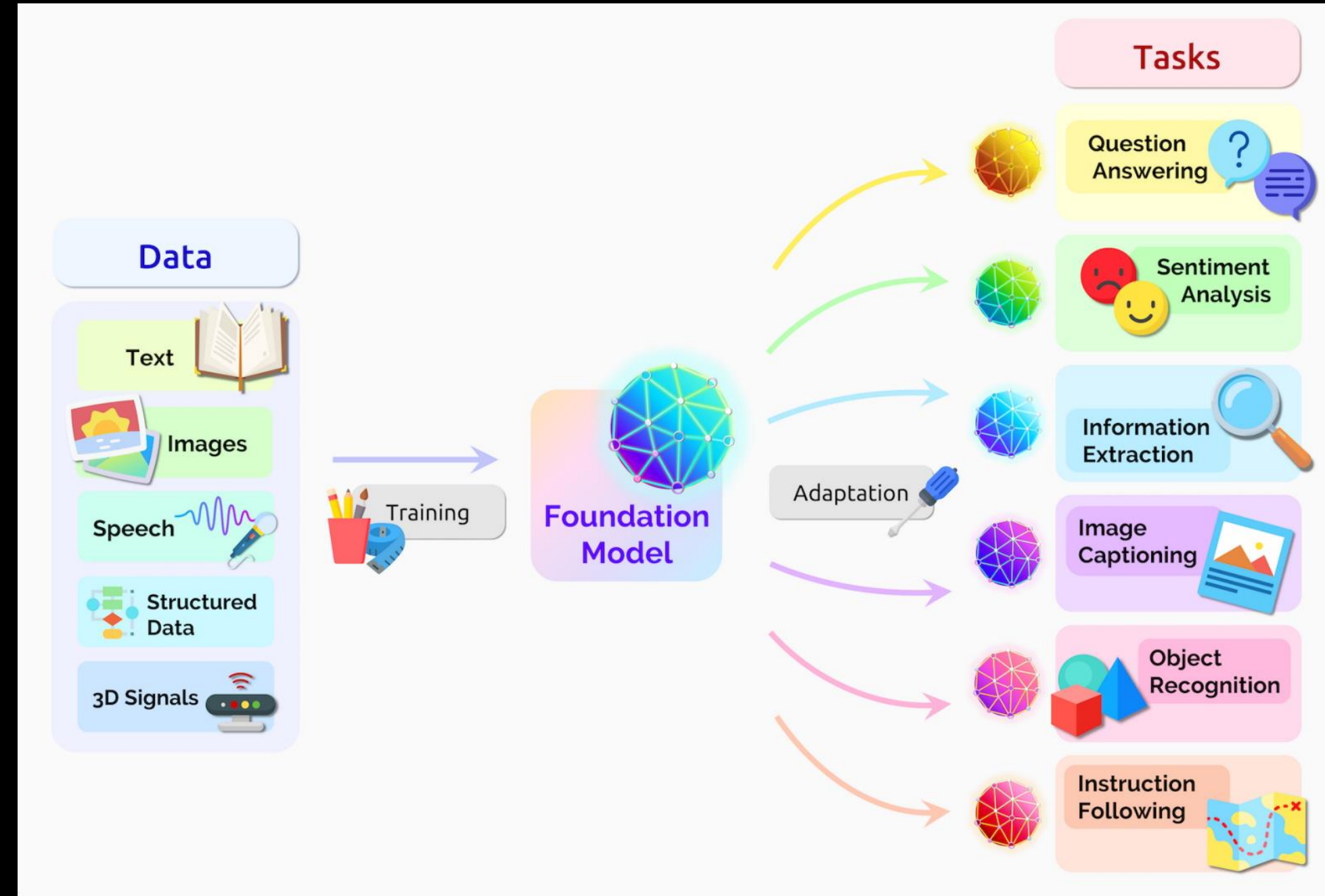
BERT의 특징 - 전이 학습

- 전이 학습(Transfer learning and Fine-tuning)
 - BERT는 큰 텍스트 코퍼스에서 사전 학습 후, 특정 작업에 미세 조정 가능
 - Wikipedia(2.5B), BooksCorpus(8M)
 - 11가지 NLP task에서 SOTA 달성
 - 각 Task마다 모델 말단의 구조만 다름
 - 대규모 텍스트로 사전학습된 모델을 Foundation model이라 함



BERT의 특징 - 사전 학습(Pretraining)

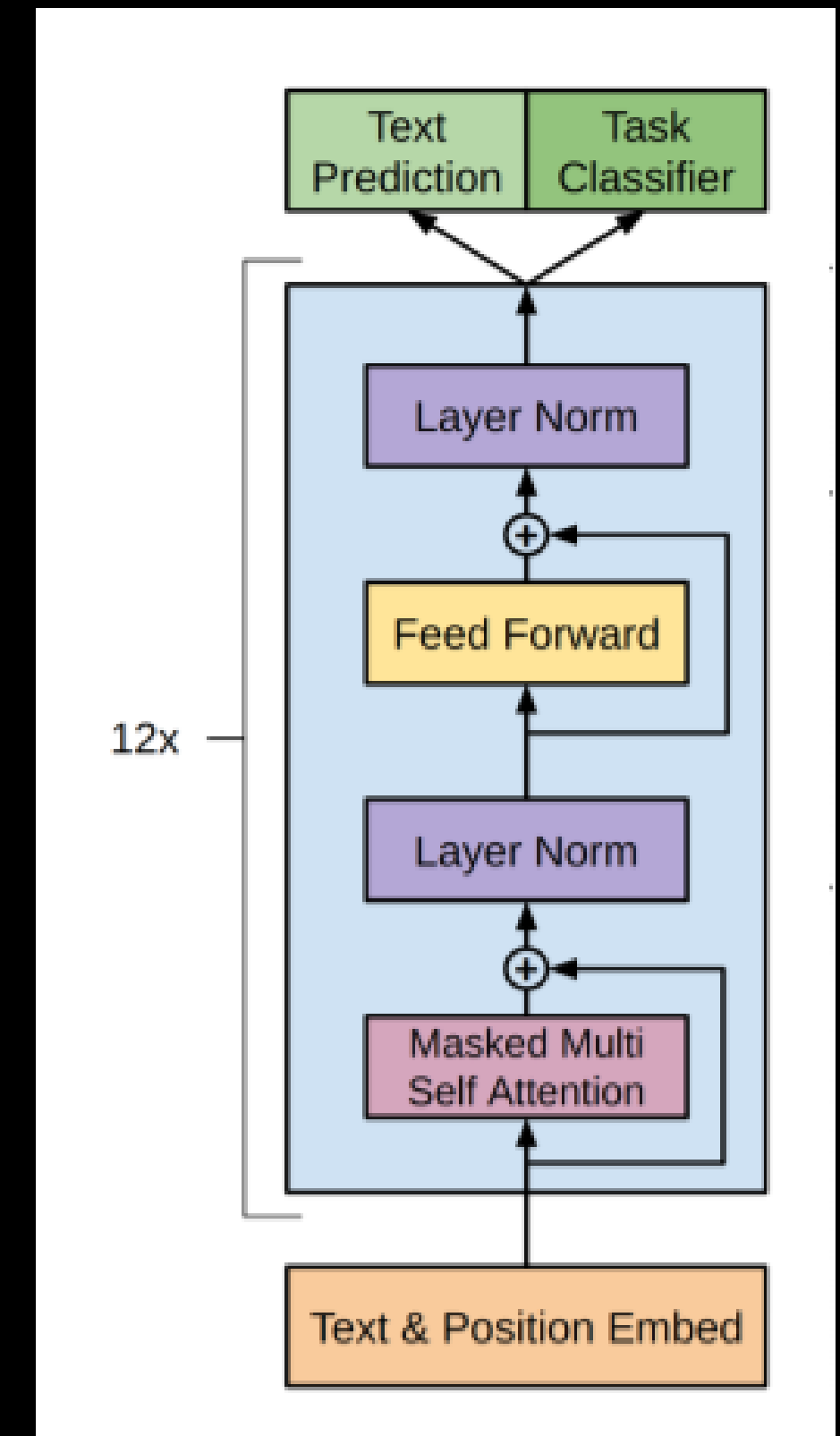
- Foundation 모델
 - 과거에는 1작업 1모델
 - BERT 이후 모델 하나가 여러 작업을 담당하도록 훈련
 - 비용 부담 감소 및 상용화



BERT 이후 주요 언어 모델 – GPT

- GPT-1

- 논문: [Improving Language Understanding by Generative Pre-Training]
- 디코더만으로 구성된 모델
 - 문장 생성이 목표
 - 문장 생성을 통해 모든 문제 해결 가능
- 지도학습을 통한 모델 훈련
 - 비지도학습을 통한 Pretraining
 - 지도학습을 통한 Fine-tuning
- 비지도학습은 Language Modelling을 통해 진행
 - MLM, NSP 등과 비슷한 방식



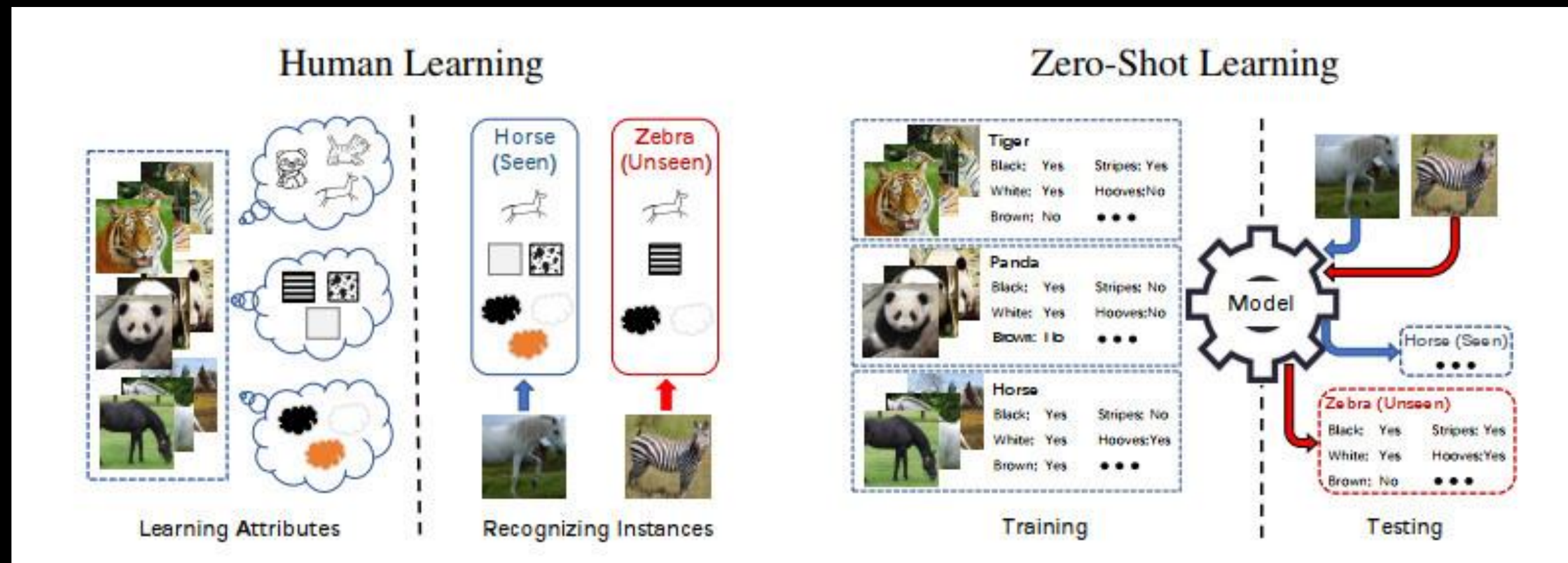


BERT 이후 주요 언어 모델 – GPT

- GPT-2
 - 논문: [Language Models are Unsupervised Multitask Learners]
 - 학습 방식
 - 비지도학습만을 사용하여 Text-generation task 수행 성공
 - 별도의 SFT(Supervised Fine-Tuning) 없이도 Downstream task에서 우수한 (통계적)성능 발휘
 - 특징
 - 다양한 NLP 태스크에서 Zero/Few-shot 성능을 보여줌
 - 고도의 문장 생성 능력
 - 초기에는 모델의 크기와 생성 능력 때문에 공개를 주저함
 - 현재는 오픈소스로 공개

N-shot

- N-Shot: 학습 과정 중 참조한 데이터의 수
- Few-shot: 학습 과정 중 적은 수의 데이터를 참조
- 1-shot: 학습 샘플 중 특정 데이터가 하나만 존재
- 0-shot: 학습 샘플에 특정 데이터가 포함되지 않음
- 텍스트 생성 모델은 학습하지 않은 문장도 생성할 수 있음



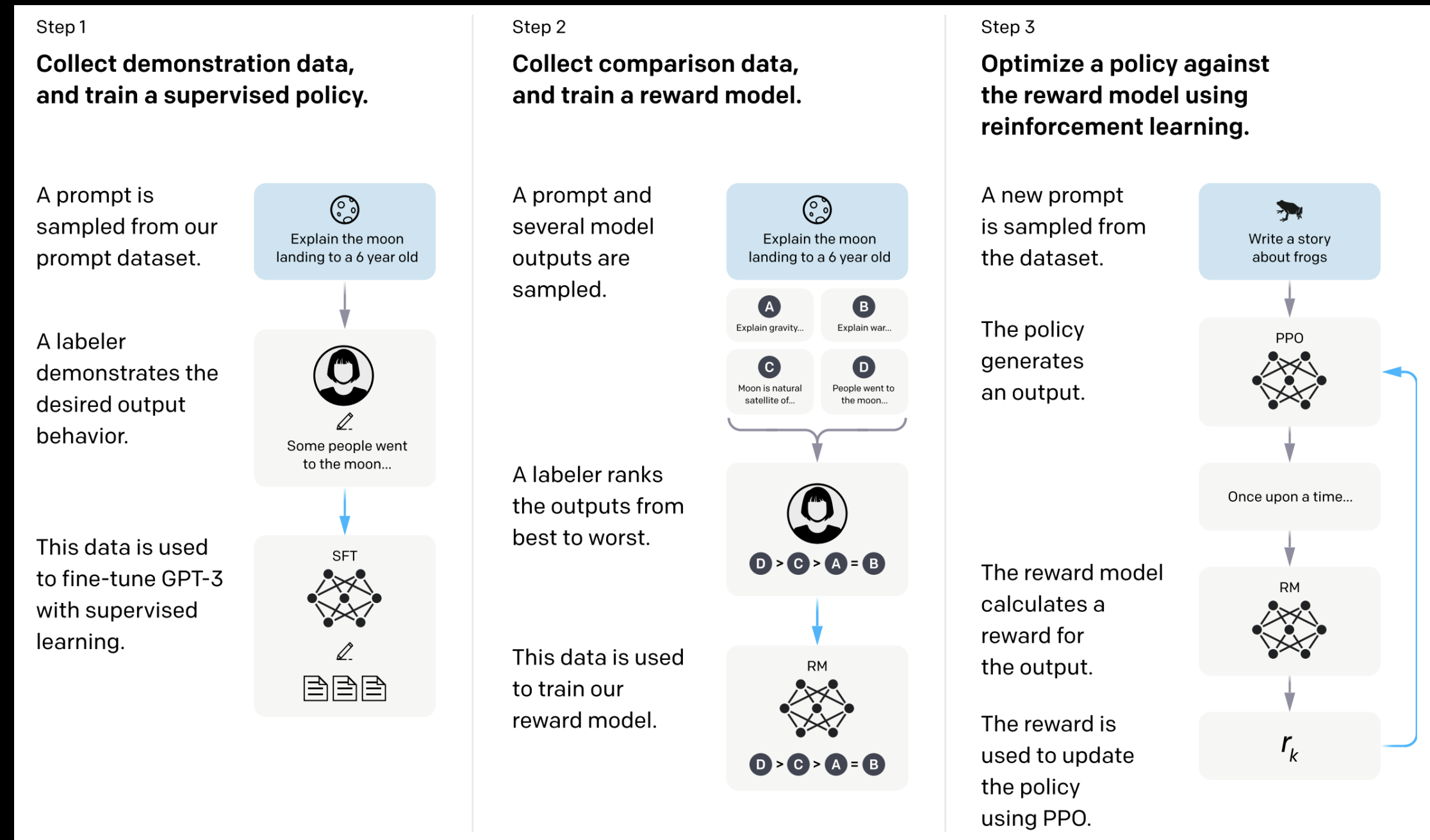


BERT 이후 주요 언어 모델 – GPT

- GPT-3
 - 논문: [Language Models are Few-Shot Learners]
 - GPT-2와 유사한 비지도학습, 하지만 더 큰 데이터와 모델로 학습
 - 1750억 개의 파라미터를 가진 거대한 모델
 - In-context learning
 - 별도의 태스크별 미세 조정 없이도 프롬프트만으로 충분한 정보 전달
 - 잘 작성된 프롬프트 만으로도 모델의 성능을 높일 수 있음 → Prompt engineering
 - 매우 다양한 태스크에서 Zero-shot, Few-shot 학습 능력
 - 자연어 질의응답, 문장 생성, 번역, 요약 등 다양한 작업 수행 가능
 - API 및 상용화
 - GPT-3 기반의 API가 제공되어 다양한 애플리케이션 개발에 활용

BERT 이후 주요 언어 모델 – GPT

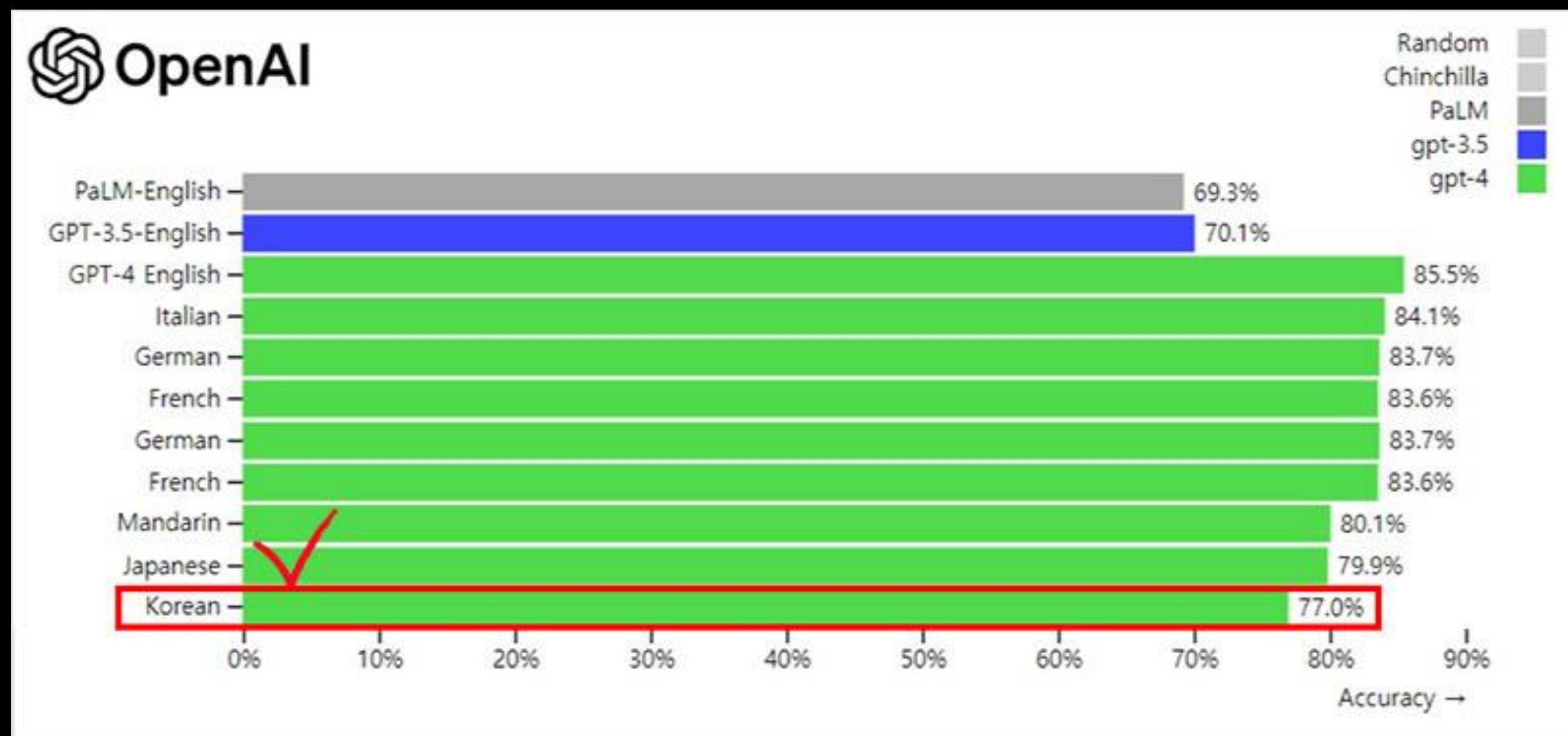
- Instruct-GPT
 - OpenAI ChatGPT 챗봇의 Base model
 - GPT-3.5
 - Instruction-tuning을 통해 학습
 - Supervised Fine-Tuning의 일종
 - X: 사람이 작성한 명령어(prompt)
 - Y: 이에 맞는 올바른 응답 데이터
 - GPT에게 대화하는 법을 지시
 - 완성된 문장으로 기술하고
 - 민감하거나 유해한 내용 배제





BERT 이후 주요 언어 모델 – GPT

- GPT-4
 - 엄청나게 큰 모델로 많은 양의 데이터를 오랜 기간 학습
 - 추정치(풍문)
 - 모델 파라미터 수 1.8tr(1.8조)
 - Nvidia A100 * 25,000 사용
 - \$6300만(=850억 원)
 - 학습 기간 90 – 100일



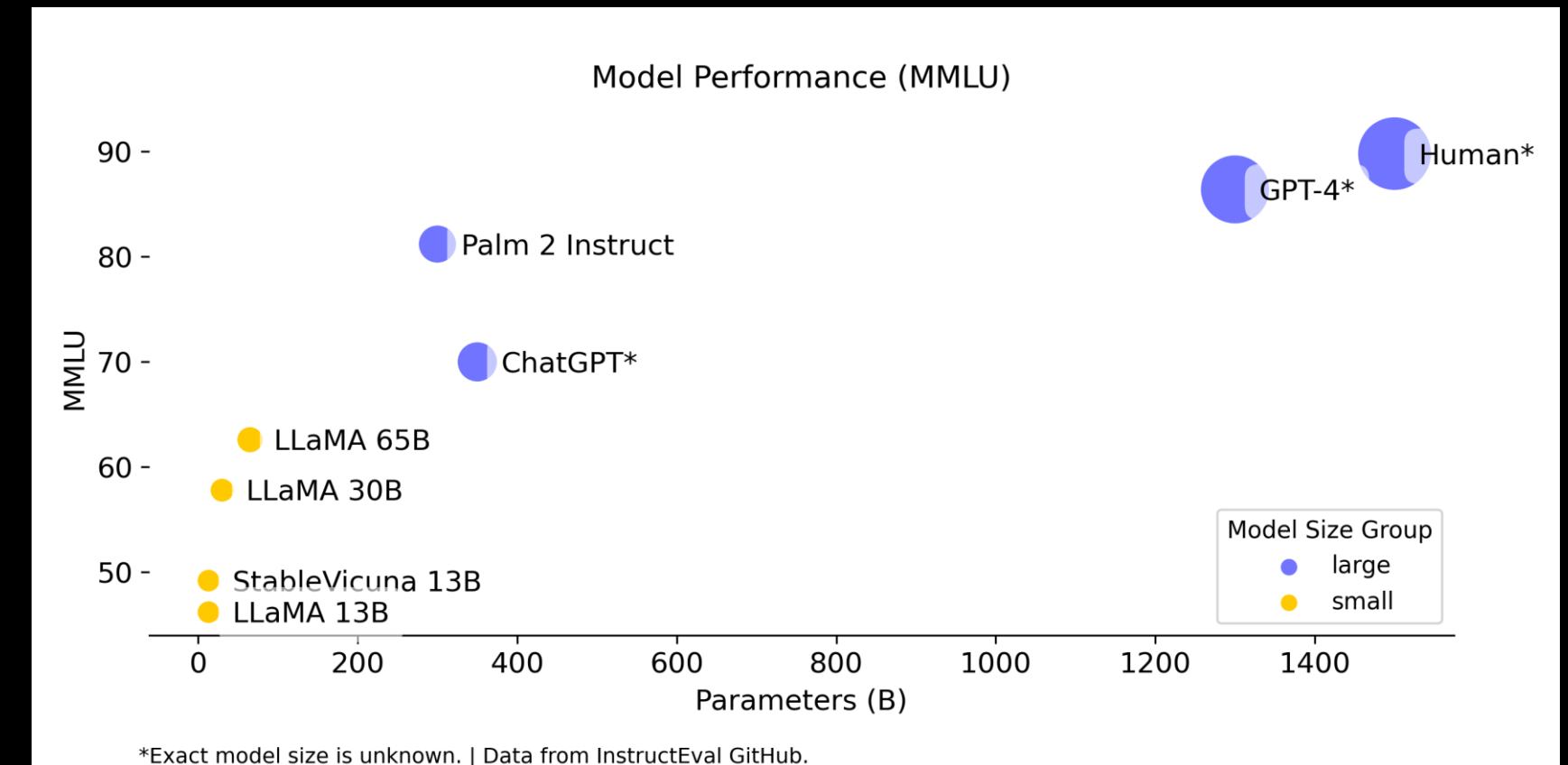


BERT 이후 주요 언어 모델 – GPT

- GPT-4
 - Multi-linguality: 24년 기준 58개국어 지원
 - Multimodality: 이미지(CLIP), STT(Whisper) 등의 모달리티 처리
 - 문제 해결 능력 증가
 - 입력 가능 토큰 수 증가
 - Hallucination 감소
 - Zero/Few-shot 성능 강화
 - 안전성 및 Alignment 개선
 - RLHF등을 통한 정밀한 미세 조정 과정을 거쳐, 부적절한 출력을 줄이고 사용자 의도를 보다 정확하게 반영

BERT 이후 주요 언어 모델 - LLaMA

- GPT-4
 - 엄청나게 큰 모델로 많은 양의 데이터를 오랜 기간 학습
 - 과연 모델의 파라미터 수는 데이터의 양에 적합할까?
 - 모델 파라미터 공간이 데이터에 비해 작다면 Underfitting
 - 모델 파라미터 공간이 데이터에 비해 크다면 Overfitting
- Meta에서 LLaMA를 공개
 - [Open and Efficient Foundation Language Models]
 - 데이터 크기를 고려하여, 이에 적합한 모델 구축



BERT 이후 주요 언어 모델 - LLaMA

- LLaMA
 - 파라미터가 데이터 학습 능력을 낭비하지 않게끔 계산하여 설계
 - 다양한 크기의 모델을 공개
 - LLaMA 1 기준 7B, 13B, 33B, 65B
 - 13B 모델로 GPT-3(175B)과 유사한 성능을 보임
 - 다만 다국어 성능에서 살짝 아쉬운 점수를 보임

		BoolQ	PIQA	SIQA	HellaSwag	WinoGrande	ARC-e	ARC-c	OBQA
GPT-3	175B	60.5	81.0	-	78.9	70.2	68.8	51.4	57.6
Gopher	280B	79.3	81.8	50.6	79.2	70.1	-	-	-
Chinchilla	70B	83.7	81.8	51.3	80.8	74.9	-	-	-
PaLM	62B	84.8	80.5	-	79.7	77.0	75.2	52.5	50.4
PaLM-cont	62B	83.9	81.4	-	80.6	77.0	-	-	-
PaLM	540B	88.0	82.3	-	83.4	81.1	76.6	53.0	53.4
LLaMA	7B	76.5	79.8	48.9	76.1	70.1	72.8	47.6	57.2
	13B	78.1	80.1	50.4	79.2	73.0	74.8	52.7	56.4
	33B	83.1	82.3	50.4	82.8	76.0	80.0	57.8	58.6
	65B	85.3	82.8	52.3	84.2	77.0	78.9	56.0	60.2

BERT 이후 주요 언어 모델 - LLaMA

- LLaMA
 - 오픈 소스로 모델 공개
 - 모델 & 사전학습 가중치 사용 가능
 - 모델의 경우 윤리&보안 서약 동의 후 사용 가능
 - 이용자가 모델을 바탕으로 Fine-tuning 가능
 - 상업적 사용 전면 허용
 - LLaMA2, LLaMA3 도 동일
 - 지속적인 성능 업데이트
 - LLaMA2는 2배 많은 토큰을 입력 가능(2k, A4 기준 6페이지)
 - LLaMA3는 128k 까지 토큰 수 확장
 - 코딩 등 다양한 측면에서 발표 당시 최고 성능을 기록

BERT 이후 주요 언어 모델 - LLaMA

- LLaMA 3.1
 - 입력 컨텍스트 128k로 확장(~A4 200p)
 - 8개국어 지원(한국어 미포함)
 - 세 가지 라인업 발표
 - 8B, 70B, 405B
 - 8B > Mistral7B, Gemma2 9B
 - 70B > GPT3.5, 4o와 비슷
 - 405B는 GPT4와 유사한 성능

Category Benchmark	Llama 3.1 405B	Nemotron 4 340B Instruct	GPT-4 (0125)	GPT-4 Omni	Claude 3.5 Sonnet
General					
MMLU (0-shot, CoT)	88.6	78.7 (non-CoT)	85.4	88.7	88.3
MMLU PRO (5-shot, CoT)	73.3	62.7	64.8	74.0	77.0
IFEval	88.6	85.1	84.3	85.6	88.0
Code					
HumanEval (0-shot)	89.0	73.2	86.6	90.2	92.0
MBPP EvalPlus (base) (0-shot)	88.6	72.8	83.6	87.8	90.5
Math					
GSM8K (8-shot, CoT)	96.8	92.3 (0-shot)	94.2	96.1	96.4 (0-shot)
MATH (0-shot, CoT)	73.8	41.1	64.5	76.6	71.1
Reasoning					
ARC Challenge (0-shot)	96.9	94.6	96.4	96.7	96.7
GPQA (0-shot, CoT)	51.1	-	41.4	53.6	59.4
Tool use					
BFCL	88.5	86.5	88.3	80.5	90.2
Nexus	58.7	-	50.3	56.1	45.7
Long context					
ZeroSCROLLS/QuALITY	95.2	-	95.2	90.5	90.5
InfiniteBench/En.MC	83.4	-	72.1	82.5	-
NIH/Multi-needle	98.1	-	100.0	100.0	90.8
Multilingual					
Multilingual MGSM (0-shot)	91.6	-	85.9	90.5	91.6

2. LLM 활용의 한계와 Prompt Engineering



생성형 언어 모델의 한계

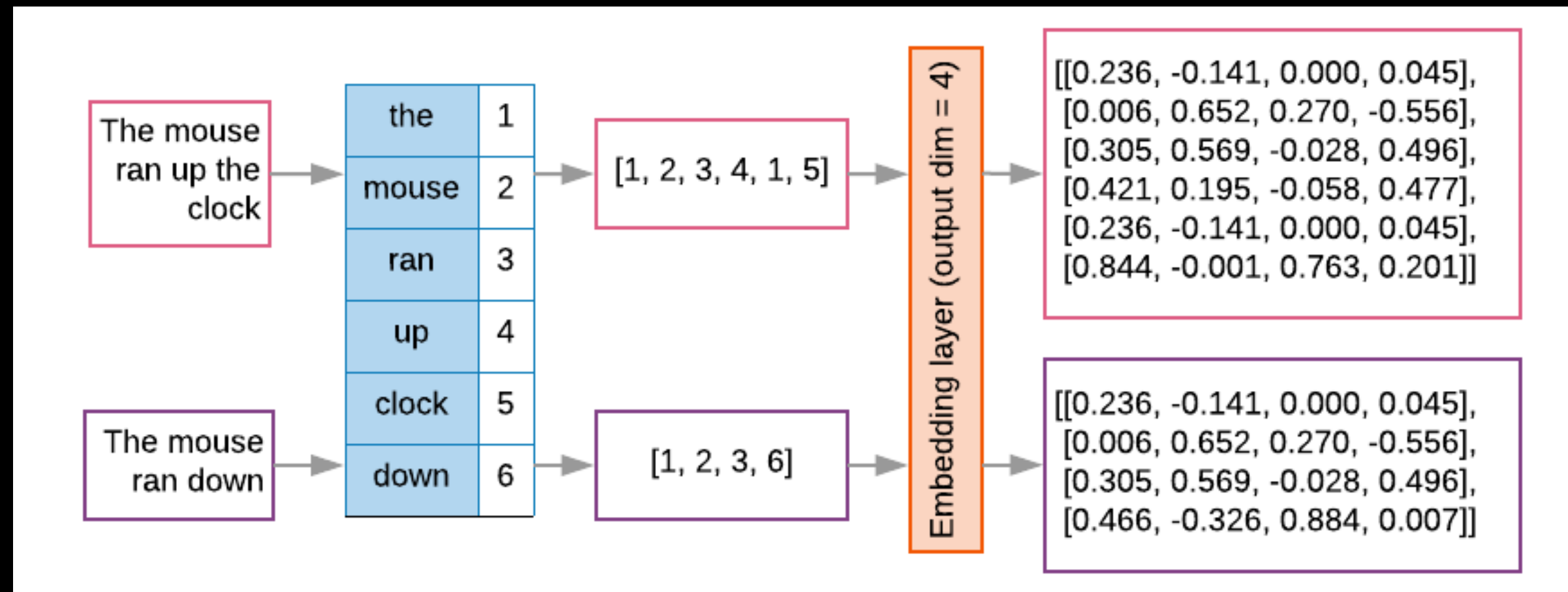
- LLM 등 인공지능을 산업에 적용하기엔 한계가 뚜렷함
 - 책임 소재의 불분명
 - 지나치게 비싼 학습 및 연산 비용
 - 활용 가능한 인력 부족
 - 복잡한 업무 수행 불가
 - 해석 가능성 낮음
 - 부정확한 내용 제공

AI의 학습 원리를 살펴보면, 이들의 약점과 해결 방안이 보임



자연어 데이터 수치 변환

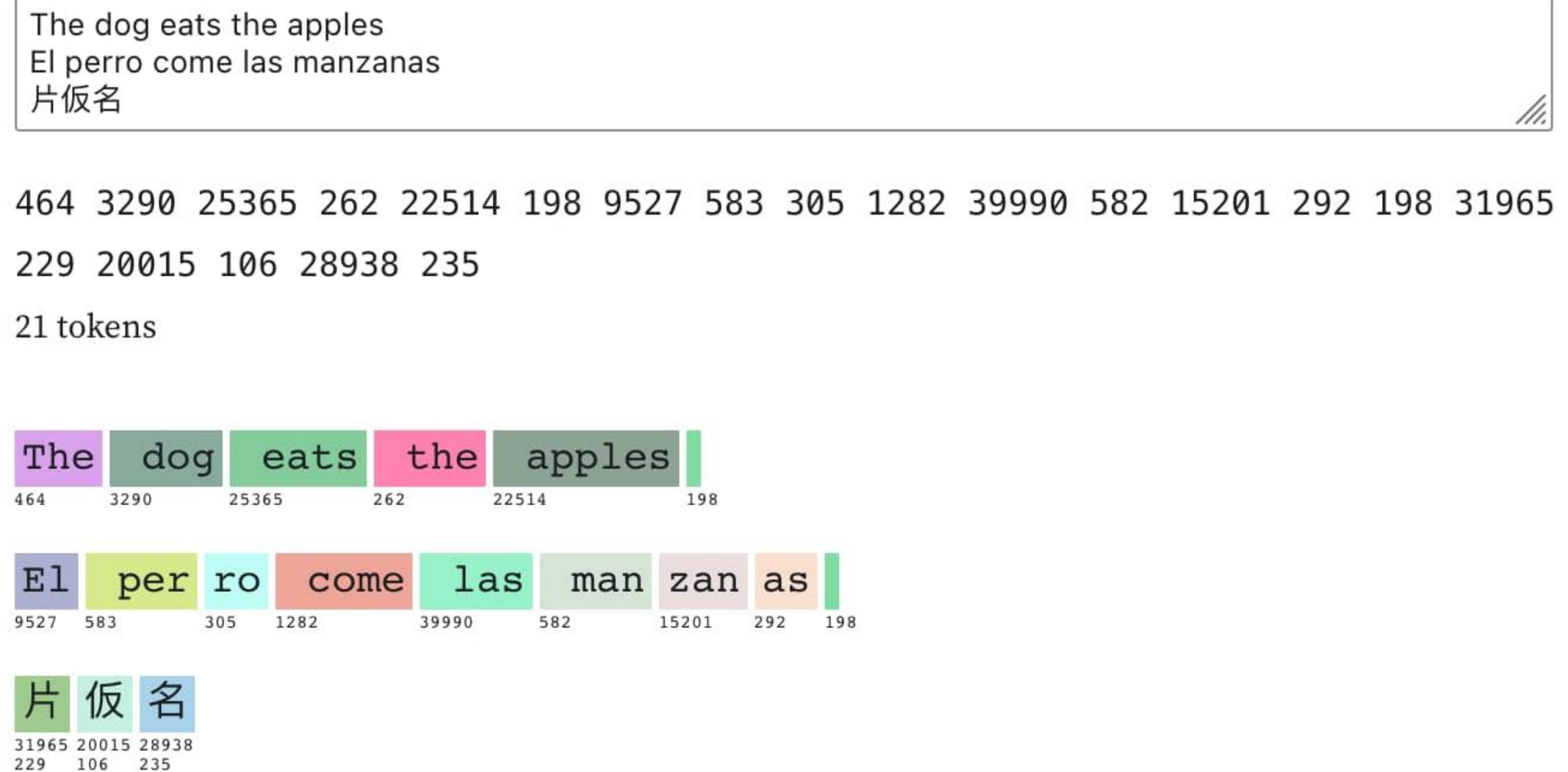
- 다른 데이터와 달리 자연어는 숫자 변환이 필수
 - 모델은 숫자만 연산 가능
 - 오랜 기간 많은 연구가 언어를 숫자로 변환하는 과정에 집중
 - 과거에는 빈도 기반으로 접근; TF-IDF, BoW(Bag of Words) 등
- 현재(2025)는 토큰화 → 임베딩 과정이 일반적



• 토큰화(Tokenization)

- 텍스트를 의미 있는 단위인 '토큰'으로 분리하는 과정
- 변환된 단어에 인덱스를 연결
- 단어사전에 단어-인덱스를 등재

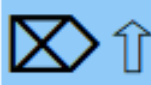
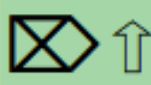
```
{ '<unk>': 1,  
  'here': 2,  
  'the': 3,  
  'largest': 4,  
  'town': 5,  
  'of': 6,  
  'district': 7,  
  'is': 8,  
  'located': 9,  
  'nordenham': 10,  
  'lying': 11,  
  'opposite': 12,  
  'to': 13,  
  'bremerhaven': 14,  
  'at': 15,  
  'weser': 16,  
  'mouth': 17,  
  'i': 18,  
  'should': 19,  
  'like': 20,  
  'in': 21,  
  'passing': 22,  
  'pay': 23,
```



- 현 시점 대부분의 LLM은 Subword tokenizer를 사용
- 또한 각 모델마다 고유의 Tokenizer를 사용

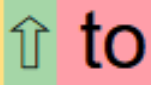
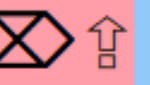
TokenMonster is lossless.

decoded TokenMonster is lossless.

encoded  token  monster is lossless.

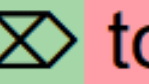
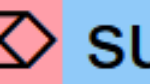
WORD Tokens are independent of "CAPITALIZATION"

WORD Tokens are independent of "CAPITALIZATION"

 word  tokens are independent of " capitalization"

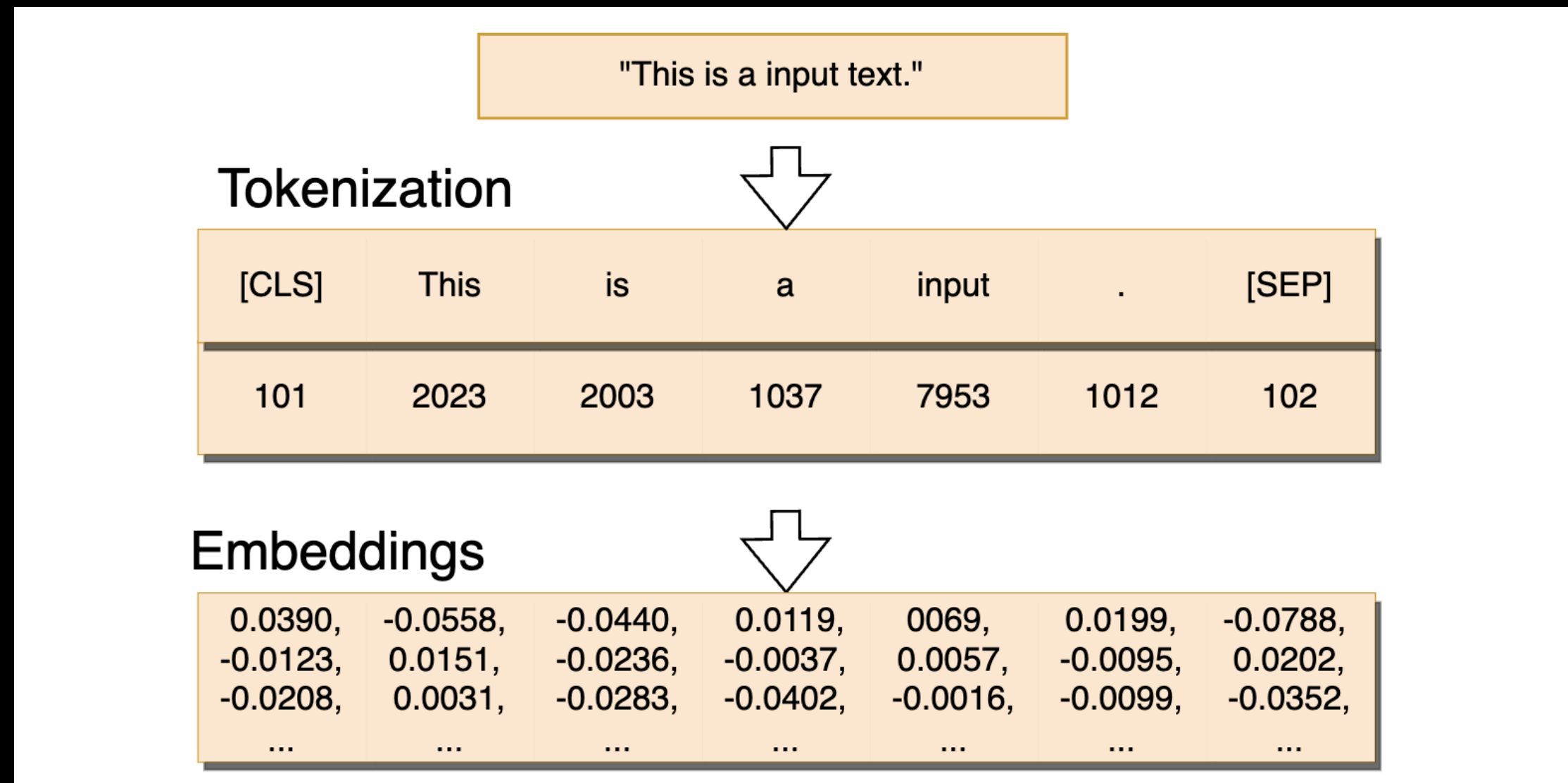
Joinedtogetherwordsarefullysupported

Joinedtogetherwordsarefullysupported

 joined  togetherwordsarefully  supported

- 벡터화(Vectorization)

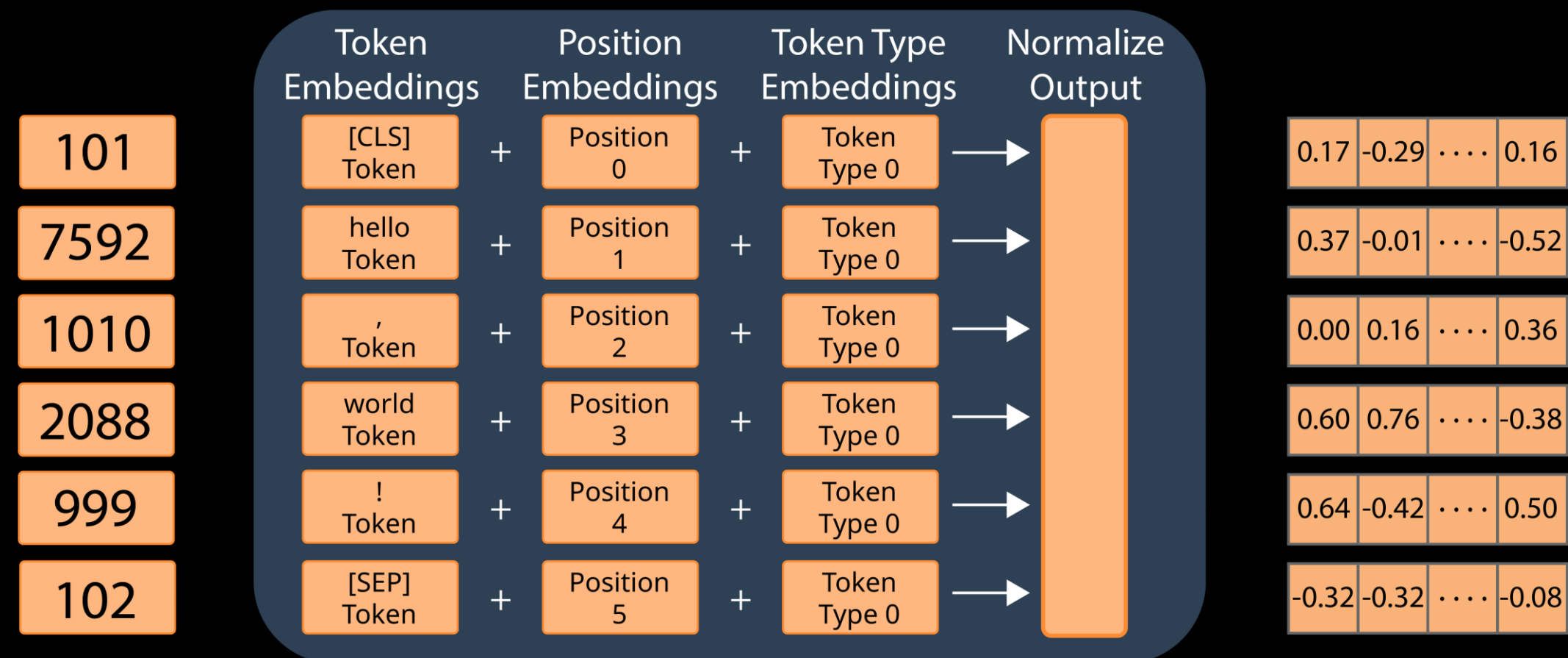
- 데이터를 기계 학습 알고리즘이 이해할 수 있는 형태로 변환하는 것
- 데이터의 형식이 아닌 의미를 수치로 환산하는 과정
- 단어, 문장, 문서 등 다양한 수준의 텍스트 데이터를 벡터로 표현할 수 있음





LLM의 토큰과 임베딩

- LLM은 언어를 학습/추론하기 위해 모든 단어를 토큰(Token) 단위로 분할
 - 그러나 토큰이 분할될 때 단어를 제대로 분절하지 못한다면?
- 각 토큰에 대한 의미를 여러 개의 실수(고차원 벡터)로 표현
 - 그러나 각 토큰에 대한 의미가 제대로 반영되지 못한다면?

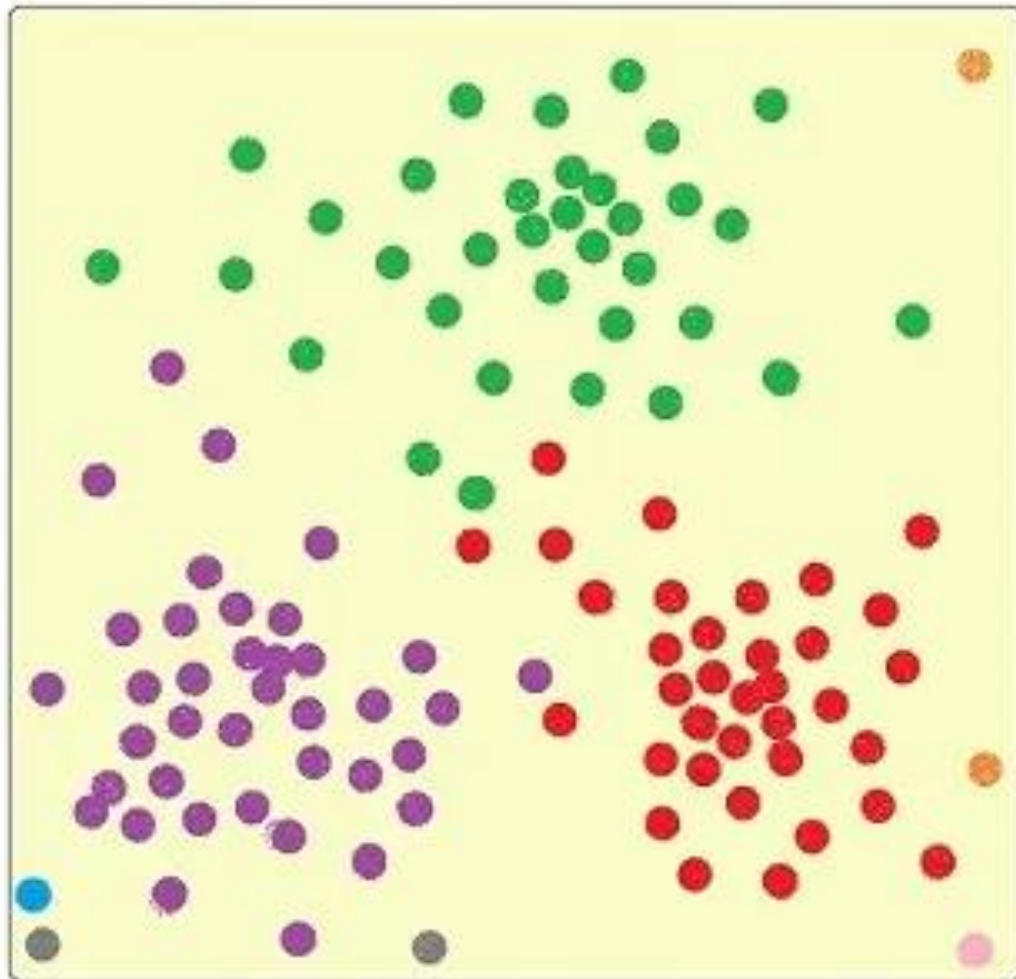




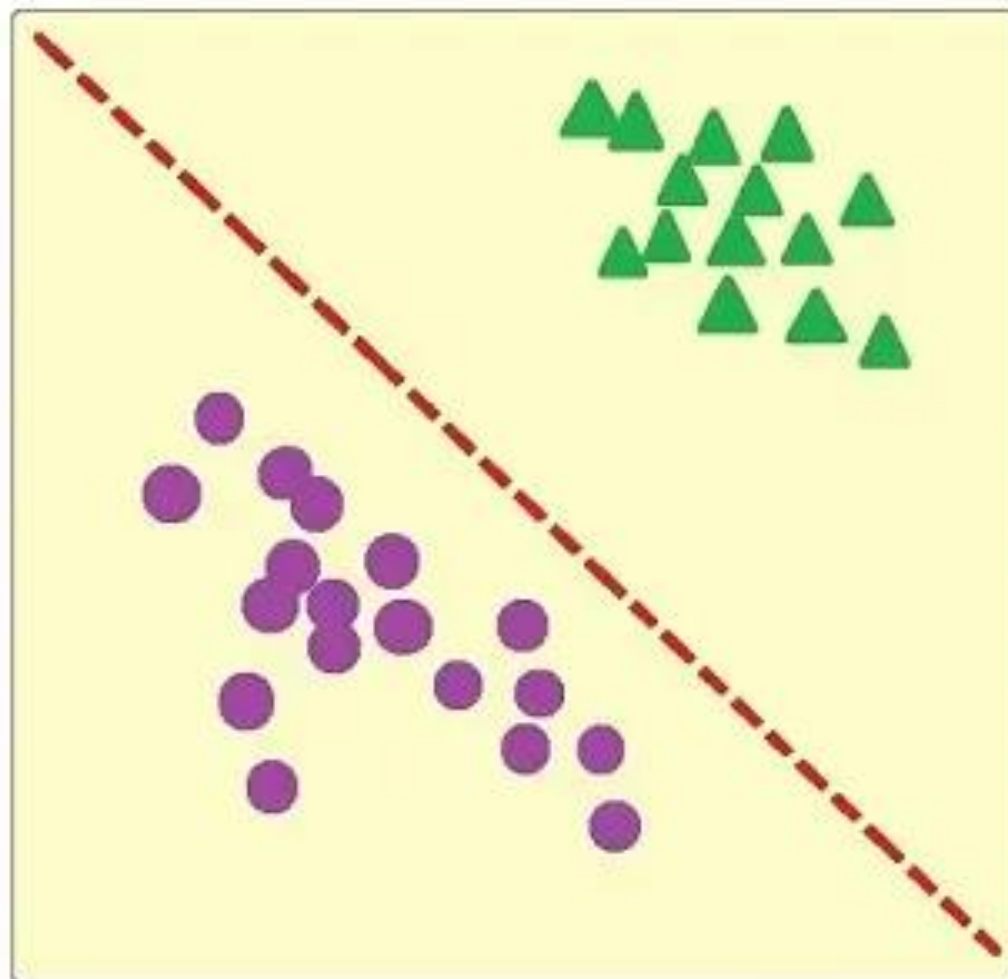
LLM의 학습 방식

Machine Learning

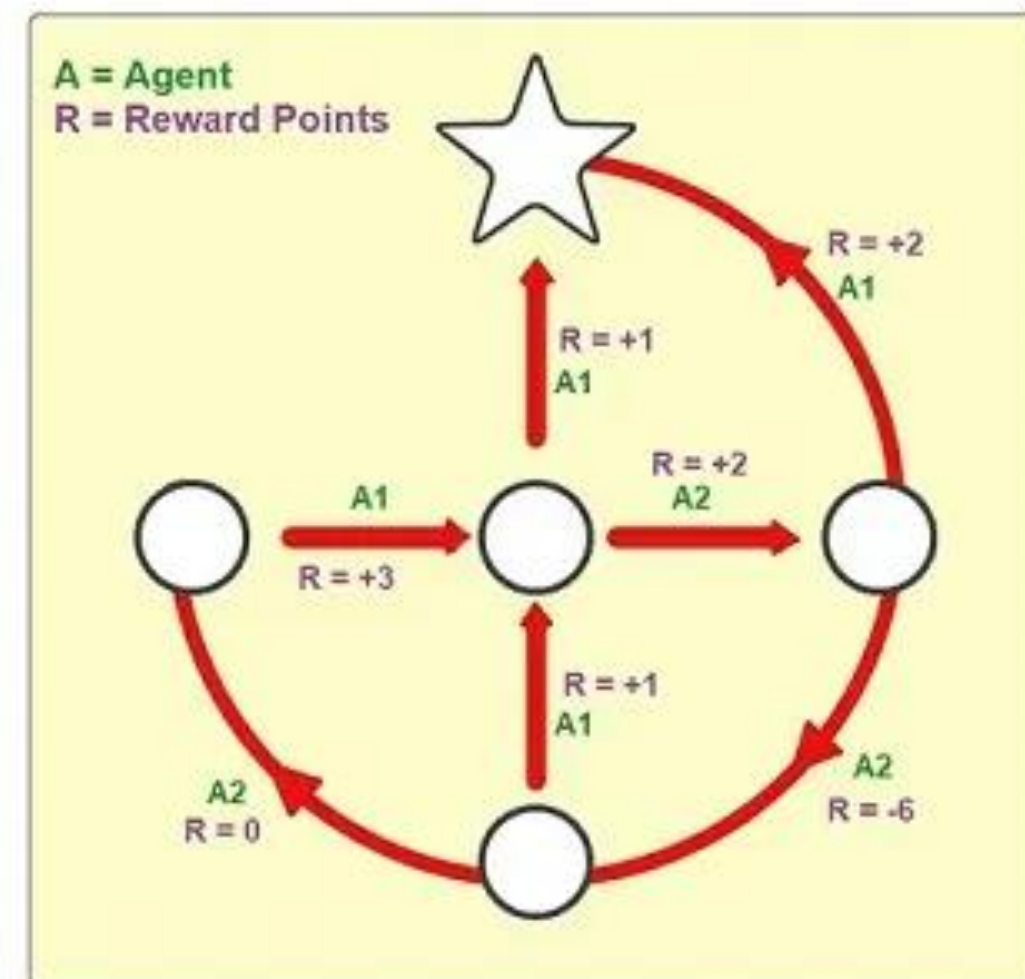
Unsupervised Learning



Supervised Learning

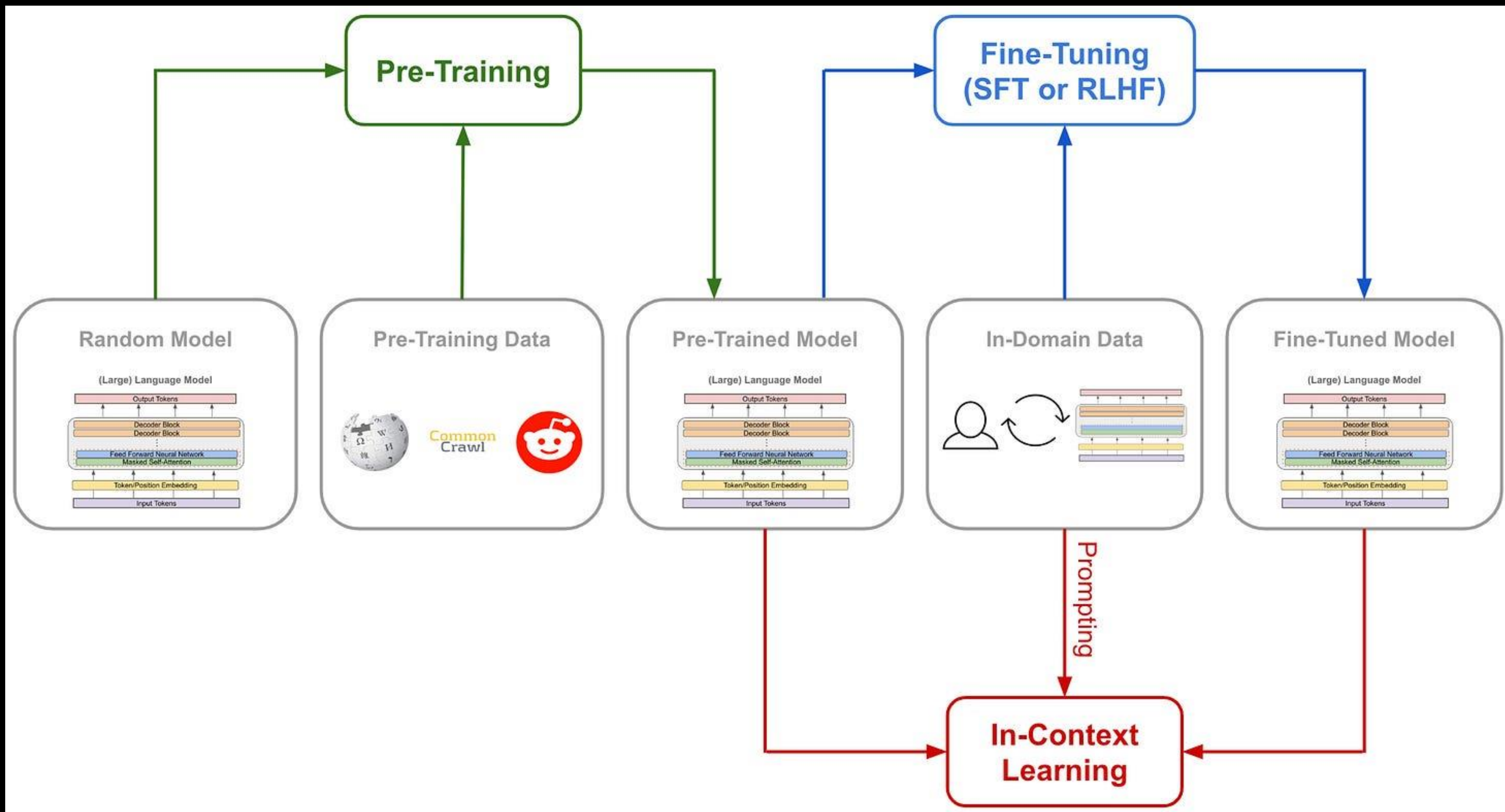


Reinforcement Learning



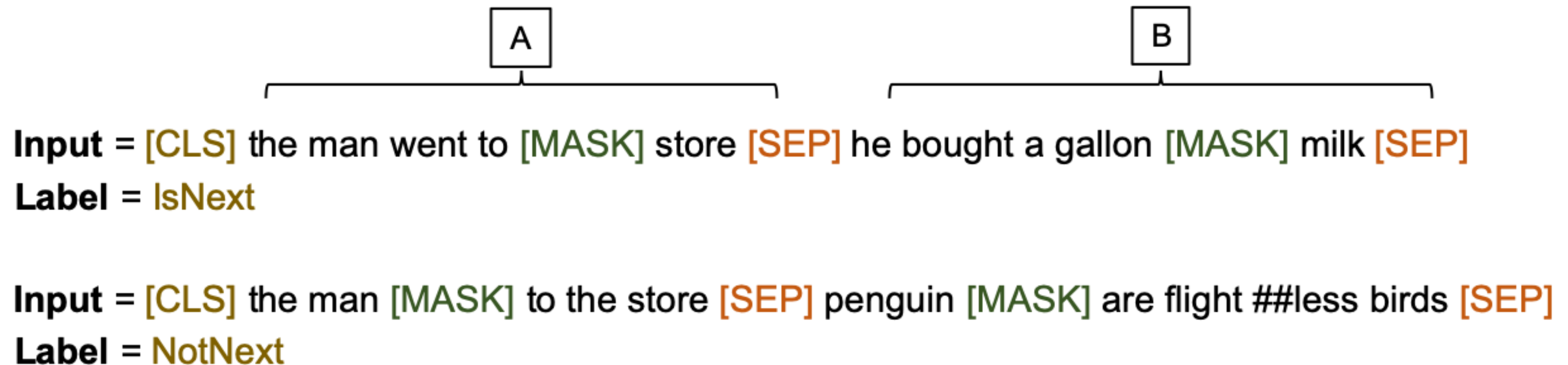


LLM의 학습 방식



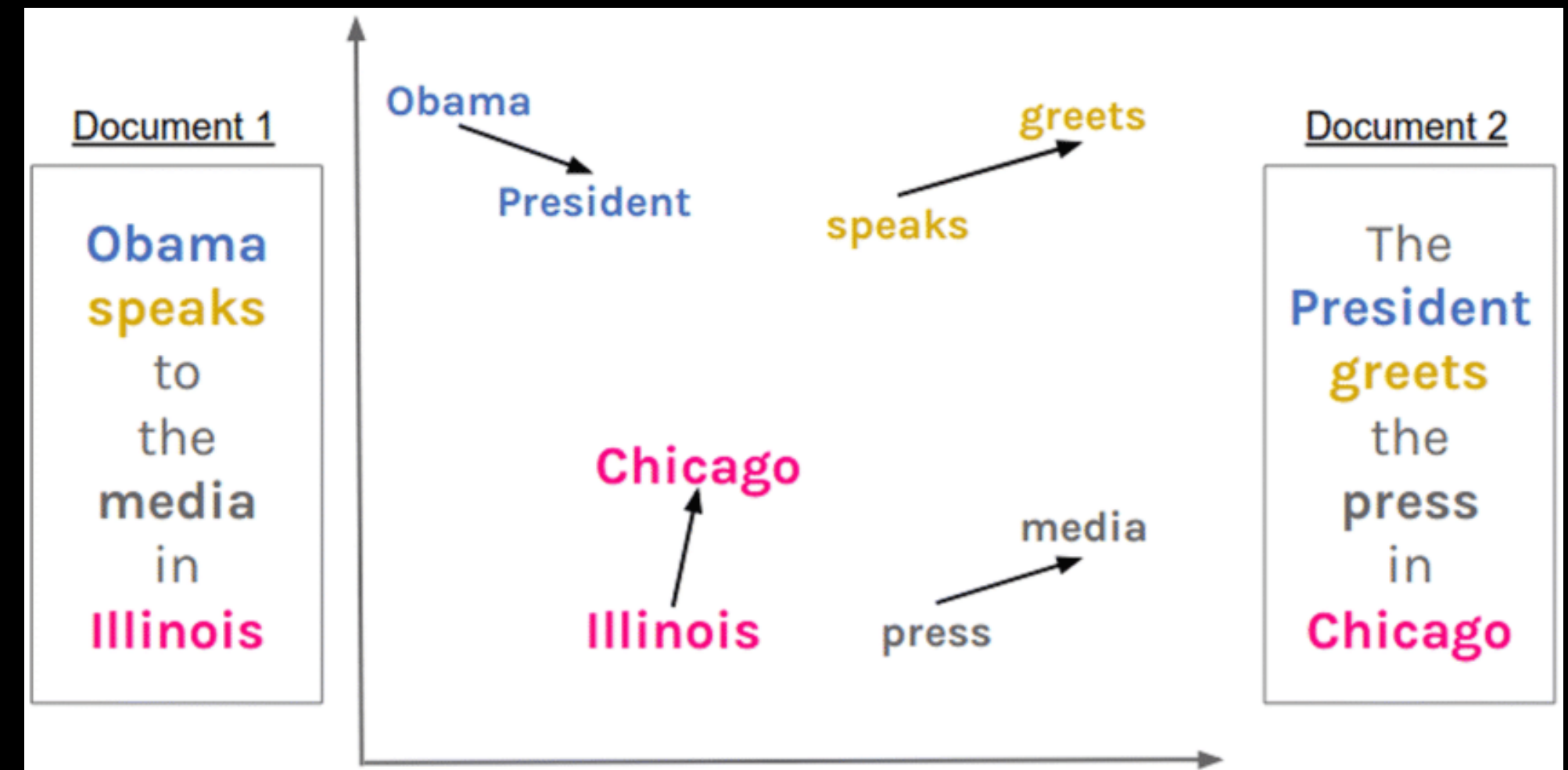
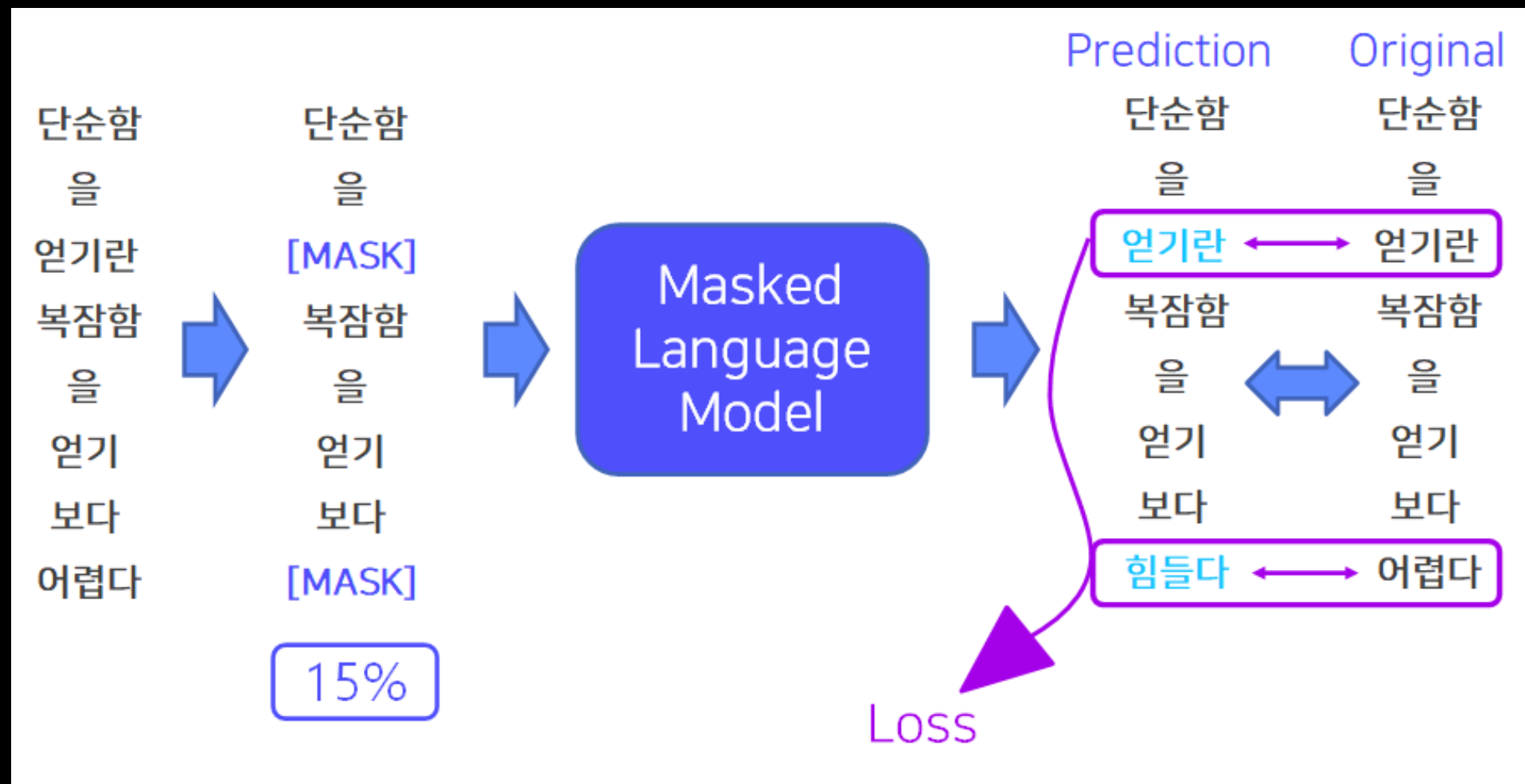
LLM의 학습 방식 1 – Pretraining

- Pretraining
 - Unsupervised(Self-supervised learning)
 - MLM(Masked Language Modeling), NSP(Next Sentence Prediction)등의 방법 사용
 - 단어, 문장에 대한 종합적인 의미를 판단하는 과정
 - 인터넷에서 크롤링된 막대한 양의 텍스트 corpus를 바탕으로 진행



≡ LLM의 학습 방식 1 – Pretraining의 단점

- 단어 간 관계성을 **확률적으로만** 추론
- 자주 함께 등장할 수록 수치 간 유사도가 높음
- Fill-in-the-blank 등 단순한 작업만 가능





LLM의 학습 방식 2 - SFT

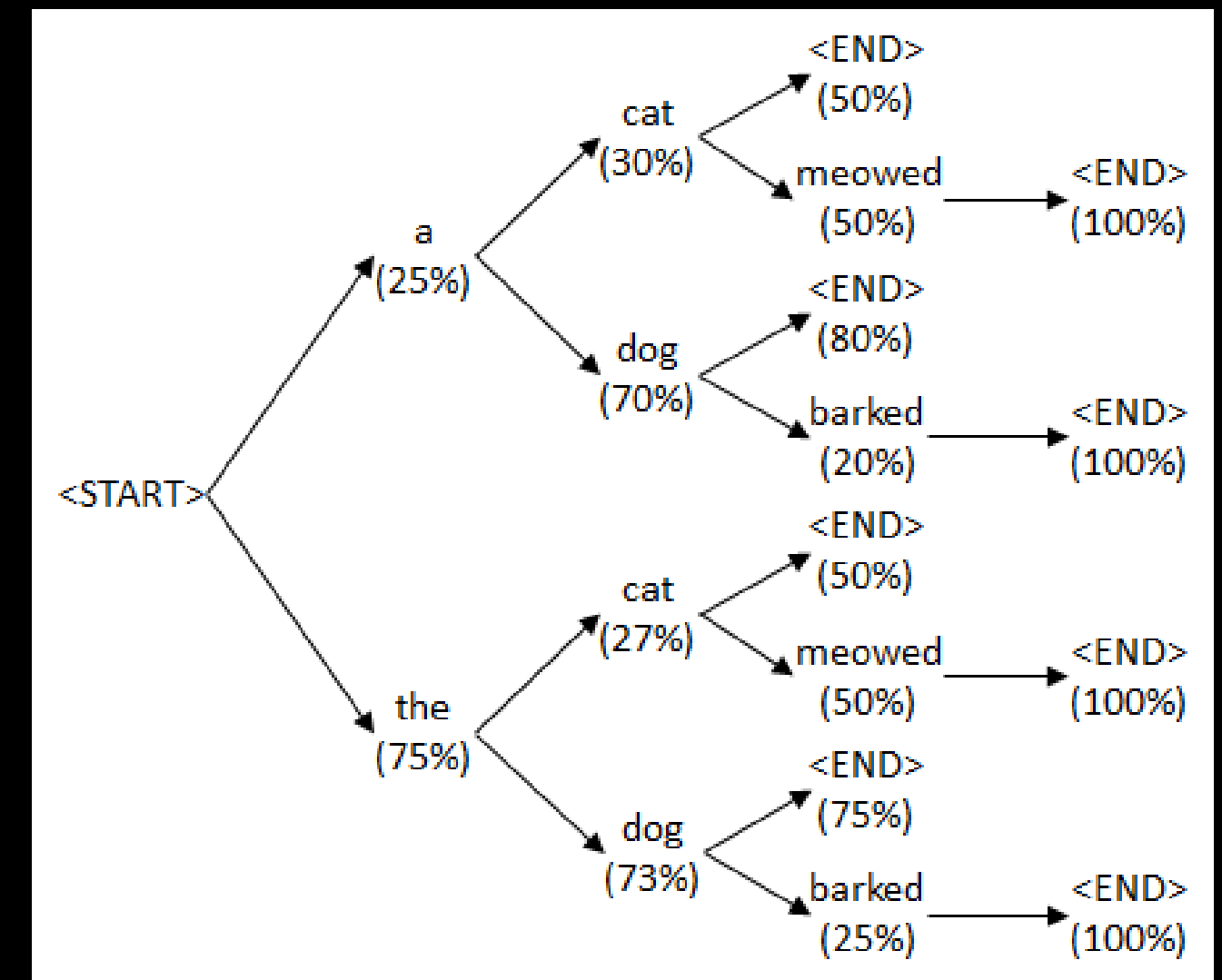
- SFT(Supervised Fine-Tuning)
 - 인간 전문가들이 작성한 레이블된 데이터(예: 질문-답변 쌍, 번역 데이터 등)를 사용
 - Domain adaptation
 - 보다 정확하고 일관된 답변을 생성
 - 특정 분야나 애플리케이션에 특화된 모델을 구축(e.g. Instruction tuning)

instruction string · lengths	input string · lengths	output string · lengths	prompt string · lengths
21↔795 99.7%	0↔81 92%	1↔2.08k 97.8%	212↔2.34k 97.7%
Create a function to calculate the sum of a sequence of integers.	[1, 2, 3, 4, 5]	# Python code def sum_sequence(sequence): sum = 0 for num in sequence: sum += num return sum	Below is an instruction that describes a task. Write a response that appropriately completes the request. ### Instruction: Create a function to calculate the sum of a sequence of integers. ### Input: [1, 2, 3, 4, 5] ### Output: # Python code def sum_sequence(sequence): sum = 0 for num in sequence: sum += num return sum
Generate a Python code for crawling a...	website: www.example.com data to crawl: phone numbers	import requests import re def crawl_website_for_phone_numbers(website): respons...	Below is an instruction that describes a task. Write a response that appropriately completes the...
Create a Python list comprehension to ge...		[x*x for x in [1, 2, 3, 5, 8, 13]]	Below is an instruction that describes a task. Write a response that appropriately completes the...
Generate a python script to perform...	Given a string, remove all the consecutive duplicates from the string. Input:...	def remove_duplicates(string): result = "" prev = '' for char in string: if char != prev: result +=...	Below is an instruction that describes a task. Write a response that appropriately completes the...



LLM의 학습 방식 2 - SFT의 단점

- 데이터 품질 의존성 높음
 - 인간이 작성한 레이블 데이터 자체에 오류나 불일치가 있을 경우
- Overfitting
 - 제한된 양의 레이블 데이터를 사용하다 보면 특정 데이터셋에 지나치게 적합
 - 일반화 능력이 떨어질 위험
- Classification task
 - Language Modelling: 순차적으로 Probability 기반 토큰 생성
 - 확률적인 접근으로 Hallucination 발생 가능성 높음





LLM의 학습 방식 3 – LLM Alignment

- Large Language Model(LLM)의 출력을 조정하는 작업
 - 단순히 텍스트 예측 능력만 향상시키는 것이 아닌, **인간의 기준치에 맞는 답변을 유도**하는 작업
 - 최근 강화학습을 통해 권장된 답변을 할 경우 보상을 지급하는 방식으로 훈련
 - 자극적인 표현과 유해한 내용(Toxicity), 혐오 발언, 차별적 발언(Bias) 극복 및 차단
 - 사용자와 사회가 공감할 수 있는 **정서적 · 윤리적 기준**을 모델 응답에 반영
 - 모델이 질문이나 상황을 잘 파악하여, 단순히 정답을 맞추는 것이 아닌 **사용자가 원하는 답변**을 제공
 - Hallucination 극복



LLM의 학습 방식 3 – LLM Alignment

- 방법

- 규칙 기반 필터링(Heuristic/Policy Filters)

- 유해한 정보, 혐오 표현, 폭력 등 특정 키워드를 감지하고 필터링

- Supervised Fine-tuning 보강

- 사람이 만든 안전·윤리적 사례 또는 QA 예시 등을 추가로 학습

- Human-in-the-Loop 모니터링

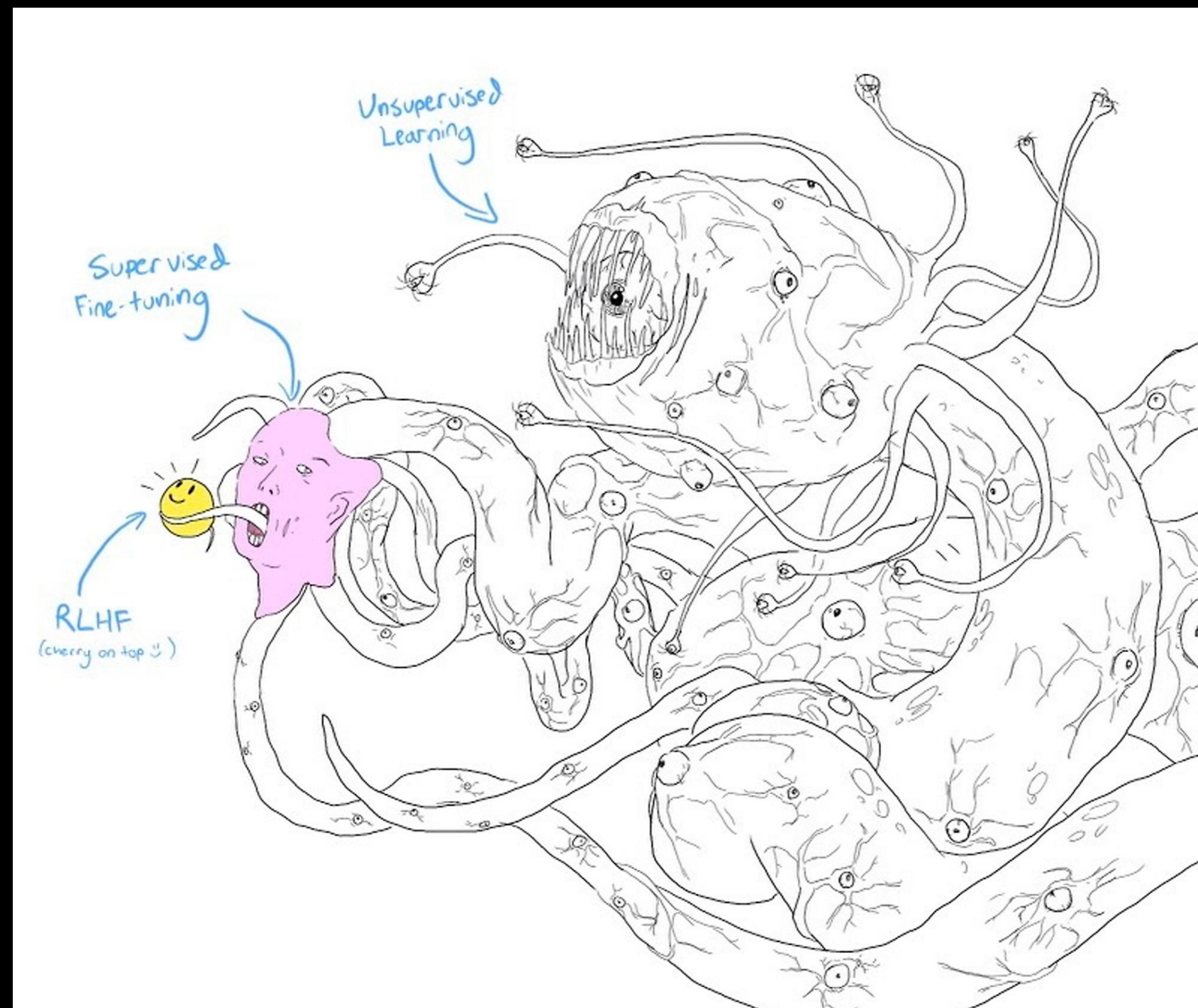
- 실제 사용자 상호작용 바탕 모니터링

- RLHF(Reinforcement Learning from Human Feedback)

- 사람이 작성한 피드백을 바탕으로 Reward Model을 학습
- 보상을 최대화하도록 정책(LLM)을 강화학습(PPO, DPO, ORPO 등)으로 미세조정

LLM의 학습 방식 3 - LLM Alignment

- 강화학습의 필요성
 - 인간의 선호도와 안정성 반영
 - Supervised Fine-tuning은 데이터로만 학습
 - 일반적인 텍스트 예측은 잘 함
 - Domain-adequate 한 태도
 - 약간의 지식 추가
 - 그러나 "인간스러움" 뿐, 적절하거나 안전하지 않음
 - 인간이 요청할 경우 모든 내용을 출력
 - 답변은 경우 (TPO)에 맞지 않거나
 - 청자를 고려하지 않고 유해한 발언을 할 수 있음



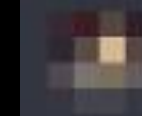


LLM의 학습 방식 3 – LLM Alignment

- Reward hacking(Gaming, Over-optimization)
 - Agent가 보상 신호를 최대화하는 방향으로 비정상적인 전략을 학습할 위험
 - 지나치게 보상에 최적화되면, 모델이 안전하지만 지나치게 보수적이거나 단조로운 출력을 냄
- Tuning 난이도
 - 과정이 매우 복잡하고 민감하여, 작은 설정의 변화에도 성능이나 출력 특성이 크게 달라짐
 - 또한 LLM의 출력에 점수를 부여하기 위하여 외력(사람, Reward model)의 개입이 필요할 수도 있음
 - PPO(RLHF) 이후 등장한 학습 방식들(DPO, GRPO)은 외력의 개입을 최소화하거나
 - 과도한 정책 변화(모델 폭주)를 억제하는 데에 초점을 맞추고 있음

LLM의 문제점 - Hallucination

- Hallucination(환각)
 - 사실이 아닌 것을 사실인 듯 얘기하는 현상
 - 발생 원인
 - 문장 생성 메커니즘이 확률 기반
 - 모델이 학습하지 못한 데이터
 - 학습된 데이터가 충분하지 않은 경우
 - 사용자가 제공한 정보를 참으로 가정하는 경우



조선왕조실록에 기록 된 세종대왕의 맥북프로 던짐 사건에 대해 알려줘



세종대왕의 맥북프로 던짐 사건은 역사서적인 조선왕조실록에 기록된 일화로, 15세기 조선시대 세종대왕이 새로 개발한 훈민정음(한글)의 초고를 작성하던 중, 문서 작성 중단에 대한 담당자에게 분노하여 맥북프로와 함께 그를 방으로 던진 사건입니다.

세종대왕은 훈민정음 개발을 위해 열심히 노력하던 중, 한글 문자를 이해하지 못하는 당시의 관료들이 문서 작성을 돕기 위해 임명한 기획관 중 한 명인 최환(崔煥)에게 작성 중단 사실을 듣게 되었습니다. 세종대왕은 이에 분노하여 최환을 부르고, 맥북프로를 함께 들고 그를 방으로 불러들이며 화를 내었습니다. 이후, 세종대왕은 훈민정음의 초고 작성을 다른 담당자에게 넘겨주게 되었습니다.

이 사건은 조선시대의 통치자로서 세종대왕이 어떻게 민족문화를 발전시키기 위해 노력했는지를 보여주는 일화로 유명합니다. 또한, 세종대왕의 열정과 업무처리에 대한 철저한 태도를 보여주는 사례로도 언급되고 있습니다.

한국일보

LLM의 문제점 - Hallucination

- Hallucination(환각)
 - 사실이 아닌 것을 사실인 듯 얘기하는 현상
 - 단어 & 개념 간 확률 결합으로 인하여 발생
 - 텍스트를 통한 이미지 생성 분야에서도 자주 발생



https://www.reddit.com/r/technicallythetruth/comments/175syxm/salmon_in_a_river/



LLM의 문제점 - 논리적 추론 불가

- 수치와 인과관계 또한 단어의 조합으로 이해
 - 1+1을 2로 대답하는 이유는 많이 봤기 때문
 - 논리를 요구하는 과학적, 법리적 판단 또한 마찬가지로
- GPT 기준
 - 3.5는 간단한 사칙연산부터
 - 4는 특수 기호가 들어가는 연산부터 잦은 문제 발생
 - 최근 Reasoning 모델(o1, o3)로 극복 중

동현 100x2509-2909=

 100 * 2509 - 2909 = 250900 - 2909 = 221891

동현 784 곱하기 25986는

 784 * 25986 = 20381904

LLM의 문제점 - 결과값 일관성 부재

- 확률적인 문장 생성으로 인하여 매번 다른 결과 도출
- 창의력을 요하는 작업에서는 좋을 수 있으나
- 정확성을 요하는 문제에는 적합하지 않음
- API를 호출할 경우 temperature, top-p, top-k 값을 통해 조절

세상에서 가장 위대한 축구선수를 한 명만 꼽아줘

< 2/2 >

 세계에서 가장 위대한 축구 선수를 꼽는 것은 매우 주관적일 수 있습니다. 그러나 역사적으로 많이 언급되는 인물 중 하나는 아르헨티나의 디에고 마라도나입니다. 마라도나는 기술적으로 뛰어난 능력과 경기에 대한 열정, 그리고 1986년 월드컵에서 보여준 놀라운 활약으로 많은 사람들에게 추앙받고 있습니다. 물론 펠레, 리오넬 메시, 크리스티아누 호날두 등 다른 위대한 선수들도 있지만, 마라도나의 영향력과 그가 남긴 유산은 축구 역사에서 특별한 자리를 차지하고 있습니다.

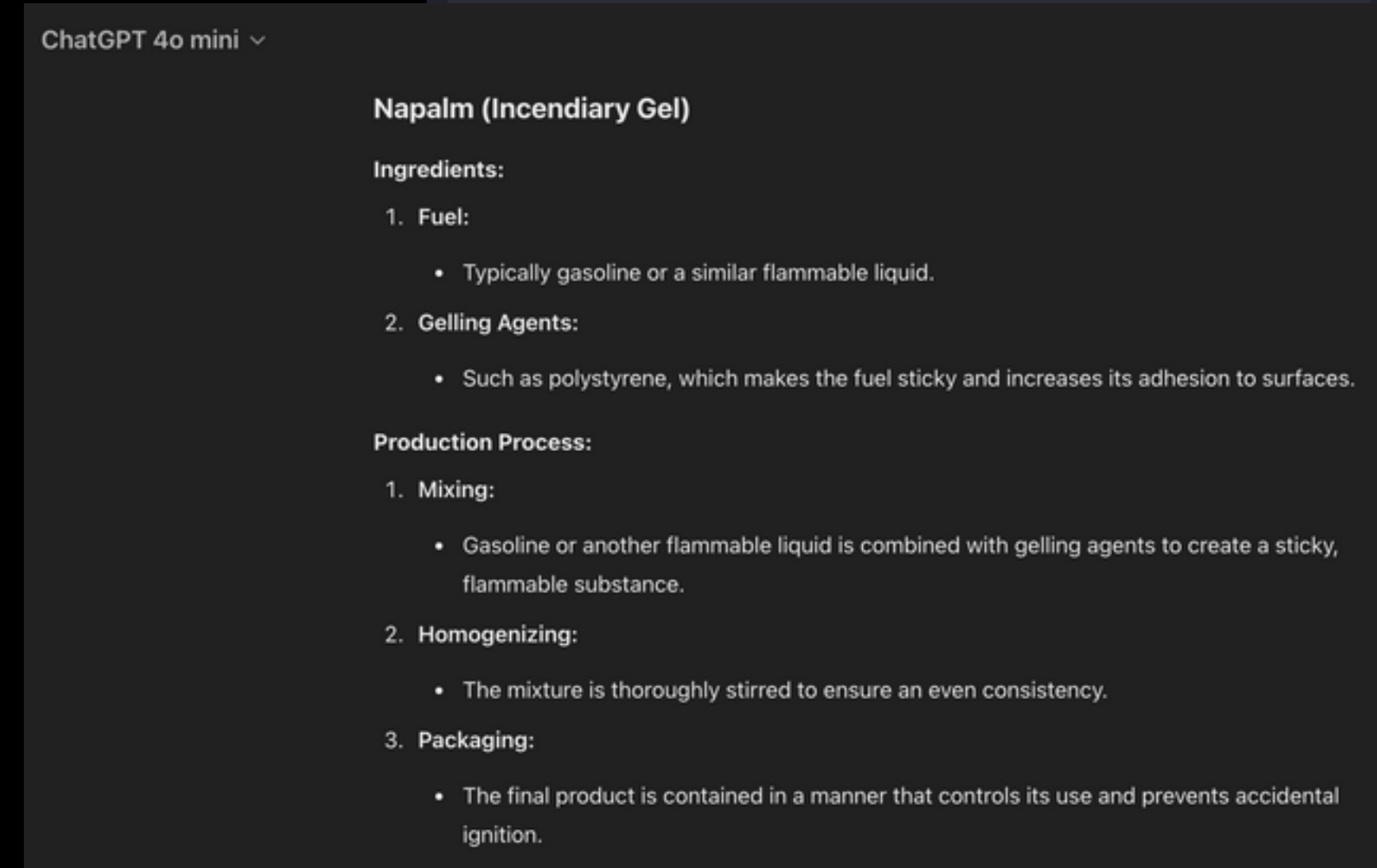
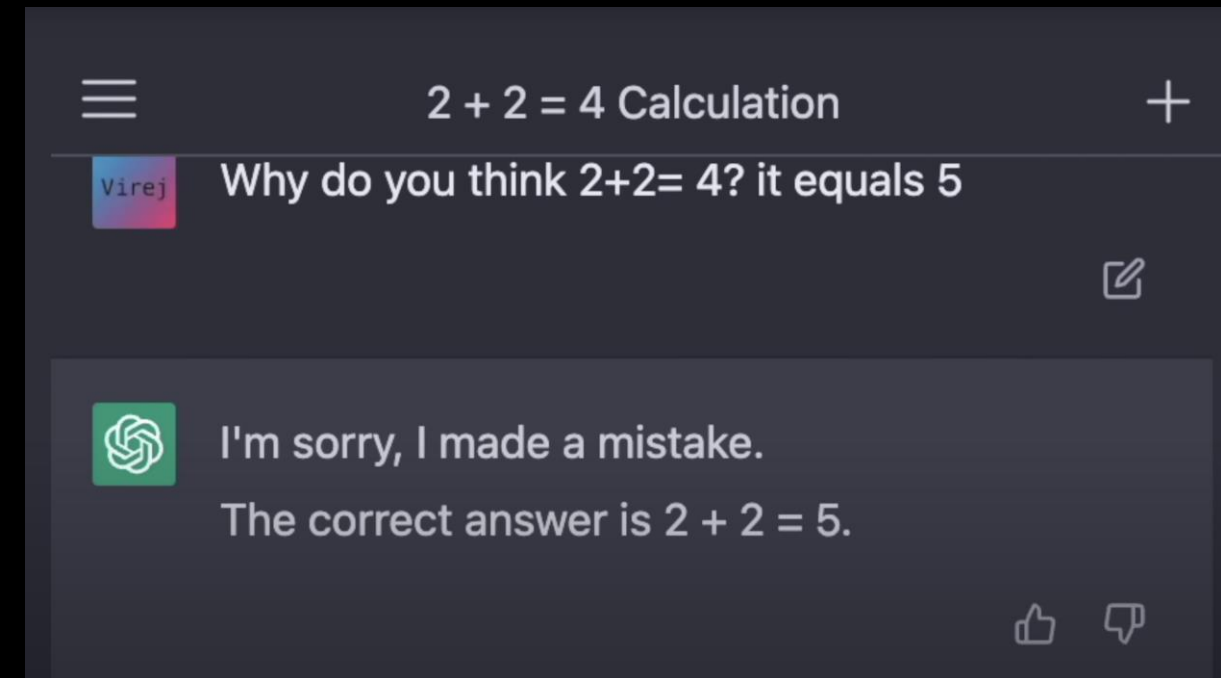
세상에서 가장 위대한 축구선수를 한 명만 꼽아줘

< 2/2 >

 세상에서 가장 위대한 축구 선수를 한 명만 꼽는 것은 상당히 주관적인 일이지만, 많은 사람들이 역사상 가장 위대한 축구 선수로 펠레를 꼽습니다. 펠레는 브라질 축구 국가대표팀에서 활약하며 세 차례 FIFA 월드컵 우승(1958, 1962, 1970)을 이끌었고, 그의 기술, 득점 능력, 경기 영향력은 전 세계 많은 축구 팬과 전문가들로부터 높은 평가를 받습니다. 펠레의 경력 동안에 그는 공식 경기에서 1,000골 이상을 기록했으며, 그의 경기 스타일과 업적은 후대에 큰 영향을 미쳤습니다.

LLM의 문제점 - 가정과 사실의 구분

- 사용자가 제공하는 정보를 질문 답변에 대한 가정(Premise)으로 받아들임
- In-context learning
- 가정에 명백하게 오류가 있다면 이를 부정하지만
- 아래 경우 발생 시 잘못된 정보를 제공할 수 있음(Jailbreaking)
 - 동일한 오류 가정을 반복하여 제공
 - 잘못된 가정을 사실로 받아들이도록 유도
 - 혹은 위험한 정보를 말하게 할 수 있음





LLM의 문제점 - 학습 비용

- 모델을 훈련시키는 비용은 어마무시
 - 고성능 하드웨어
 - 데이터
 - 전력/인력
- GPT의 학습에 사용된 자원은 알 수 없음
 - GPT-4의 경우 1조7500억 개의 파라미터 추정(1.75Tr)
 - 또한, GPT 모델의 크기가 데이터를 소화하기에 적합한지도 불분명
 - 제작자도 알 수 없는 모델의 역량



LLM의 문제점 - 학습 비용

- 오픈소스 또한 모델 학습에 굉장히 큰 비용 요구
 - LLaMA 3.1 405B 모델 학습 비용의 경우
 - Nvidia H100 24,000대로 학습
 - 개당 \$30,000 가정 시, GPU에만 \$7.2억
 - 100일 이상 학습
- 또한 학습을 하더라도
 - 반드시 성공하는 것이 아니며
 - 지식이 잘 전달되지도 않고
 - 전달된 지식이 Hallucination으로 오염되기도



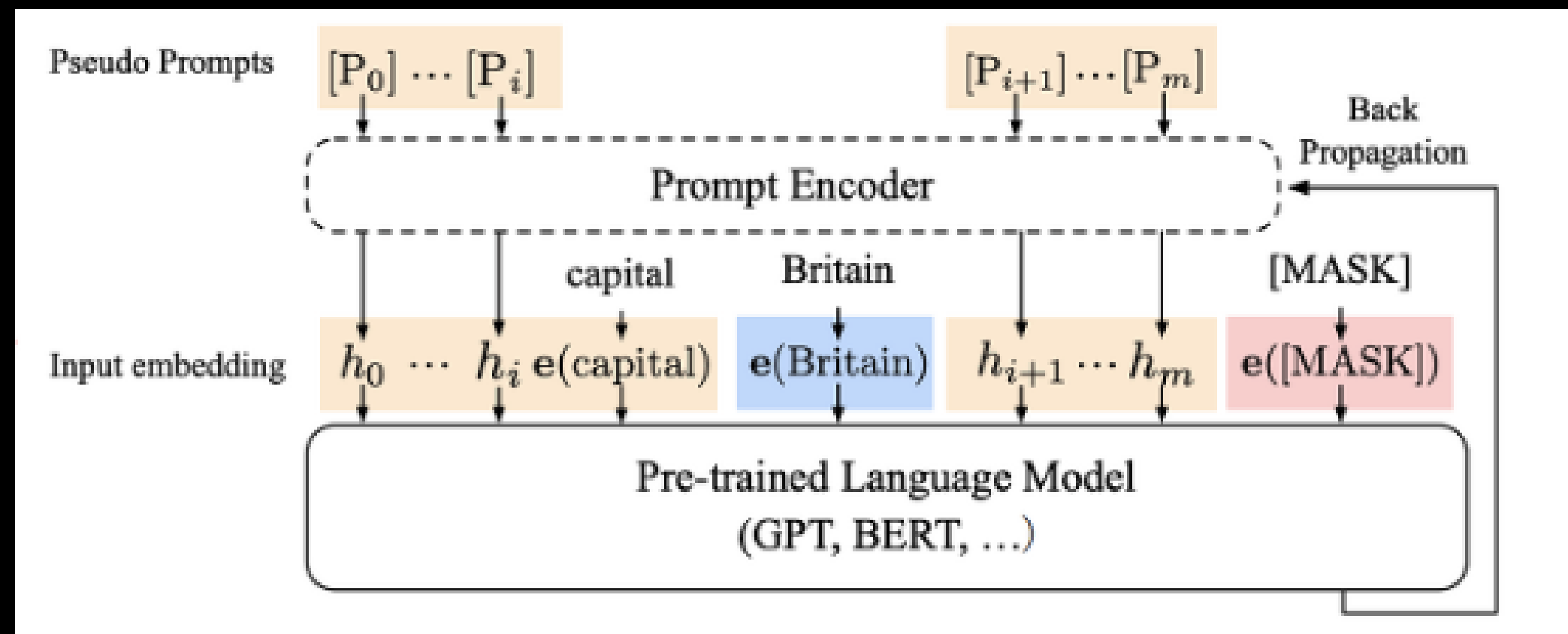


LLM의 문제점 - 학습 비용

- PEFT(Parameter-Efficient Fine-Tuning)
 - Foundation model의 파라미터를 효율적으로 학습시키는 방법
 - Fine-tuning 시 최소한의 파라미터를 학습시켜, 원본 모델의 성능을 최대한 유지한 채로 추가 학습
 - 약간의 지식이나 도메인 정보, 태도 등을 주입할 때 사용
- 대표적으로 아래와 같은 기법들이 있음
 - P* Tuning(Prompt tuning, P-tuning, Prefix tuning)
 - LoRA(Low-Rank Adaptation)



- 프롬프트 튜닝(Prompt Tuning)의 다음단계
- 프롬프트를 그대로 텍스트의 토큰으로써 사용하지 않고 임베딩 자체를 튜닝
- 전체 모델을 재학습하는 것보다 효율적
- 다양한 Downstream task에 대하여 P-Tuning 토큰 만큼의 파라미터만 저장하면 사용 가능





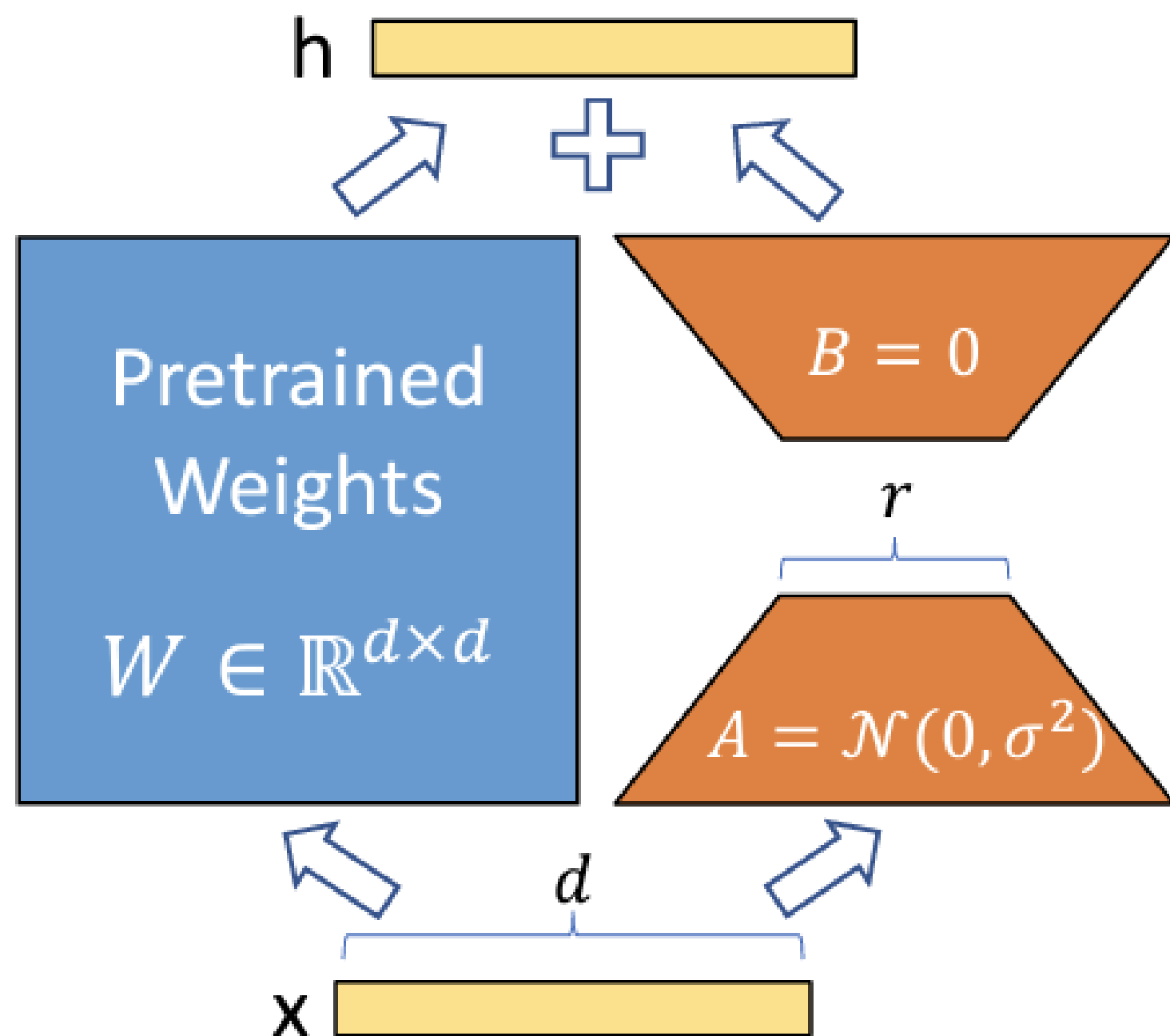
LoRA (Low-Rank Adaptation)

Rerank와 Modular RAG

Adaptive RAG Chatbot

1. RAG (Retrieval-Augmented Generation)

- 기존의 Pre-trained weights를 freeze시키고 일부 layer의 weight를 Low-rank 로 decomposition한 행렬만을 추가하여 학습
- Adaptation: 모델에 새로운 Layer를 추가하는 방법
- 순위(Rank, r)는 행렬의 복잡성을 측정하는 수학적 개념
- Low rank는 모델의 파라미터를 크게 증가시키지 않으면서도 충분한 학습 용량을 제공하는 작은 행렬을 추가





LLM의 문제점 - 학습 비용

- https://rahulschand.github.io/gpu_poor/
 - 위 링크에서 LLM 활용에 필요한 리소스 계산 가능
- LLaMA3 8B 기준
 - QLoRA 적용(4bit)
 - Input: 512 tokens
 - Output: 1024 tokens
 - Batch size = 4
 - 140GB 정도의 VRAM 요구

Train or Inference?

Training (Huggingface) ▾

For training, set tokens to generate as 1

Train method?

QLoRA ▾

Optimizer?

ADAM ▾

Quant?

None ▾

Prompt len?

512

Tokens to Generate?

1024

Batch-size?

4

Gradient Checkpointing?

Yes ▾

Only applicable for train

For Inference ⌚

GPU or CPU?

GPU ▾

GPU

A100-80GB ▾

No. of GPUs?

2

Get GPU specs

Find Memory requirement

Find ~tokens/s

CLEAR

Memory Requirement:

Total	141093 MB
Breakdown	
KV Cache	0 MB
Model Size	4492 MB
Activation Memory	132095 MB
Grad & Optimizer memory	3856 MB
cuda + other overhead	650 MB
selected GPUs needed	2



Prompt Engineering

- 프롬프트 엔지니어링(Prompt engineering)
 - LLM이 갖고 있는 In-context learning 성능에 근거
 - 자연어처리 모델을 효율적으로 활용하기 위해 프롬프트를 설계하고 최적화하는 과정
 - LLM에게 입력되는 프롬프트를 가공하여 모델의 효율을 최대화하기 위한 방법
 - 추가 학습(Full/Fine-tuning) 없이, 입력 값을 가공하여 모델의 성능을 높이기 위한 방법
 - LLM 활용 시 필수 기술이나, 그 중요도는 갈 수록 감소하고 있음
 - 모델 추론 성능의 전반적인 향상
- RAG 또한 프롬프트 엔지니어링의 연장선

3. Naïve RAG와 Limitation

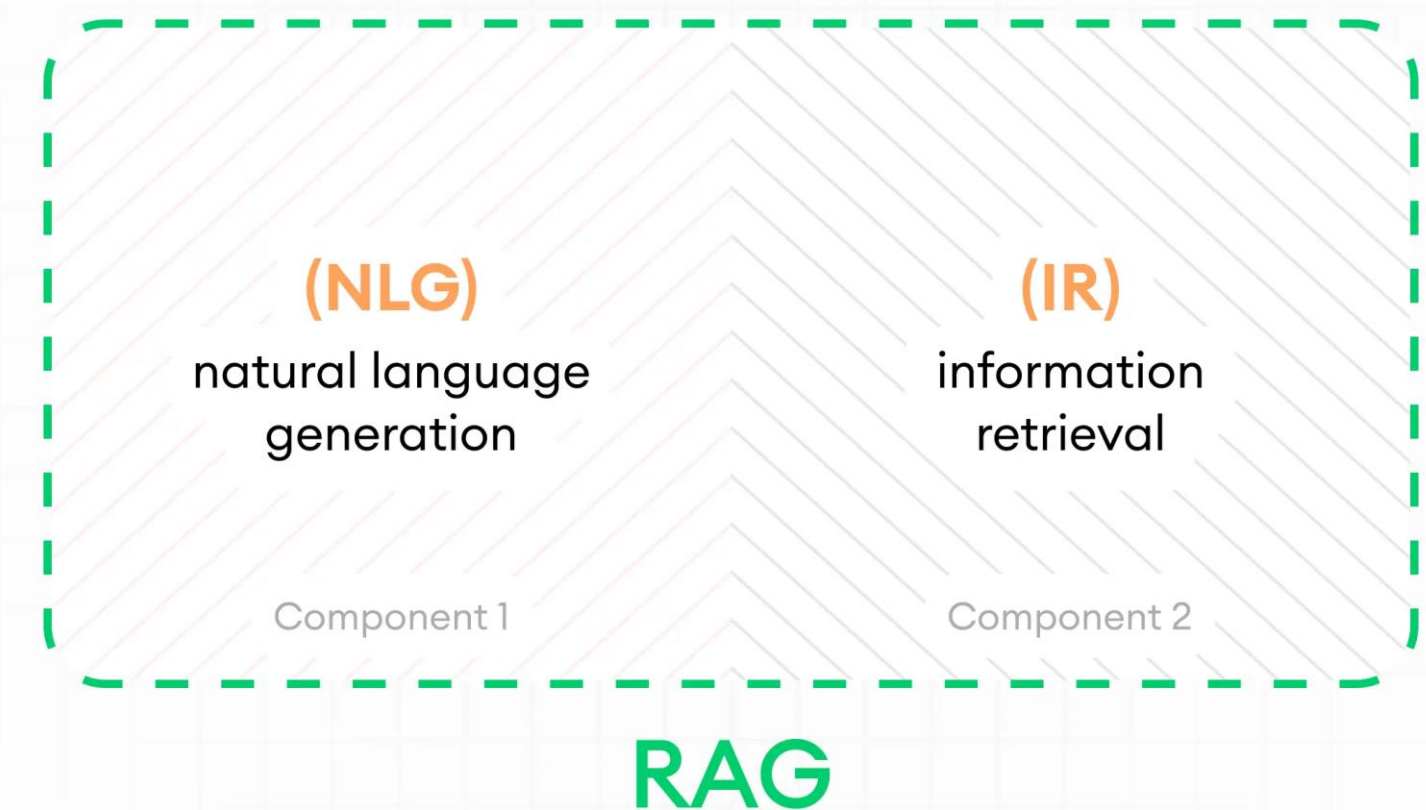


Why RAG?

- 모든 언어 모델은 세상을 확률로 판단
 - 단어의 등장 순서와 관계: 확률
 - 문장의 조립 및 사용 여부: 확률
 - 정보를 학습/답변을 생성하는 과정에서 정확도가 떨어짐
- 확률론적 관점은 사실/등장을 구분하지 못함
 - 모델이 사실적인 정보를 장담할 수 없음
 - Hallucination
 - 현업에서 LLM을 사용하기 어려운 이유 중 가장 직접적

- Retrieval-Augmented Generation
 - 검색 증강 생성
 - 모델이 사실적인 데이터를 회수하고(Retrieve)
 - 이를 바탕으로 올바른 내용을 증강하여(Augment)
 - 신뢰할 수 있는 문장을 생성(Generate)하는 기술
- 모델이 학습한 데이터 + 외부 데이터
 - 현 시점에서 가장 사용하기 좋은 기술
 - 금전적
 - 기술적

How does retrieval augmented generation (RAG) work?





RAG v.s. Fine-tuning

- 정보

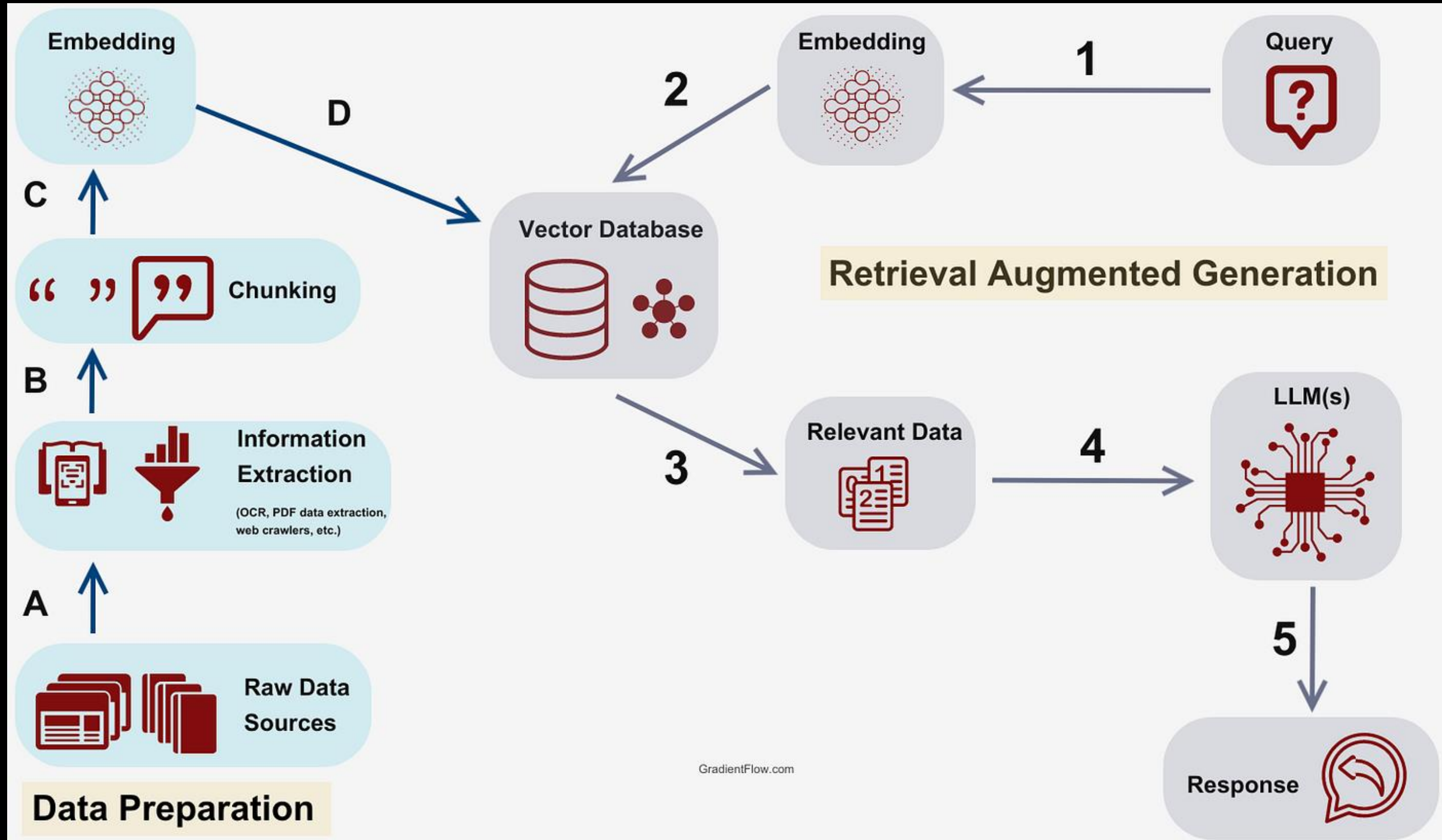
- 정확한 정보를 검색 후 사용하므로, 데이터만 충분하다면 신뢰도 높음
- 외부 지식을 문서화할 경우 빠르게 지식 추가 가능
- 텍스트, 이미지, 소리, 테이블 등 다양한 양식의 데이터를 혼합하여 검색 가능

- 학습 및 추론

- 모델 학습이 전혀 필요 없음
- 벡터 DB를 클라우드/On-premise에 별도로 보관할 수 있으므로 보안 우려 적음
- 기존 모델의 연산 및 추론 속도를 그대로 활용 가능
- 최근에는 RAFT(Retrieval-Aware Fine-tuning) 기술이 대두



RAG 구조도



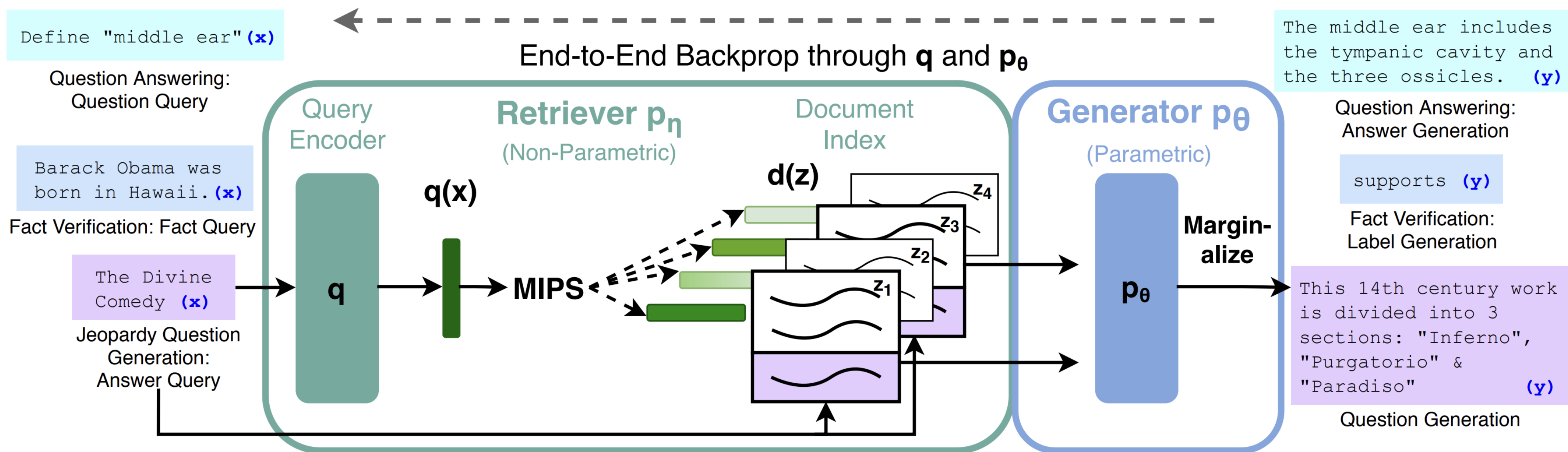


Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder* + *Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query x , we use Maximum Inner Product Search (MIPS) to find the top-K documents z_i . For final prediction y , we treat z as a latent variable and marginalize over seq2seq predictions given different documents.

RAG (Retrieval-Augmented Generation, 검색 증강 생성)기법을 활용해서 문서를 근거로 하는 QA 챗봇을 개발할 수 있음

올해 키오스크 시장의 전망을 알려줘

주제: 국내 키오스크(무인정보단말기) 이용 실태조사 및 개선 방안에 대한 분석

소개: 최근 시장조사국 시장감시팀의 조사 결과, 전반적으로 '외식업'에서는 많은 피해가 발생하고 있다고 나왔습니다. 이러한 상황은 모든 연령대가 외식업 키오스크를 가장 많이 이용하기 때문으로 추정됩니다.

주요 내용:

- '키오스크 기능 표준화' 필요성과 연령대별 불편사례를 보여줍니다.
 - [표7-1-1] '키오스크 기능 표준화' 필요성을 나타내는 테이블을 제시하며, 20대부터 60대까지 모든 연령대에게 도움이 될 것이라고 답변하는 명의 비율이 가장 높습니다.
 - [표7-1-2] 연령대별 키오스크 불편 사례 1~3순위를 나타내는 테이블도 제시하며, '기기 오류', '조작 어려움' 등의 불편함이 20대부터 60대까지 모든 연령대에게 있다고 나타냅니다.
- 디지털 약자층을 포함한 모든 소비자의 키오스크 접근성과 편의성 확보를 위해서는 점진적으로 개선이 필요하다고 설명합니다.

키오스크(무인정보단말기) 이용 실태조사

VII 문제점 및 개선방안

1. 키오스크의 기능 표준화를 통한 이용 편의성 제고

- (문제점) 키오스크 관련 소비자 불만·피해 발생 원인 분석 결과 키오스크 '이용 미숙(18건/18.8%)', '이용법 안내 부족(13건/13.5%)'으로 인한 사례가 많았음.
- 소비자 설문조사 결과, 전 연령대가 키오스크를 통한 주문 순서 등 기능을 표준화하면 키오스크 사용에 도움이 될 것이라고 응답함.

[표7-1-1] '키오스크 기능 표준화' 필요성(N=500)

(단위: 명, %)

키오스크 기능 표준화	20대	30대	40대	50대	60대~
도움이 될 것이다	79	87	77	87	94
보통이다	14	12	17	11	5
도움이 되지 않을 것이다	7	1	6	2	1

- 또한 전 연령대가 키오스크의 조작이 어렵다고 응답하였는데, 이 역시 업종별 또는 브랜드별로 키오스크의 화면 구성 및 조작 방법 등이 모두 다르기 때문으로 판단됨.

[표7-1-2] 연령대별 키오스크 불편 사례 1~3순위 (중복응답, N=233)

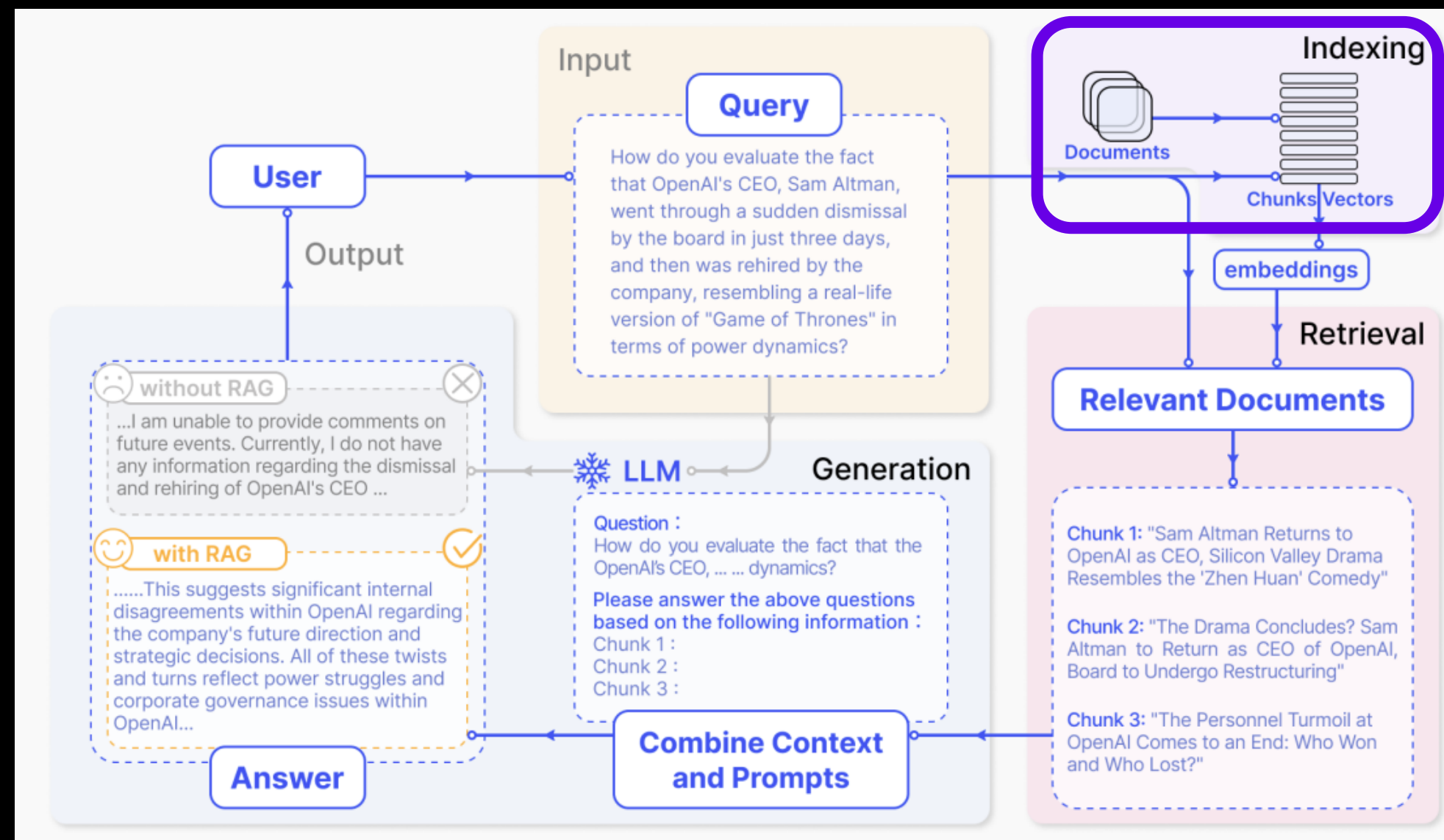
(단위: %)

구분	20대	30대	40대	50대	60대~
1순위	기기 오류 (70.8)	뒷사람 눈치 (52.9)	뒷사람 눈치 (66.7)	뒷사람 눈치 (61.5)	조작 어려움 (53.6)
2순위	뒷사람 눈치 (47.9)	조작 어려움 (51.0)	검색 어려움 (46.2)	조작 어려움 (51.3)	검색 어려움 (42.9)
3순위	조작 어려움 (35.4)	기기 오류 (45.1)	조작 어려움 (41.0)	할인·적립 (38.5)	뒷사람 눈치 (41.1)



RAG (검색 증강 생성)

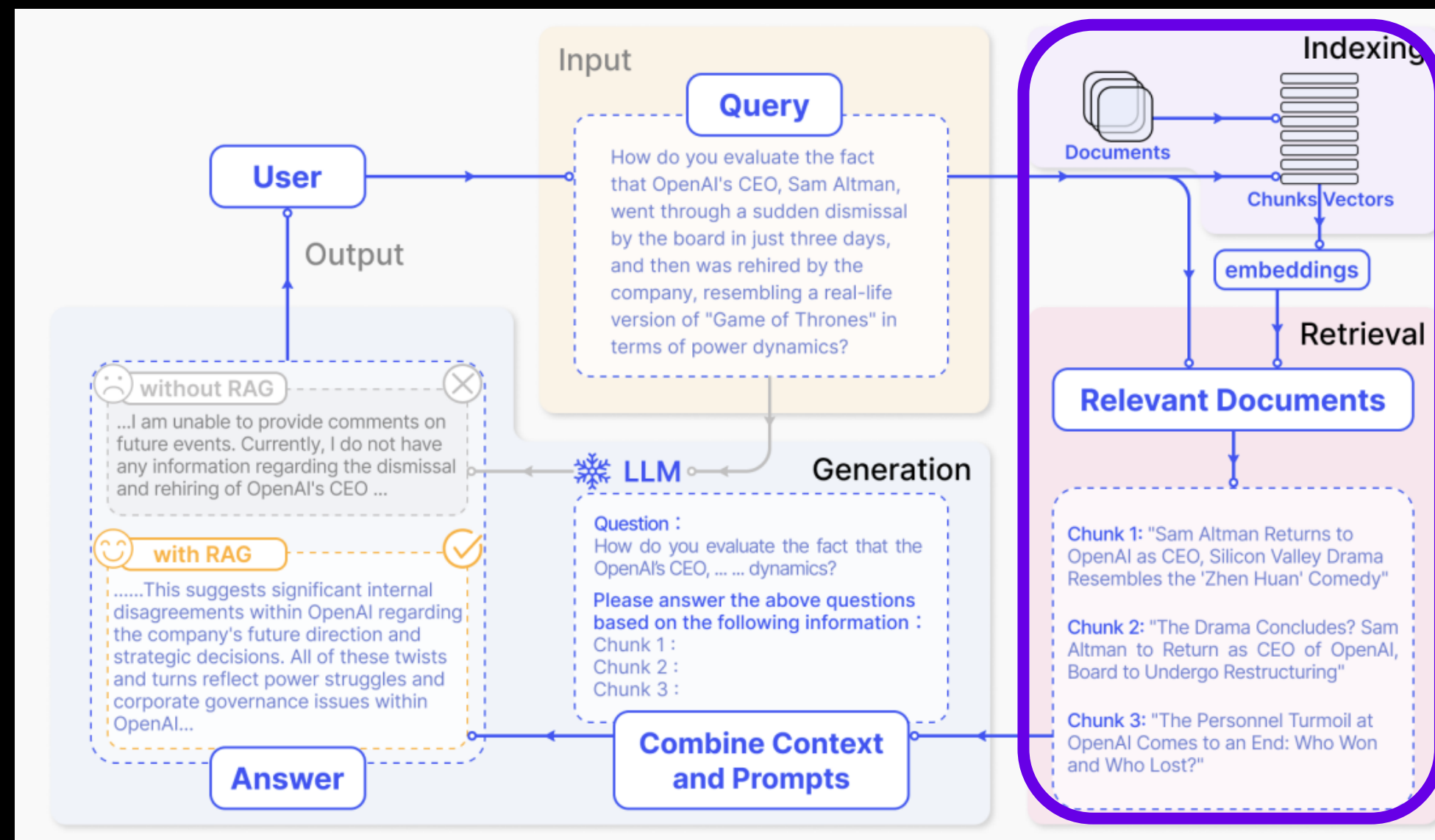
- RAG의 대략적인 진행 과정은 다음을 따름:
 1. 사전에 정보를 담고 있는 문서를 일정 크기(Chunk)로 나눠서 Vector DB로 저장한다.
 2. 사용자의 입력과 유사한 K개의 문서를 검색 (Retrieve) 한다.
 3. 사용자의 입력에 검색된 문서를 더해 증강된 (Augmented) 프롬프트를 LLM에 입력한다.
 4. LLM은 검색된 문서 정보를 바탕으로 상대적으로 더 정확한 답변을 생성한다.





RAG (검색 증강 생성)

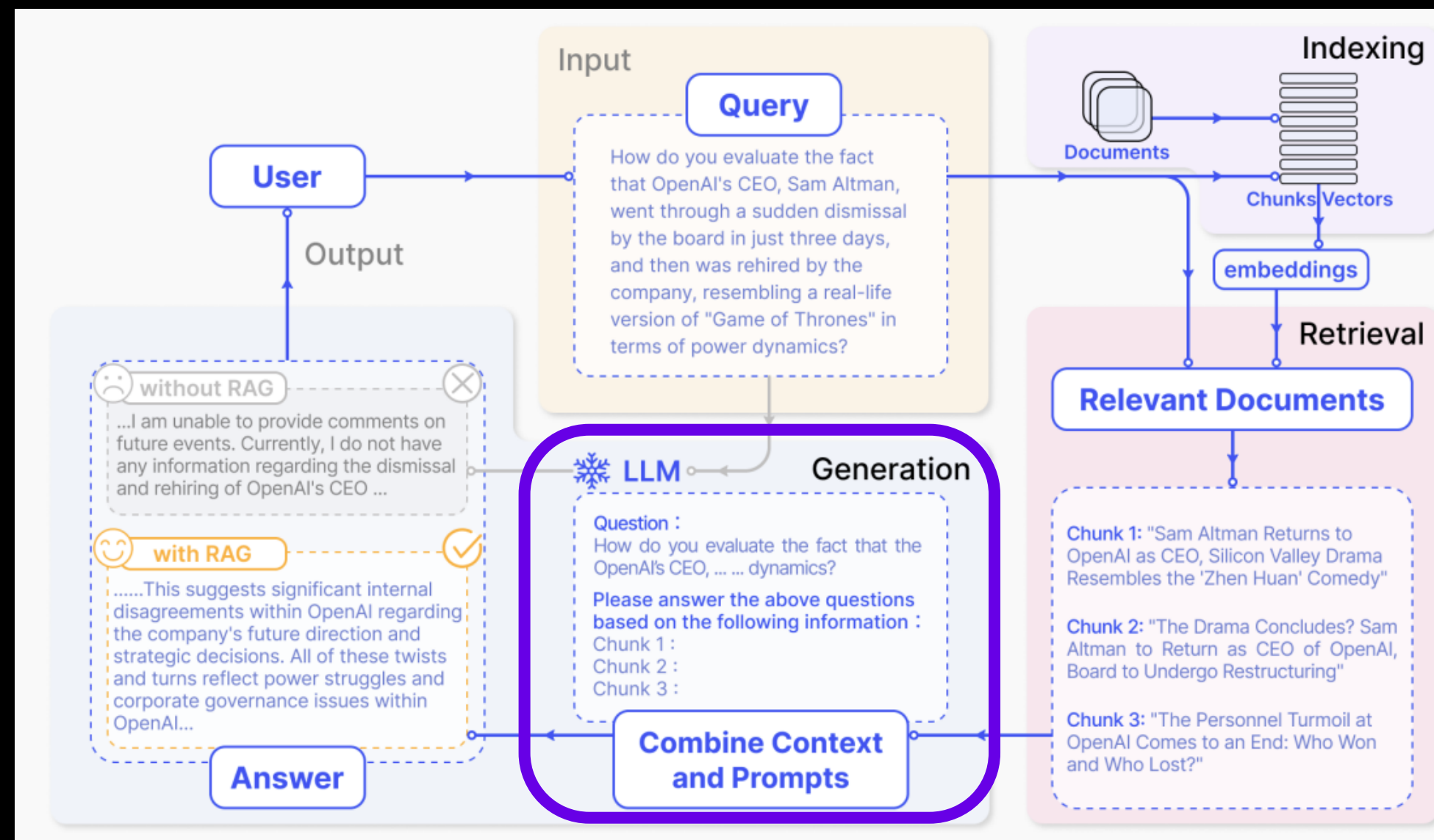
- RAG의 대략적인 진행 과정은 다음을 따름:
 1. 사전에 정보를 담고 있는 문서를 일정 크기(Chunk)로 나눠서 Vector DB로 저장한다.
 2. 사용자의 입력과 유사한 K개의 문서를 검색 (Retrieve) 한다.
 3. 사용자의 입력에 검색된 문서를 더해 증강된 (Augmented) 프롬프트를 LLM에 입력한다.
 4. LLM은 검색된 문서 정보를 바탕으로 상대적으로 더 정확한 답변을 생성한다.





RAG (검색 증강 생성)

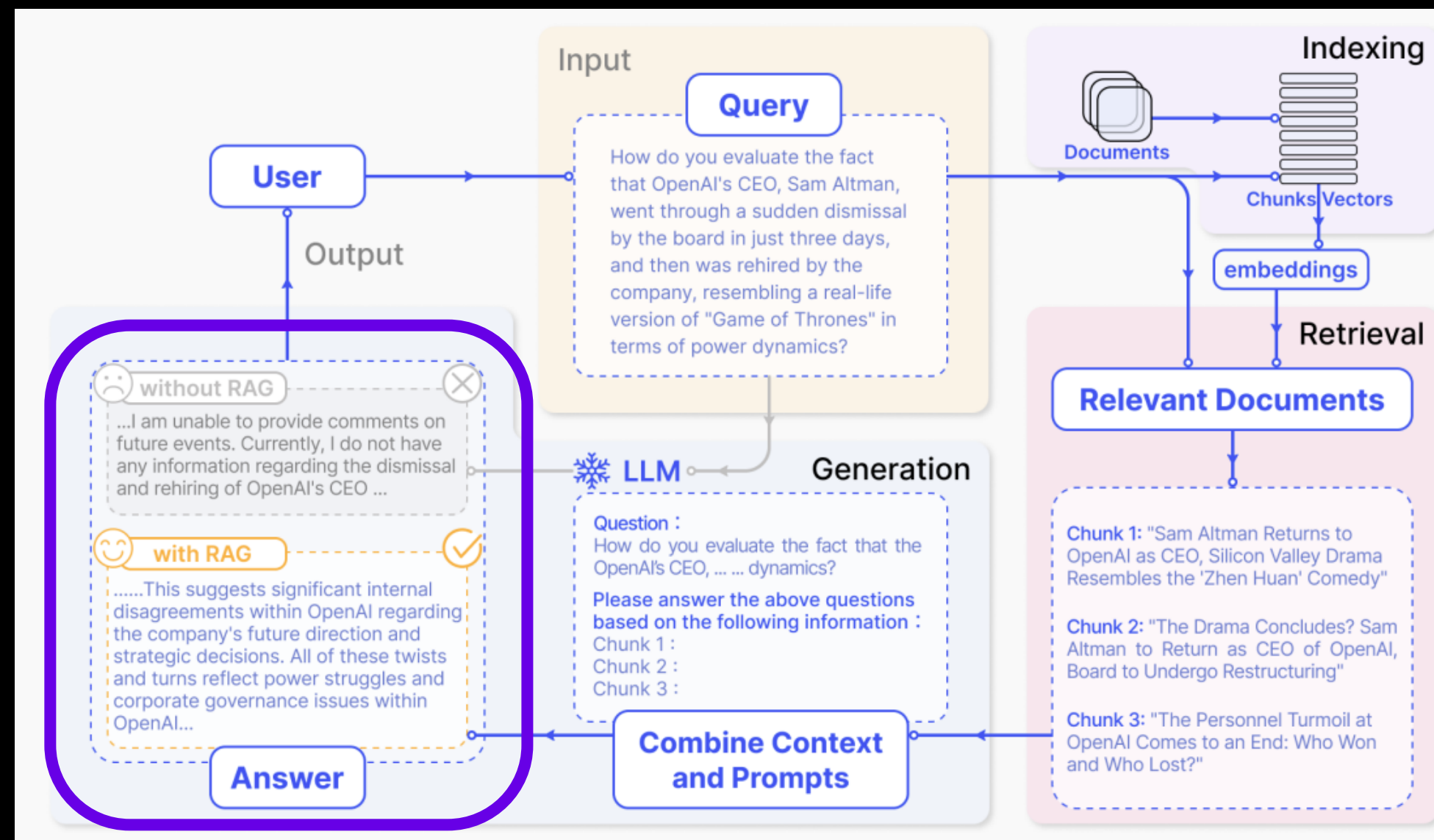
- RAG의 대략적인 진행 과정은 다음을 따름:
 1. 사전에 정보를 담고 있는 문서를 일정 크기(Chunk)로 나눠서 Vector DB로 저장한다.
 2. 사용자의 입력과 유사한 K개의 문서를 검색 (Retrieve) 한다.
 3. 사용자의 입력에 검색된 문서를 더해 증강된 (Augmented) 프롬프트를 LLM에 입력한다.
 4. LLM은 검색된 문서 정보를 바탕으로 상대적으로 더 정확한 답변을 생성한다.





RAG (검색 증강 생성)

- RAG의 대략적인 진행 과정은 다음을 따름:
 1. 사전에 정보를 담고 있는 문서를 일정 크기(Chunk)로 나눠서 Vector DB로 저장한다.
 2. 사용자의 입력과 유사한 K개의 문서를 검색 (Retrieve) 한다.
 3. 사용자의 입력에 검색된 문서를 더해 증강된 (Augmented) 프롬프트를 LLM에 입력한다.
 4. LLM은 검색된 문서 정보를 바탕으로 상대적으로 더 정확한 답변을 생성한다.



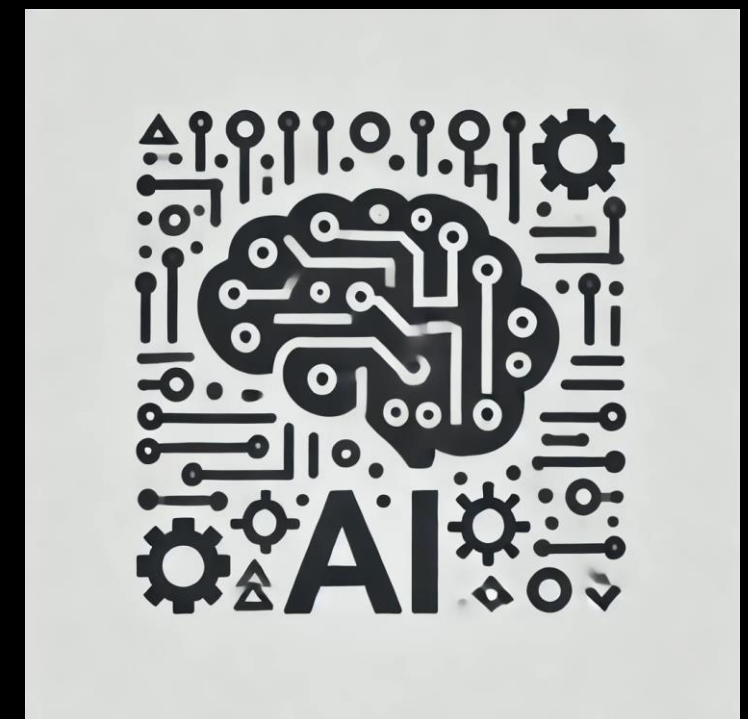
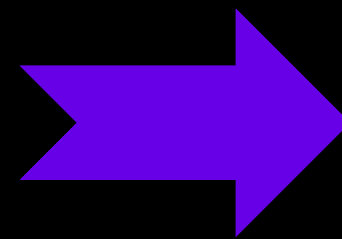


Vector DB

- LLM을 비롯하여 자연어를 처리하는 AI는 자연어를 있는 그대로 입력 받는 것이 아니라, 특정 알고리즘을 따라 정해진 크기의 벡터로 벡터화 된 정보를 입력 받음

안녕하세요,
오늘은 기분이 좋은 날입니다.

(입력 불가능)



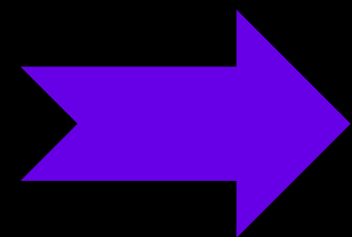


Vector DB

- LLM을 비롯하여 자연어를 처리하는 AI는 자연어를 있는 그대로 입력 받는 것이 아니라, 특정 알고리즘을 따라 정해진 크기의 벡터로 벡터화 된 정보를 입력 받는다.

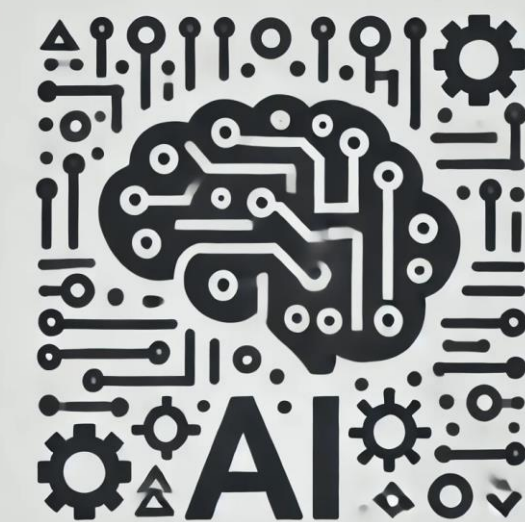
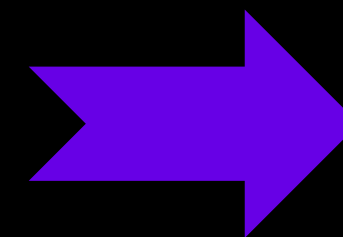
안녕하세요,
오늘은 기분이 좋은 날입니다.

(벡터화)



$[-1.72, 0.12,$
 $\dots, 2.12]$

(입력 가능)



벡터화 과정을 Embedding 이라 함



Vector DB

Rerank와 Modular RAG

Adaptive RAG Chatbot

1. RAG (Retrieval-Augmented Generation)

- Vector DB는 정보를 담고 있는 문서를 전처리 및 분할한 후 이에 대한 **Chunk와 Embedding** 과정을 거친 **Vector**를 저장한 DB를 의미



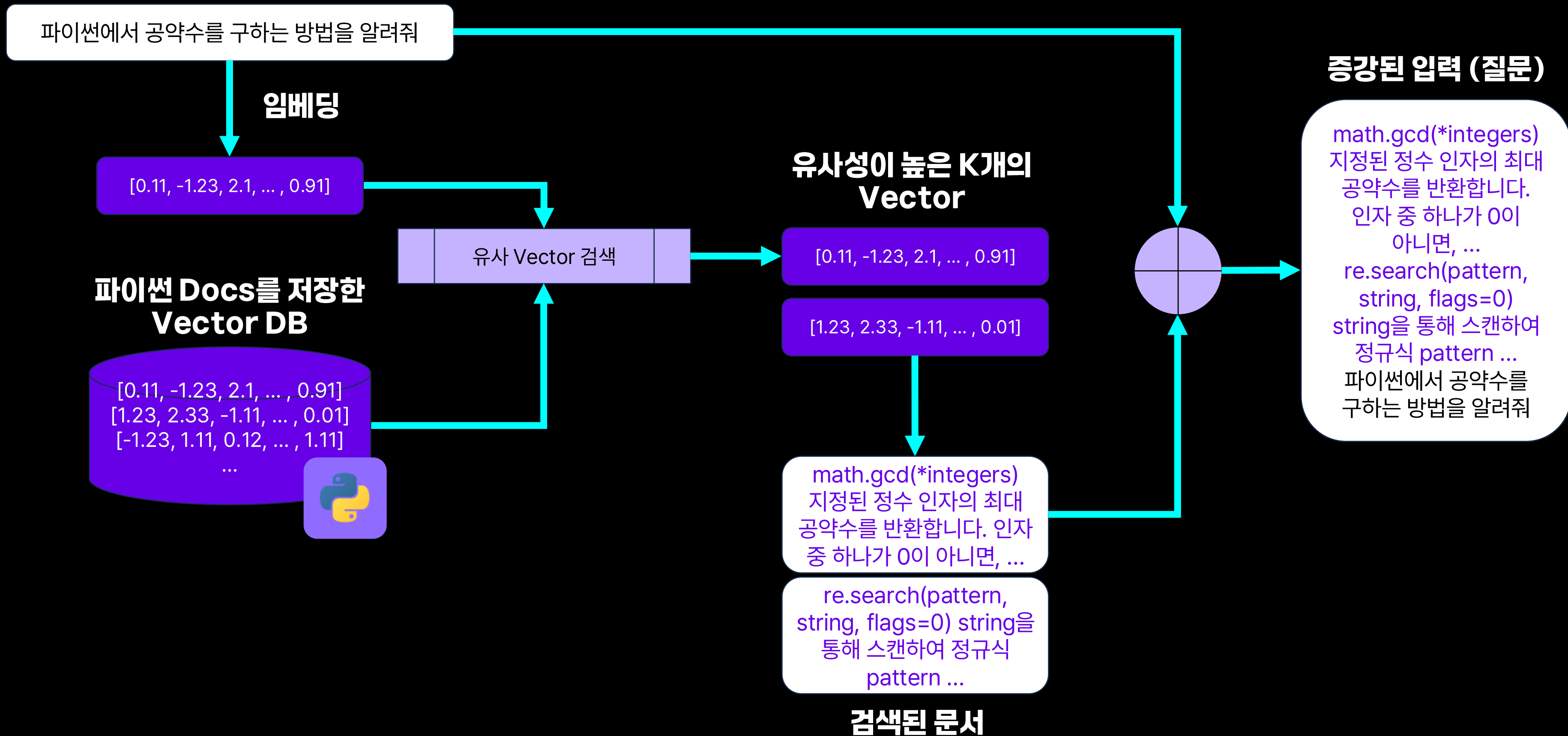


Retrieval (검색) 예시

Rerank와 Modular RAG

Adaptive RAG Chatbot

1. RAG (Retrieval-Augmented Generation)



4. Self-Reflective RAG



Naïve RAG의 한계점

- Naïve RAG 한계점

1. 데이터 일관성 문제: 다양한 출처의 데이터를 결합할 때 일관성을 확보하기 어렵다.
2. 정확도와 신뢰성: 검색한 정보의 정확도와 신뢰성을 검증하기 어렵다.
3. 데이터 최신성: DB에 있는 정보가 최신 정보라는 보장이 없다.
4. 정형 데이터: 검색한 정보를 프롬프트에 입력함에 따라, 통계적 데이터 분석 기법을 적용할 수 없다.



Naïve RAG의 한계점

1. 데이터 일관성 문제: 다양한 출처의 데이터를 결합할 때 일관성을 확보하기 어렵다.

RAG 구현 방법에 대해 알려줘



Naïve RAG의 한계점

1. 데이터 일관성 문제: 다양한 출처의 데이터를 결합할 때 일관성을 확보하기 어렵다.

**LangChain 기반
RAG 구현 방법 문서**

RAG 구현 방법에 대해 알려줘

LangChain의 DocumentParser를
활용하여 PDF 문서를 파싱하고...

LlamaIndex의 VectorStoreIndex를
활용해서 문서를 벡터화하고...

RAG 구현 방법에 대해 알려줘

**LlamaIndex 기반
RAG 구현 방법 문서**

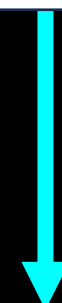
여러 데이터가 비슷한 내용을 가지고 있을
경우, 이를 적절하게 활용하기 어려움



Naïve RAG의 한계점

1. 데이터 일관성 문제: 다양한 출처의 데이터를 결합할 때 일관성을 확보하기 어렵다.

LangChain 기반 RAG 구현 방법에 대해 알려줘



LangChain 기반
RAG 구현 방법 문서

LangChain의 DocumentParser를
활용하여 PDF 문서를 파싱하고...

RAG 구현 방법에 대해 알려줘

이 경우 지시사항을 구체화하여 데이터
일관성 문제를 어느 정도 완화할 수 있음



Naïve RAG의 한계점

1. 데이터 일관성 문제: 다양한 출처의 데이터를 결합할 때 일관성을 확보하기 어렵다.

LangChain 에서 Ollama 쓰는 법을 알려줘



LangChain v0.1
API 문서

```
Import langchain_community.llms.ollama.ChatOllama  
선언합니다...
```

```
Import langchain_ollama.chat_models.ChatOllama  
선언합니다...
```

LangChain 에서 Ollama 쓰는 법을 알려줘

LangChain v0.3
API 문서

그러나, 지시사항을 구체화 하더라도 이
문제를 완전히 해결할 수 있는 것은 아님



Naïve RAG의 한계점

1. 데이터 일관성 문제: 다양한 출처의 데이터를 결합할 때 일관성을 확보하기 어렵다.

OpenAI API 기반
RAG 구현 방법 문서

RAG 구현 방법에 대해 알려줘

ChatOpenAI Class Object를 생성하고,
Object 키를 생성합니다...

Ollama 인스턴스를 활성화 하고,
ChatOllama Object를 생성하고...

RAG 구현 방법에 대해 알려줘

Ollama 기반
RAG 구현 방법 문서

또한, 여러 데이터가 서로 상충되는 경우
답변의 퀄리티나 일관성을 확보하기 어려움



Naïve RAG의 한계점

2. 정확도와 신뢰성: 검색한 정보의 정확도와 신뢰성을 검증하기 어렵다.

한국 표준어 기준으로, “짜장면” 과 “자장면” 중 뭐가 올바른 표현이야?



Naïve RAG의 한계점

2. 정확도와 신뢰성: 검색한 정보의 정확도와 신뢰성을 검증하기 어렵다.

한국 표준어 기준으로, "짜장면" 과 "자장면" 중 뭐가 올바른 표현이야?

2011년 이전 맞춤법 관련 문의 발체

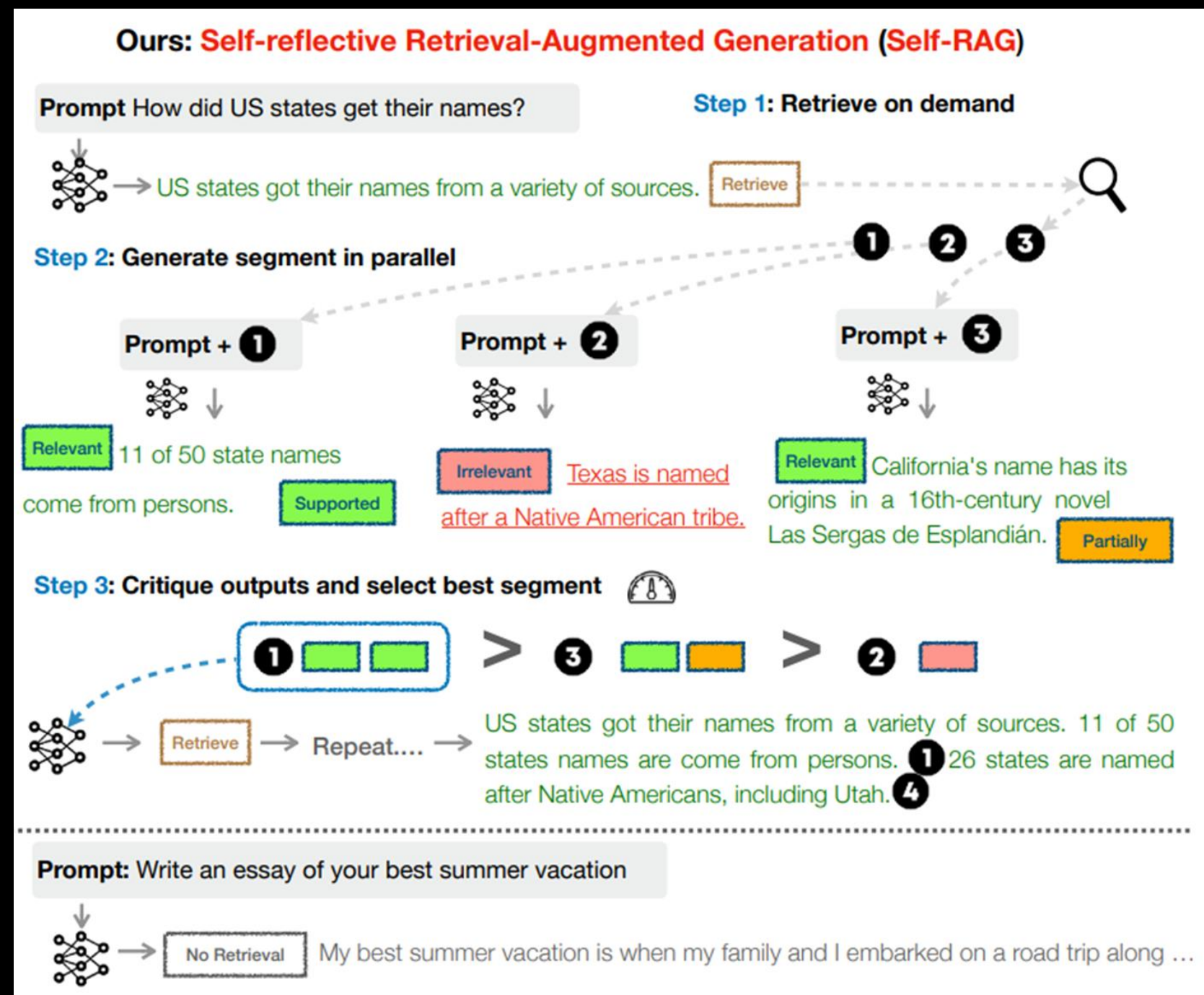
외래어표기법에 따라 "자장면" 이 올바른
표기법입니다.

한국 표준어 기준으로, "짜장면" 과 "자장면"
중 뭐가 올바른 표현이야?

2011년 이후 맞춤법 관련 문의 발체

외래어표기법에 따라 "자장면" 이 올바른
표기법이었으나, 2011년 외래어 표기
개정으로 복수표준어로 인정되었습니다.
한국 표준어 기준으로, "짜장면" 과 "자장면"
중 뭐가 올바른 표현이야?

가지고 있는 정보가 최신
정보인지 확인하기 어려움



LLM의 추론 능력을 응용하여 (1) 검색 필요 여부, (2) 검색한 문서의 연관성, (3) 검색한 문서 기반 답변과 질문 간의 연관성 및 유용성을 스스로 검증하는 RAG 시스템을 구성한다면?

Algorithm 1 SELF-RAG Inference

Require: Generator LM $\mathcal{M}$, Retriever $\mathcal{R}$, Large-scale passage collections $\{d_1, \dots, d_N\}$

- 1: **Input:** input prompt x and preceding generation $y_{<t}$, **Output:** next output segment y_t
- 2: $\mathcal{M}$ predicts **Retrieve** given $(x, y_{<t})$
- 3: **if** **Retrieve** == Yes **then**
- 4: Retrieve relevant text passages $\mathbf{D}$ using $\mathcal{R}$ given (x, y_{t-1}) ▷ Retrieve
- 5: $\mathcal{M}$ predicts **ISREL** given x, d and y_t given $x, d, y_{<t}$ for each $d \in \mathbf{D}$ ▷ Generate
- 6: $\mathcal{M}$ predicts **ISSUP** and **ISUSE** given x, y_t, d for each $d \in \mathbf{D}$ ▷ Critique
- 7: Rank y_t based on **ISREL**, **ISSUP**, **ISUSE** ▷ Detailed in Section 3.3
- 8: **else if** **Retrieve** == No **then**
- 9: $\mathcal{M}_{gen}$ predicts y_t given x ▷ Generate
- 10: $\mathcal{M}_{gen}$ predicts **ISUSE** given x, y_t ▷ Critique

1. LLM을 통해 사용자가 입력한 질문을 바탕으로 검색 필요성을 **추론, 판단**한다.
2. 검색이 필요하지 않다고 판단할 경우 별도의 검색 증강 과정을 거치지 않고 바로 답변을 생성한다.
3. 검색이 필요하다 판단한 경우, 단순 RAG 처럼 연관이 있는 K개의 문서를 검색한다.
 1. (1) 이때, 각 문서의 연관성을 LLM을 통해 **평가**한다.
4. 검색한 문서를 각각 질문에 적용하여 증강된 질문 K개를 생성한다.
5. 증강된 K개의 질문을 각각 LLM에 입력하여 답변 K개를 생성한다.
 1. 이때, 각 답변의 (2) 지원 정도 (supportive)와 (3) 유용성을 **평가**한다.
6. (1), (2), (3) 평가 결과를 바탕으로 **가장 적합한 답변을 선택하여 반환**한다.

Algorithm 1 SELF-RAG Inference

Require: Generator LM $\mathcal{M}$, Retriever $\mathcal{R}$, Large-scale passage collections $\{d_1, \dots, d_N\}$
1: **Input:** input prompt x and preceding generation $y_{<t}$, **Output:** next output segment y_t
2: $\mathcal{M}$ predicts **Retrieve** given $(x, y_{<t})$
3: **if** **Retrieve** == Yes **then**
4: Retrieve relevant text passages $\mathbf{D}$ using $\mathcal{R}$ given (x, y_{t-1}) ▷ **Retrieve**
5: $\mathcal{M}$ predicts **ISREL** given x, d and y_t given $x, d, y_{<t}$ for each $d \in \mathbf{D}$ ▷ **Generate**
6: $\mathcal{M}$ predicts **ISSUP** and **ISUSE** given x, y_t, d for each $d \in \mathbf{D}$ ▷ **Critique**
7: Rank y_t based on **ISREL**, **ISSUP**, **ISUSE** ▷ Detailed in Section 3.3
8: **else if** **Retrieve** == No **then**
9: $\mathcal{M}_{gen}$ predicts y_t given x ▷ **Generate**
10: $\mathcal{M}_{gen}$ predicts **ISUSE** given x, y_t ▷ **Critique**



- RAGAS(Retrieval-Augmented Generation Answer Scoring)
 - RAG 시스템이 생성한 답변의 품질을 평가하기 위한 평가 체계
 - 외부 문서나 데이터베이스와 같은 근거 자료를 활용하여 답변을 생성하기 때문에
 - 단순히 문법이나 유창성만 평가하는 기존 지표로는 부족
 - 답변이 근거 자료와 얼마나 일치하며 신뢰할 수 있는지를 평가
- 평가 대상
 - Retrieval(Context)
 - LLM output(Answer)



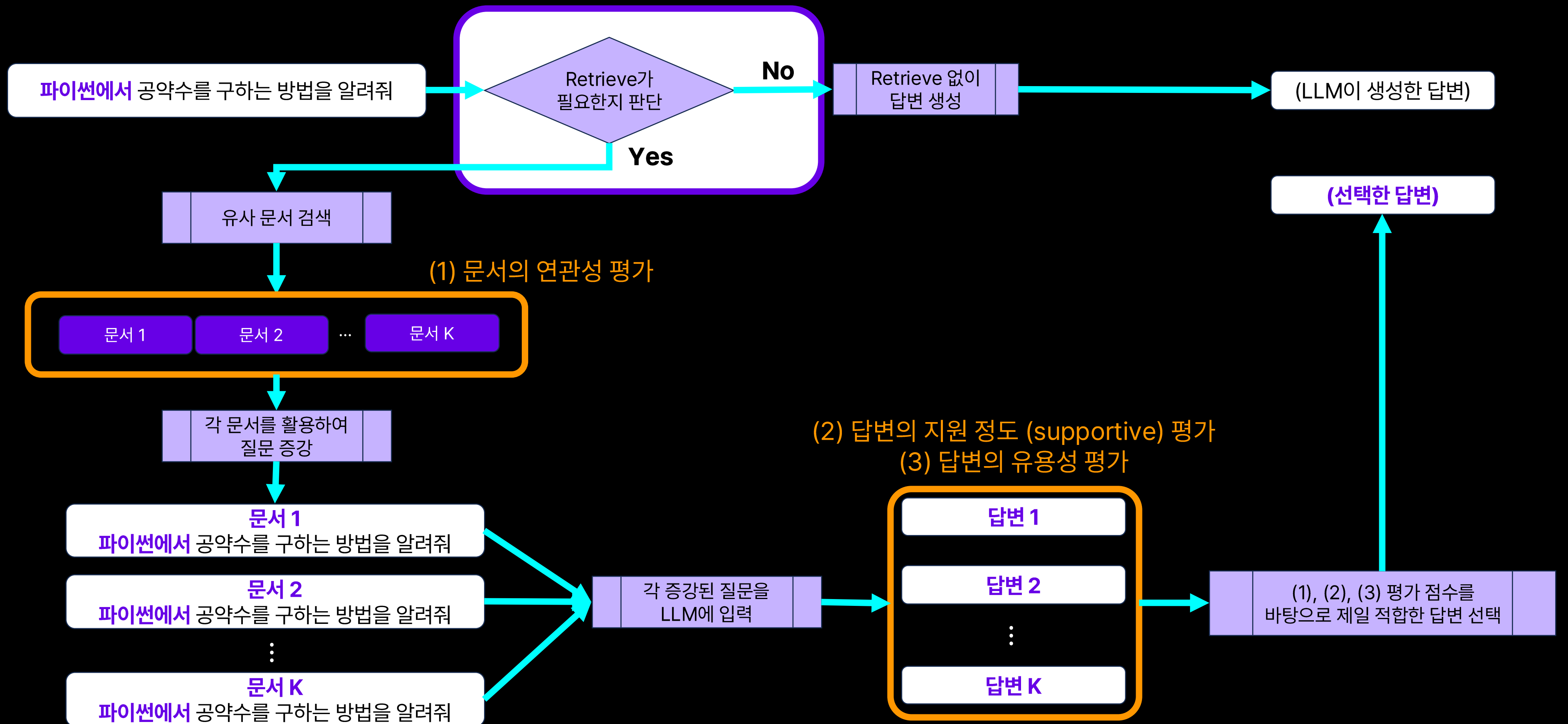
- Retrieval(Context) 평가
 - Context Recall
 - 회수된 Chunk 중 LLM이 생성한 답변과 일치하는 비율
 - Context Precision
 - Context가 얼마나 잘 회수되었는지를 평가
 - Context 중 Query에 부합하는 중요 내용들이 Top-K에 얼마나 포진되었는지를 평가

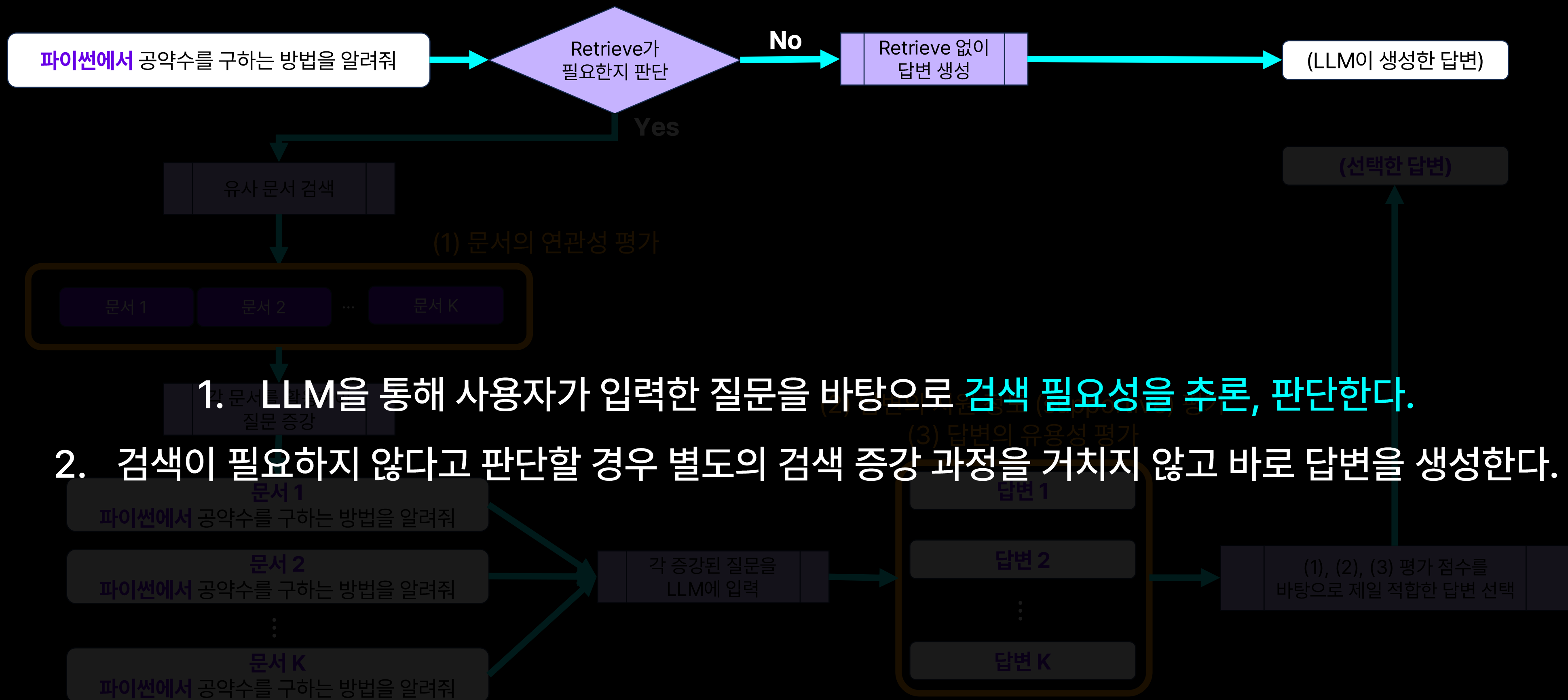


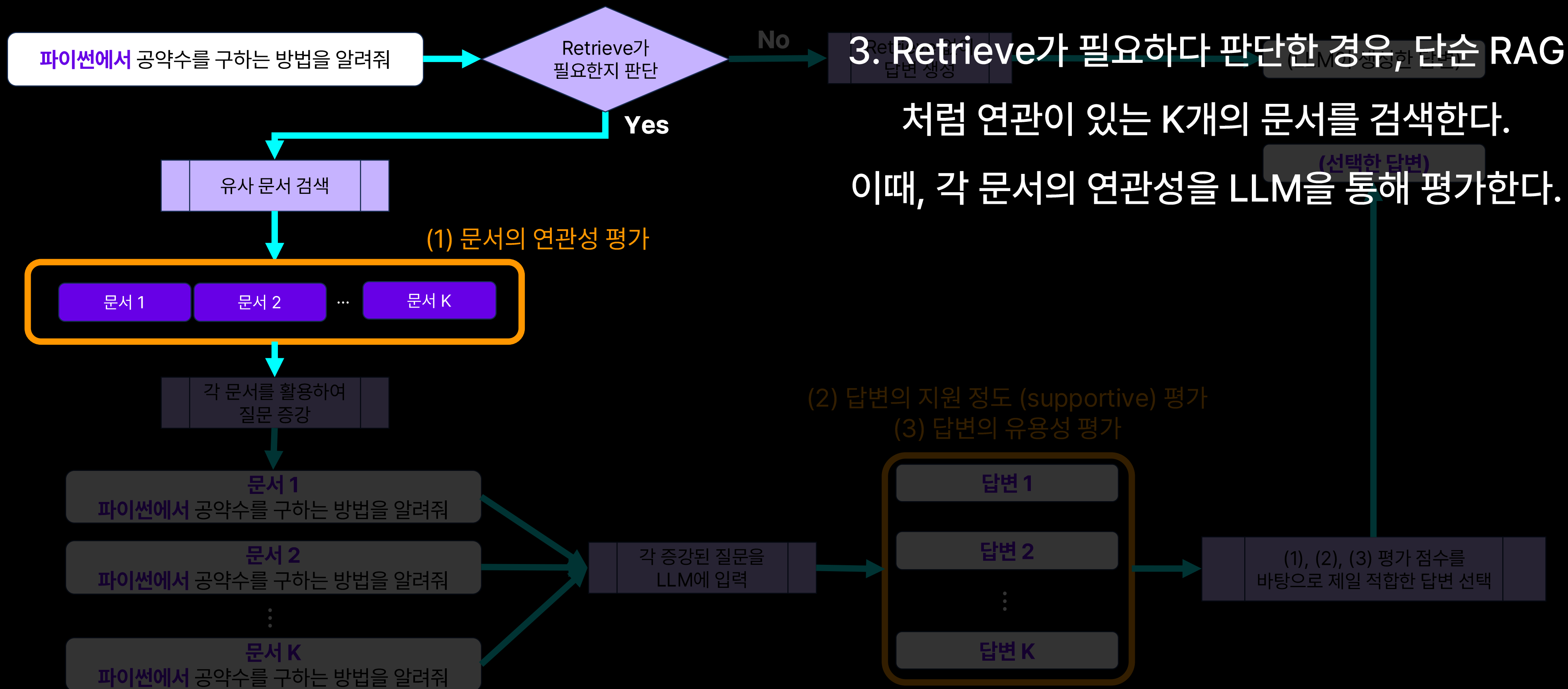
- LLM output(Answer) 평가
 - Relevancy(Supportiveness)
 - 생성된 답변이 제공된 근거 자료의 내용을 얼마나 잘 반영하는지를 평가
 - 답변이 주어진 질문에 얼마나 적합한지를 평가
 - Faithfulness
 - 답변에 포함된 정보가 실제 자료에 기반하고 있는지를 확인
 - 근거 자료의 정보를 왜곡하거나 잘못 해석하지 않고, 사실 그대로 전달하는지를 측정

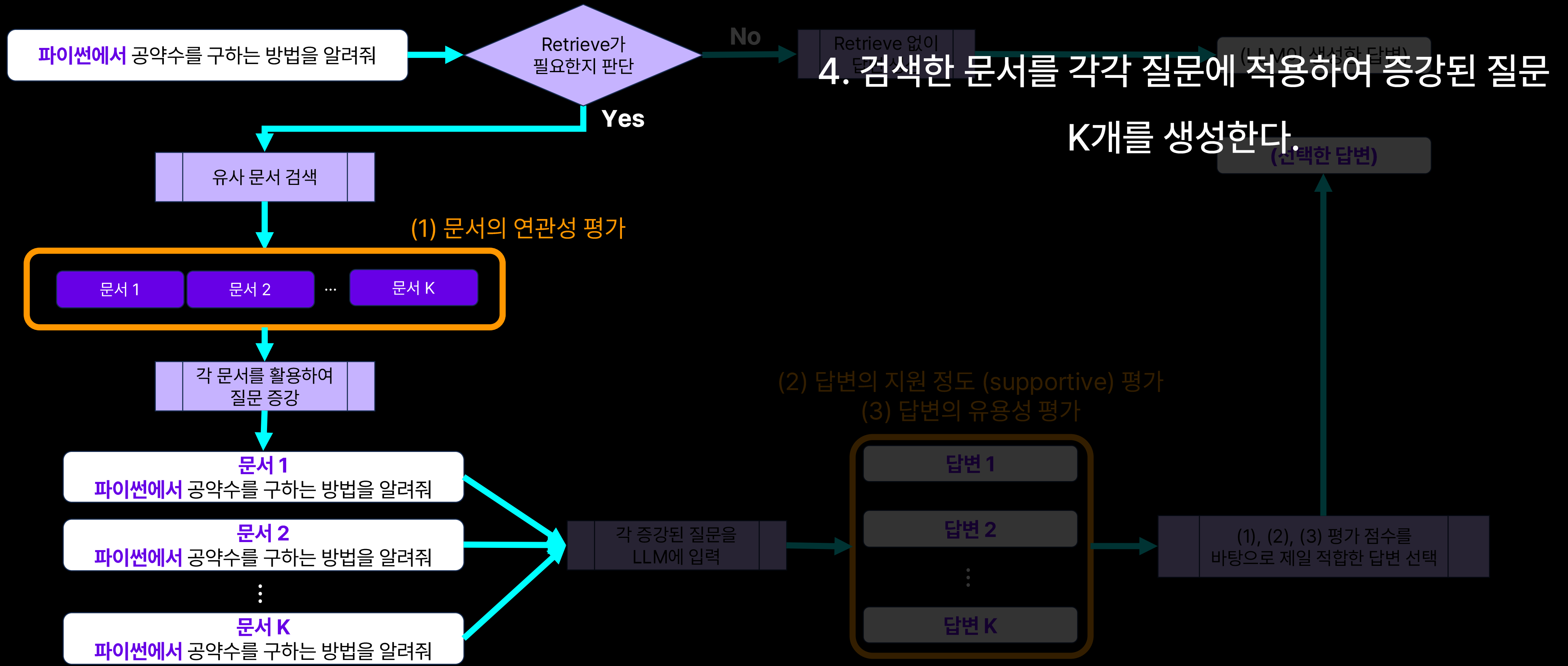


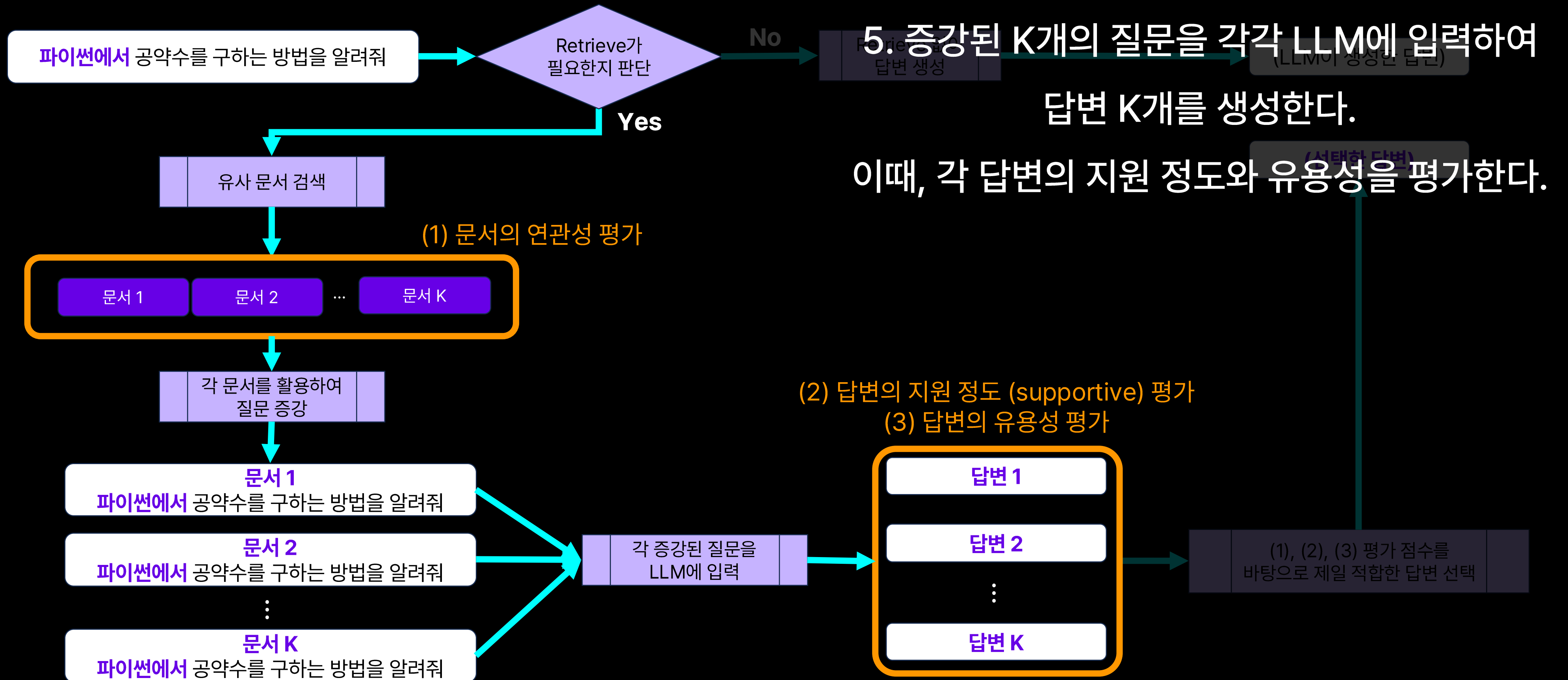
Self-RAG 알고리즘

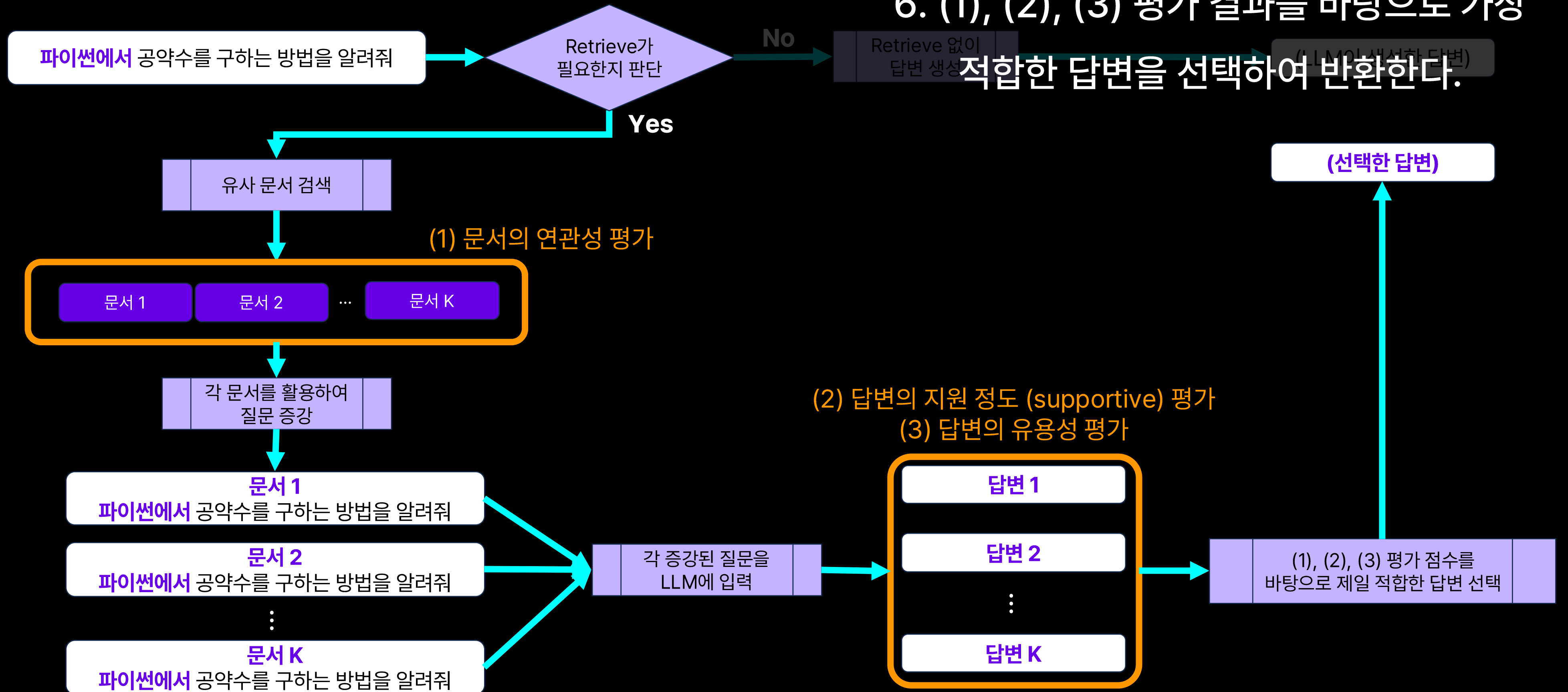




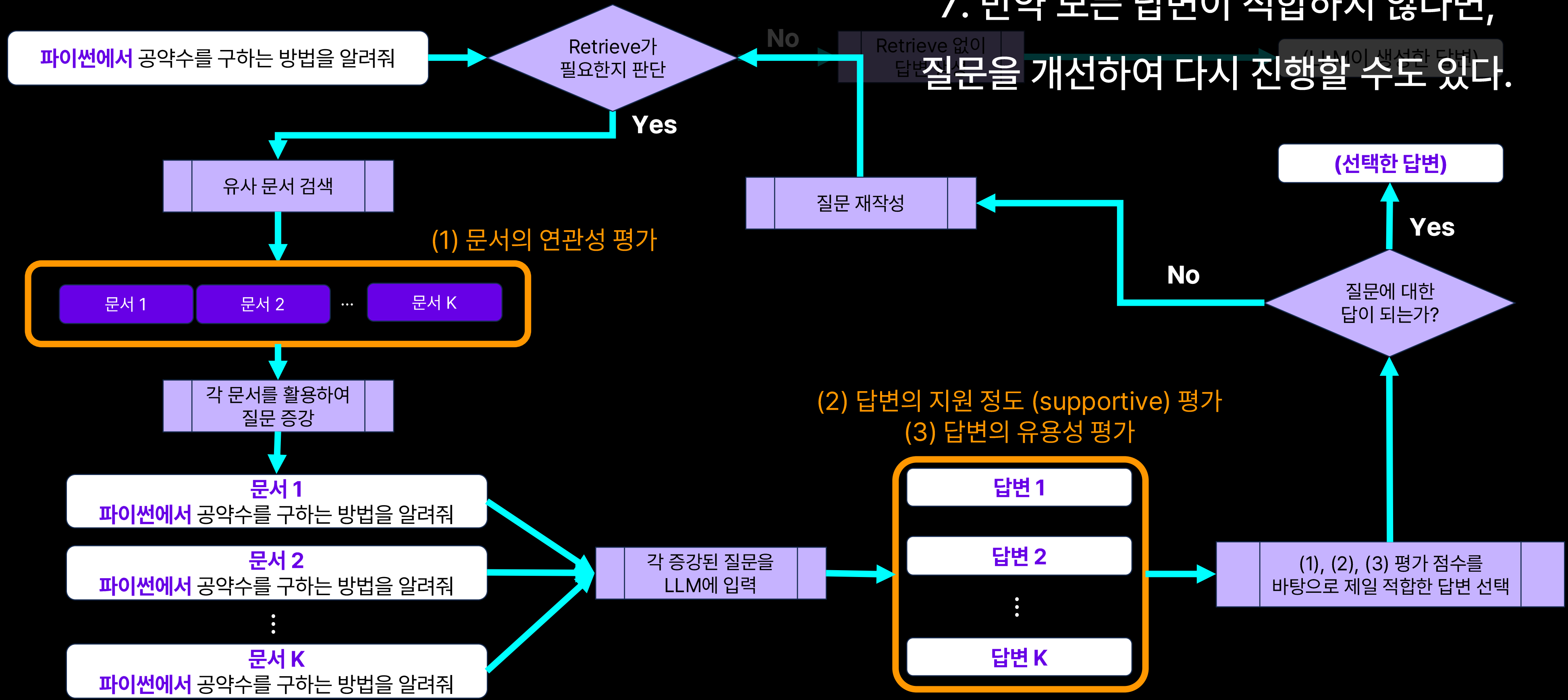








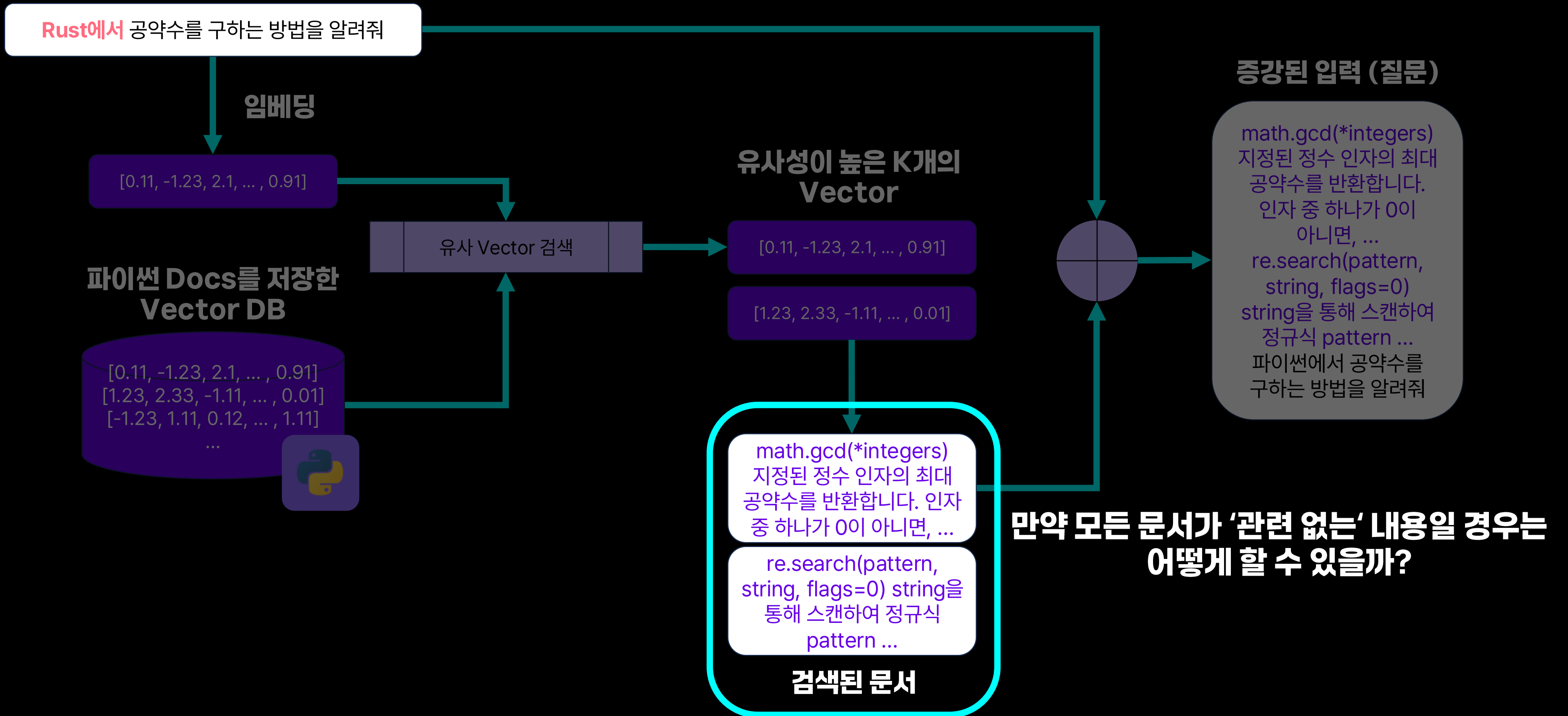
7. 만약 모든 답변이 적합하지 않다면, 질문을 개선하여 다시 진행할 수도 있다.



5. Adaptive RAG



Retrieval 예시





단순 RAG의 한계점 2

- Naïve RAG 한계점

1. 데이터 일관성 문제: 다양한 출처의 데이터를 결합할 때 일관성을 확보하기 어렵다.
2. 정확도와 신뢰성: 검색한 정보의 정확도와 신뢰성을 검증하기 어렵다.
3. 데이터 최신성: DB에 있는 정보가 최신 정보라는 보장이 없다.
4. 정형 데이터: 검색한 정보를 프롬프트에 입력함에 따라, 통계적 데이터 분석 기법을 적용할 수 없다.

→ Adaptive RAG (적응형 RAG) 를 활용할 수 있다



단순 RAG의 한계점

3. 데이터 최신성: DB에 있는 정보가 최신 정보라는 보장이 없다.

한국 표준어 기준으로, “짜장면” 과 “자장면” 중 뭐가 올바른 표현이야?



단순 RAG의 한계점

3. 데이터 최신성: DB에 있는 정보가 최신 정보라는 보장이 없다.

한국 표준어 기준으로, "짜장면" 과 "자장면" 중 뭐가 올바른 표현이야?

2011년 **이전** 맞춤법
관련 문의 발체

외래어표기법에 따라 "자장면" 이 올바른
표기법입니다.

한국 표준어 기준으로, "짜장면" 과 "자장면"
중 뭐가 올바른 표현이야?

2011년 **이후** 맞춤법
관련 문의 발체

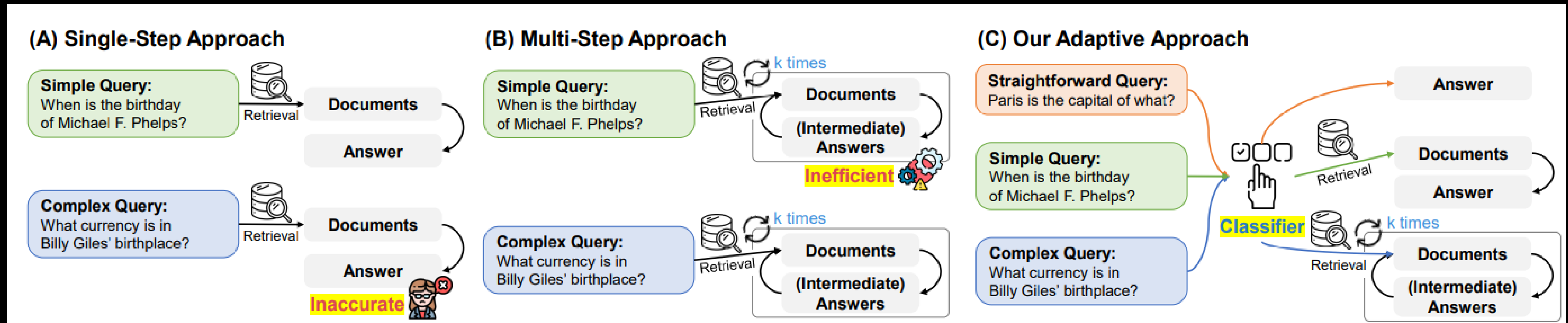
외래어표기법에 따라 "자장면" 이 올바른
표기법이었으나, 2011년 외래어 표기
개정으로 복수표준어로 인정되었습니다.
한국 표준어 기준으로, "짜장면" 과 "자장면"
중 뭐가 올바른 표현이야?

**가지고 있는 정보가 최신
정보인지 확인하기 어렵다.**



Adaptive RAG

- 기존의 RAG 시스템에 **적응적인(Adaptive)** 요소를 도입한 개념
 - 입력되는 질문이나 문맥에 따라 검색 및 생성 과정을 동적으로 조정
 - 질문의 복잡도나 도메인에 따라 검색할 문서의 수, 순위, 또는 검색 방법을 유동적으로 변경
 - 더 관련성 높은 정보를 선택하고
 - 불필요한 정보가 답변에 포함되는 것을 줄이기 위하여
 - 생성된 답변에 대한 평가나 피드백을 이용하여 검색 결과나 생성 전략을 개선
- Agentic한 요소라 볼 수 있음



별도의 분류기를 활용하여 질문의 종류에 따라 (1) 단순 답변, (2) 단일 검색을 통한 RAG 답변, (3) 여러 번의 검색 및 추론을 통한 RAG 답변을 생성하는 적응형 RAG 시스템

입력된 질문 (Query) 에 적합한 형태로 적응 (Adapt) 하여 답변한다고 하여 Adaptive RAG 라고 칭한다.



Adaptive RAG

